

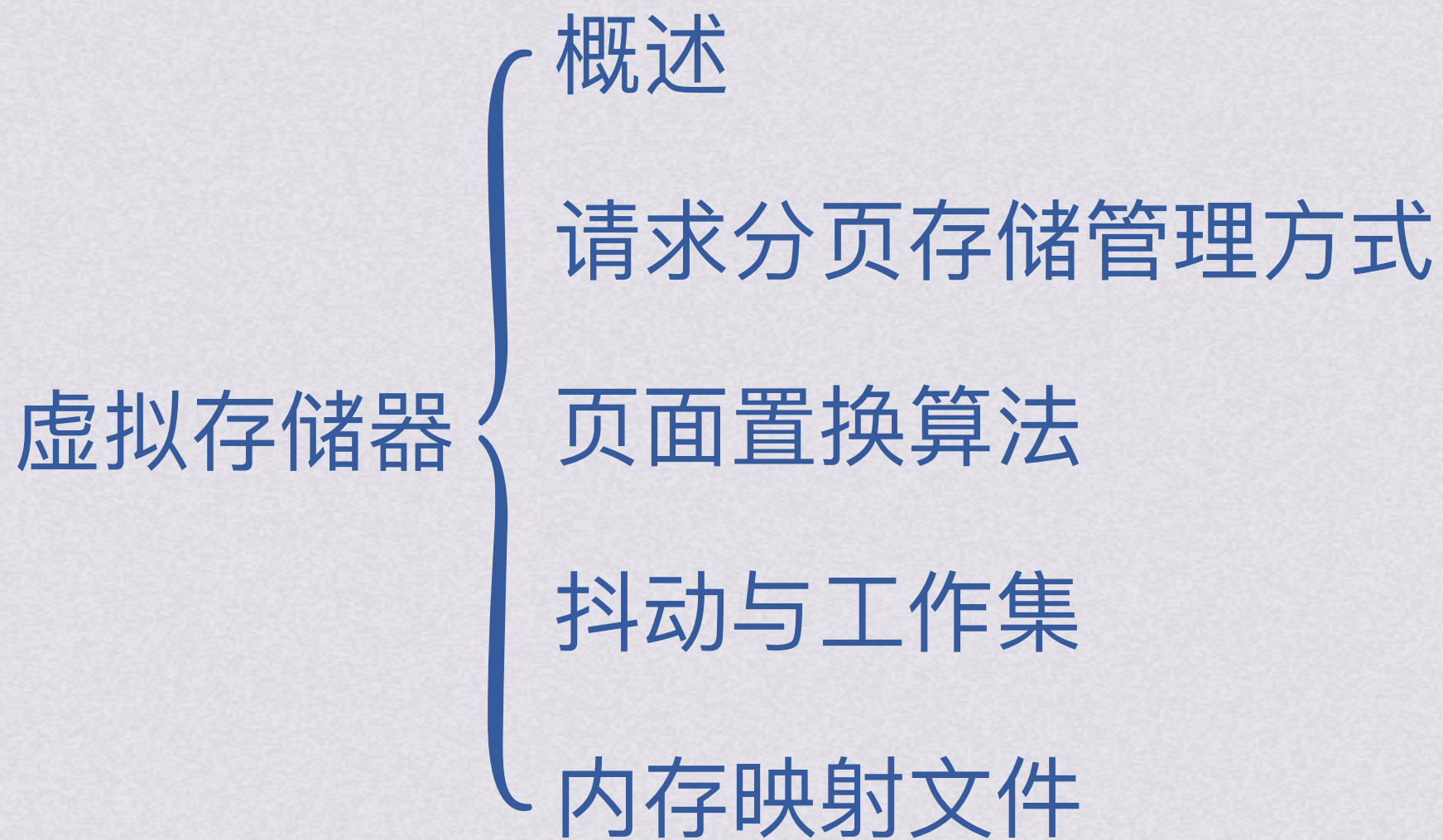
bok25

**基
礎**

課、操作系統



虚拟存储器

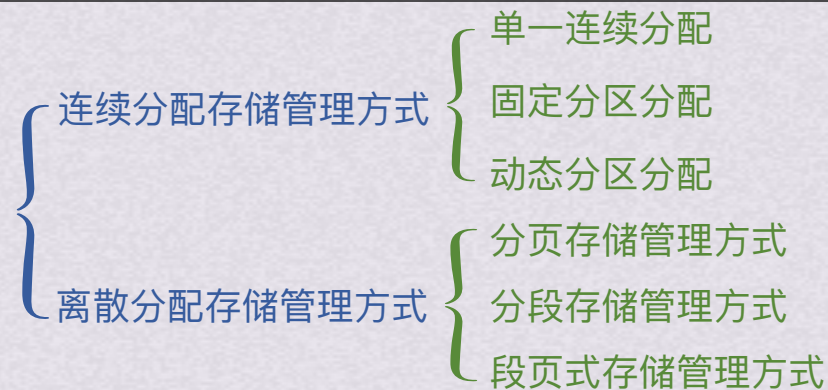




概述



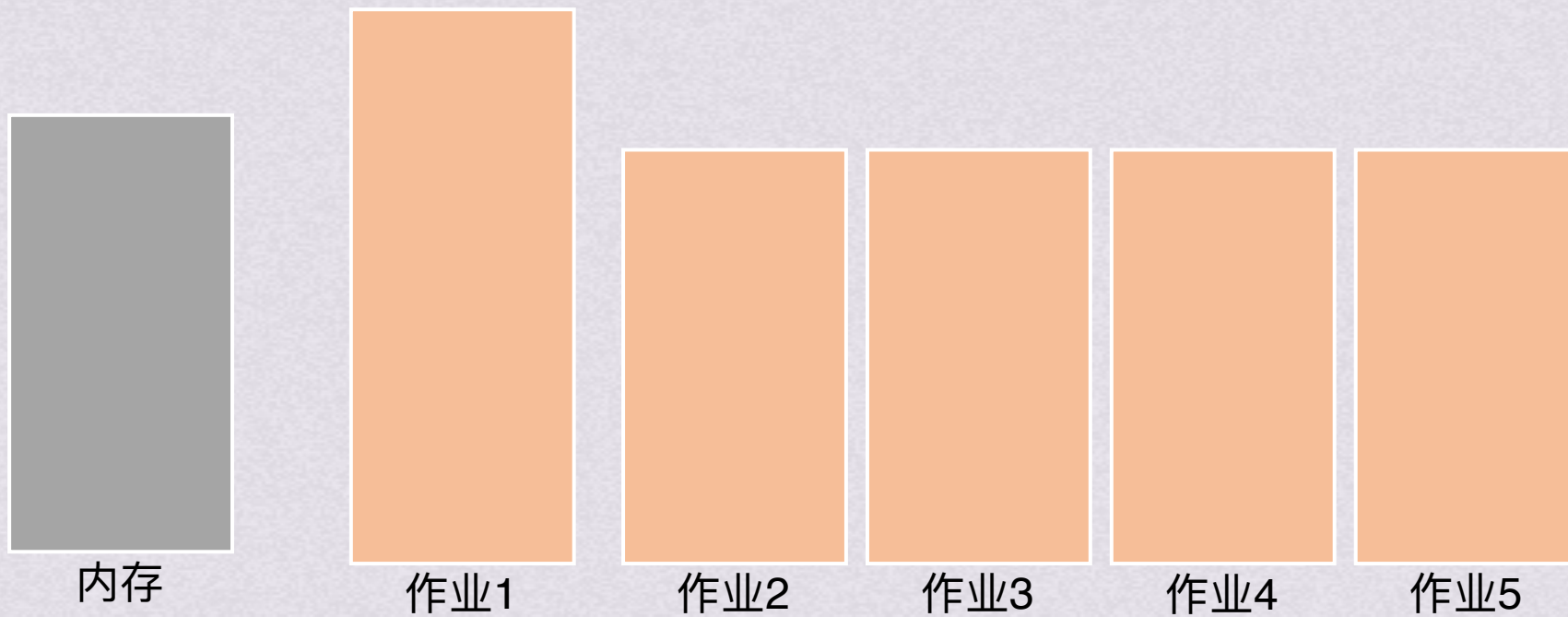
之前介绍的所有存储器管理方式



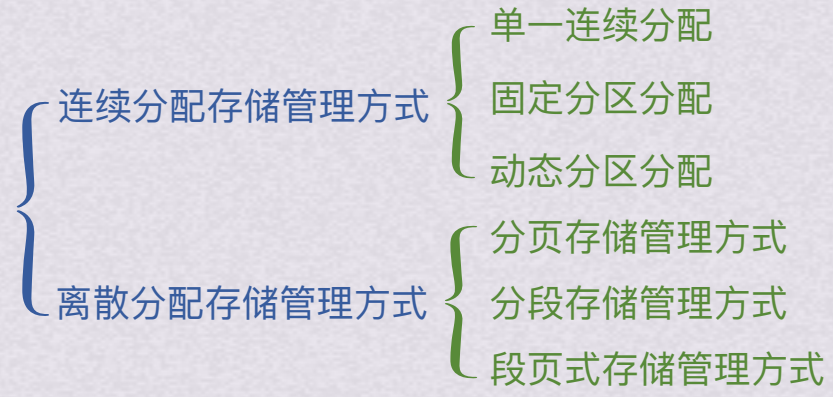
都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

这会导致出现下面的情况：

- 1.有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待



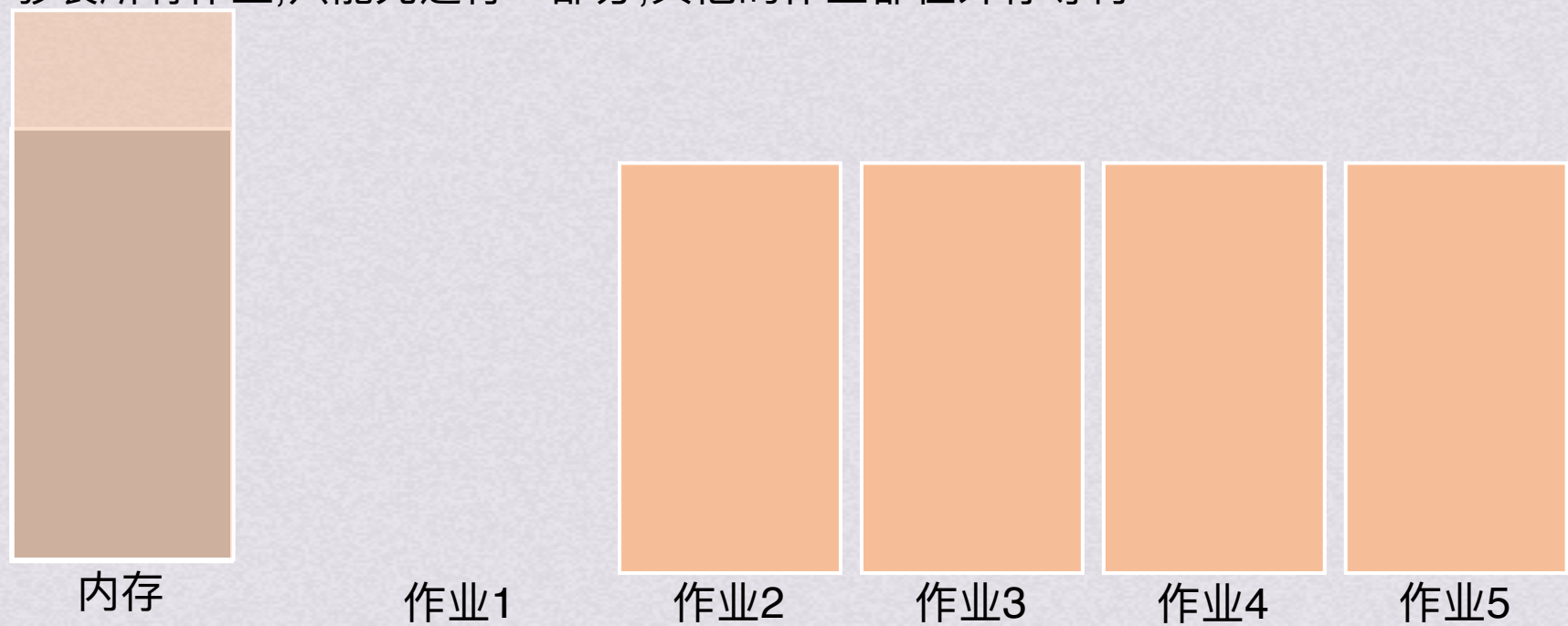
之前介绍的所有存储器管理方式



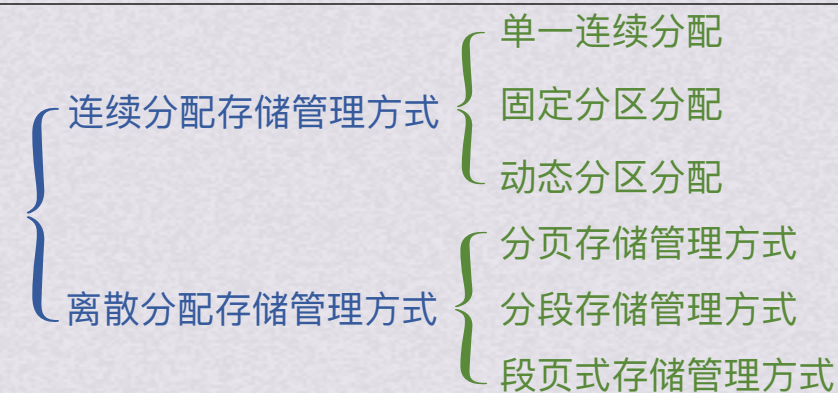
都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

这会导致出现下面的情况：

- 1.有的游戏作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待



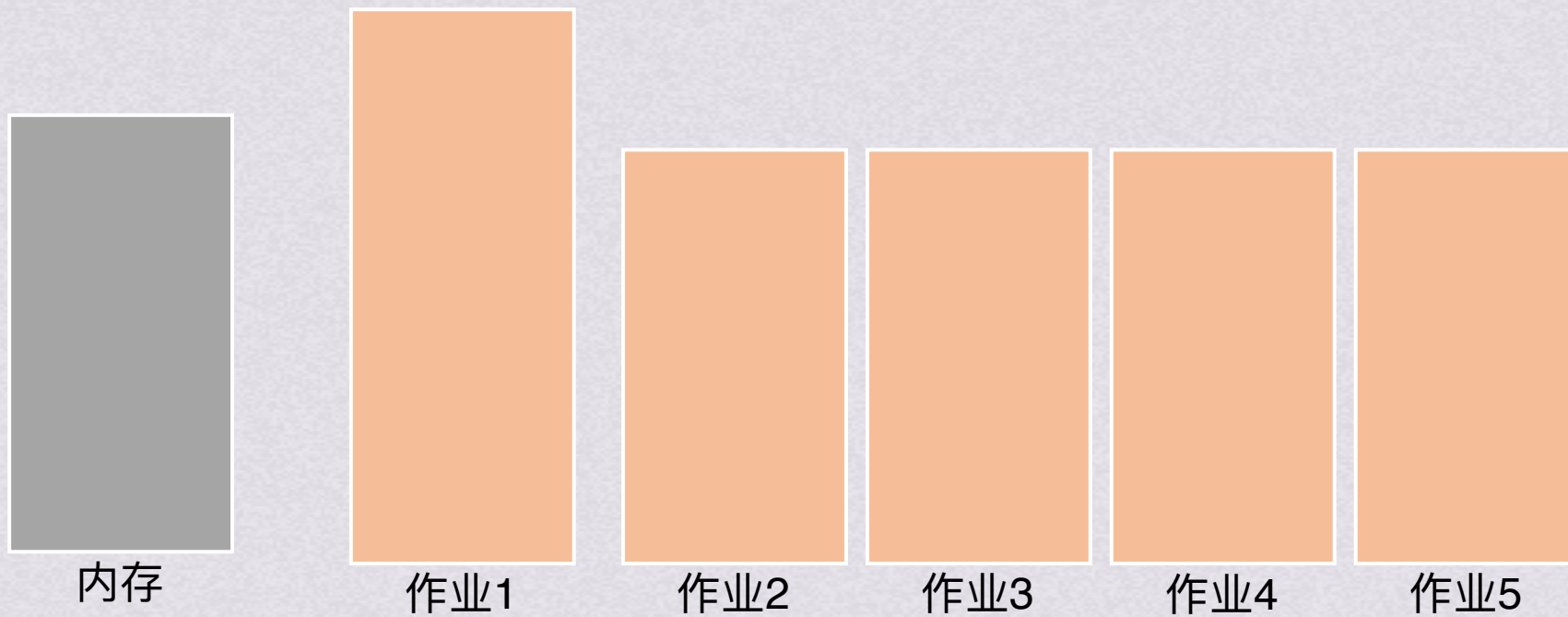
之前介绍的所有存储器管理方式



都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

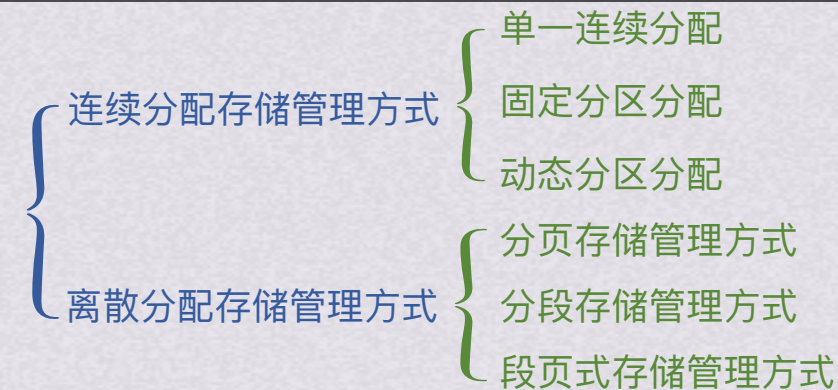
这会导致出现下面的情况：

- 1.有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待





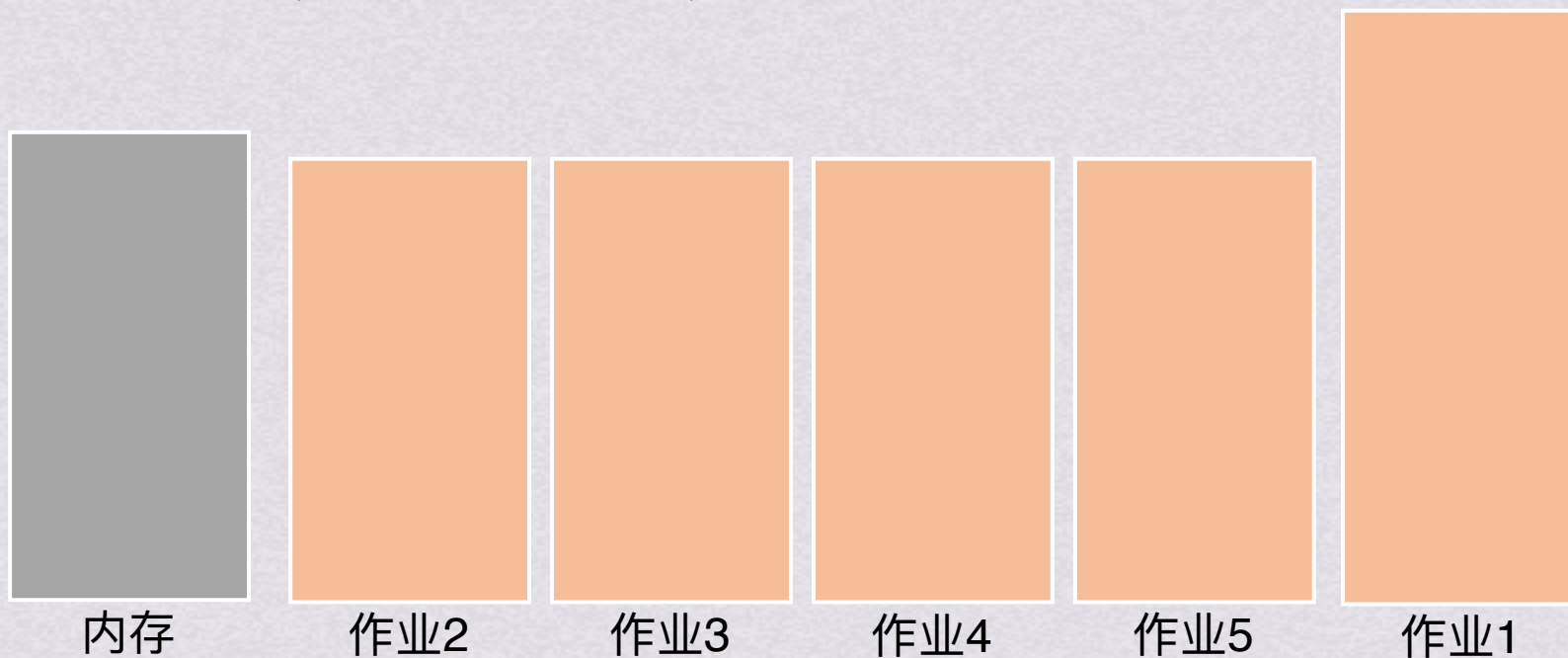
之前介绍的所有存储器管理方式



都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

这会导致出现下面的情况：

- 1.有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待



虚拟存储器概述



之前介绍的所有存储器管理方式

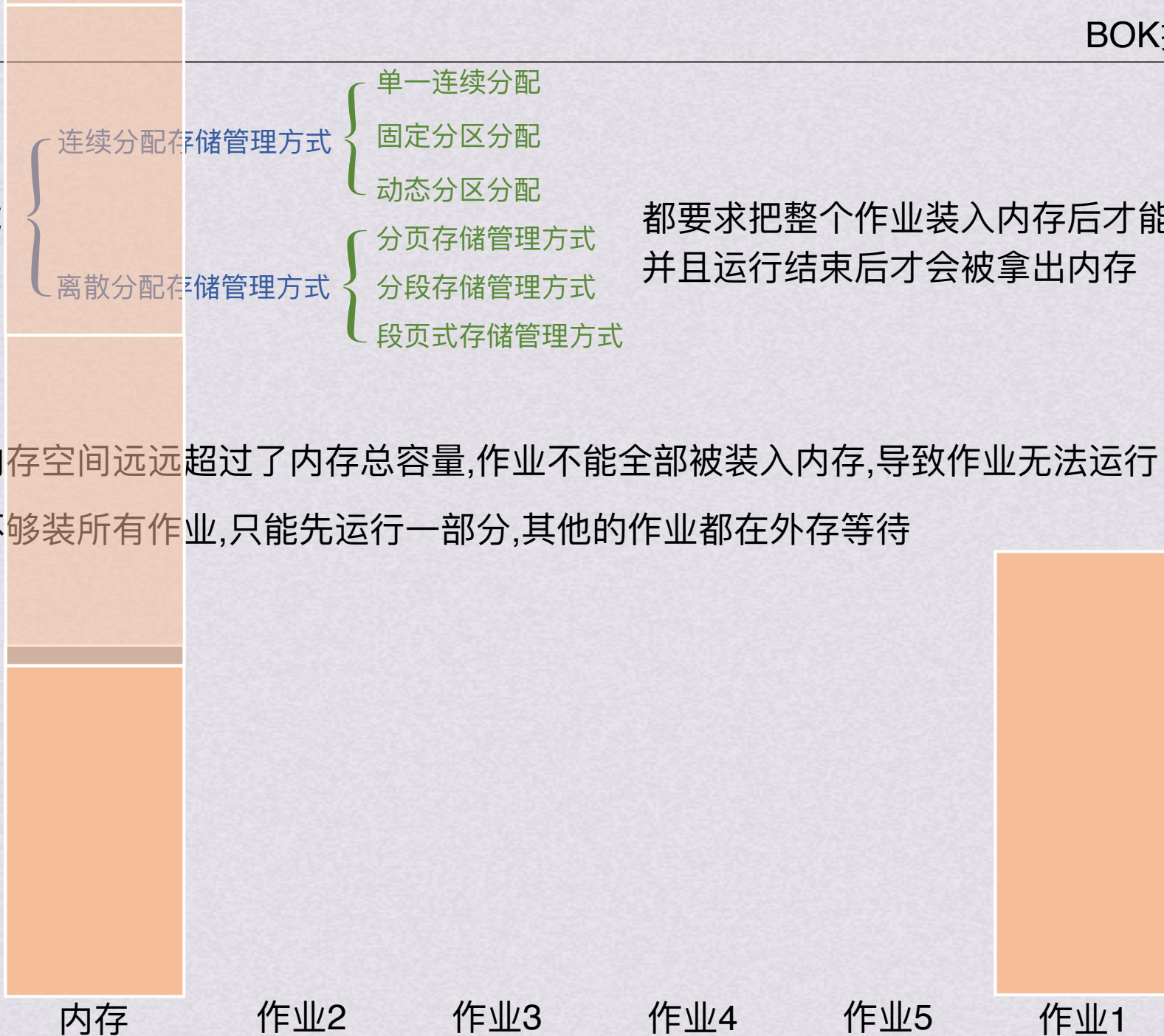
连续分配存储管理方式
离散分配存储管理方式

- 单一连续分配
- 固定分区分配
- 动态分区分配
- 分页存储管理方式
- 分段存储管理方式
- 段页式存储管理方式

都要求把整个作业装入内存后才能运行
并且运行结束后才会被拿出内存

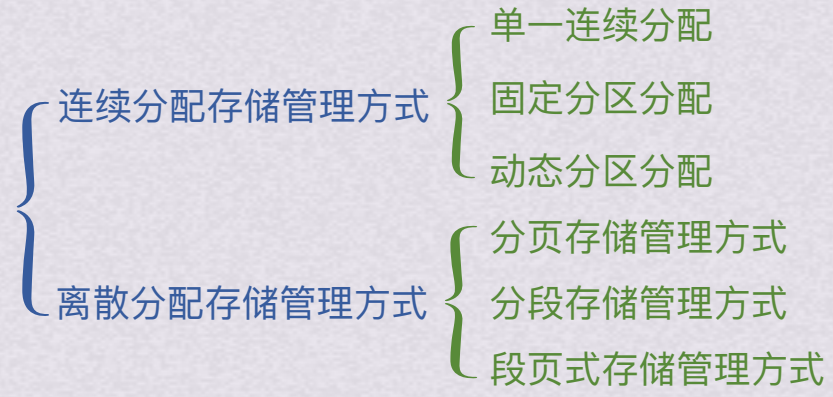
这会导致出现下面的情况：

- 1.有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待



请求分页存储管理方式
页面置换算法
抖动与工作集
内存映射文件

之前介绍的所有存储器管理方式



都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

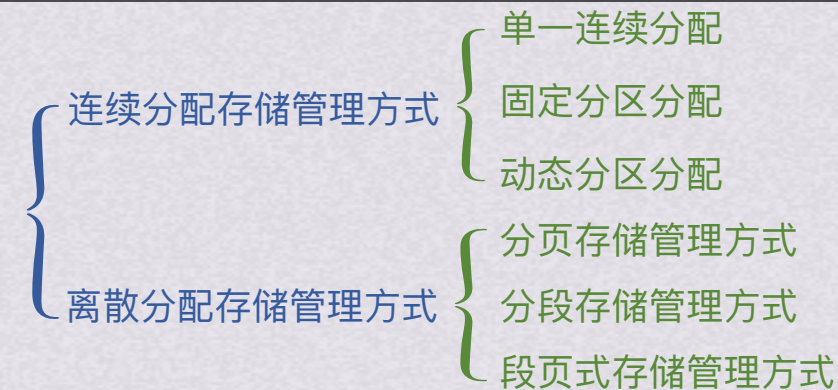
这会导致出现下面的情况：

- 1. 有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2. 有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待





之前介绍的所有存储器管理方式



都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

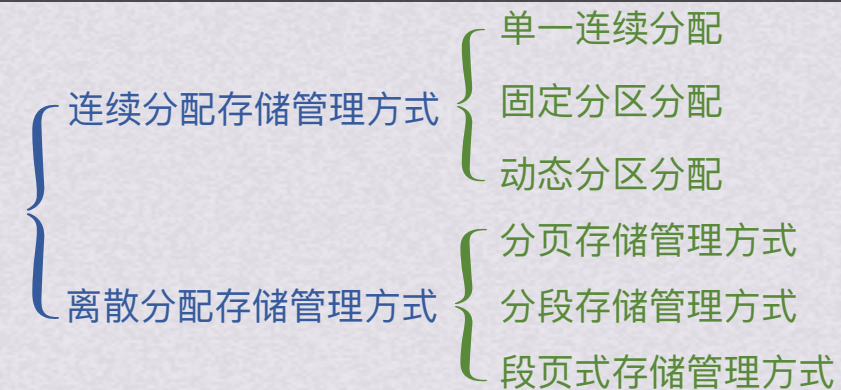
这会导致出现下面的情况：

- 1.有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待

解决方法：1.加钱买更大的内存
2.从逻辑上扩充内存容量



之前介绍的所有存储器管理方式



都要求把整个作业装入内存后才能运行并且运行结束后才会被拿出内存

这会导致出现下面的情况：

- 1.有的~~游戏~~作业巨大,所要求的内存空间远远超过了内存总容量,作业不能全部被装入内存,导致作业无法运行
- 2.有大量作业要运行,但是内存不够装所有作业,只能先运行一部分,其他的作业都在外存等待

1.加钱买更大的内存

解决方法：2.从逻辑上扩充内存容量

引入虚拟存储技术



概述

引入虚拟存储技术

局部性原理

之前的学习中得知,程序具有局部性,也就是在一较短时间内,程序的执行仅局限于某个部分,程序所访问的存储空间也局限于某个区域

局限性又表现在两个方面:

1.时间局限性: 如果程序中某条指令被执行,那么不久后该指令可能再次执行; 如果某数据被访问过,不久之后该数据可能再次被访问

产生原因: 程序中存在大量的循环操作

2.空间局限性: 一旦程序访问了某个存储单元,不久之后,其附近的存储单元也将被访问

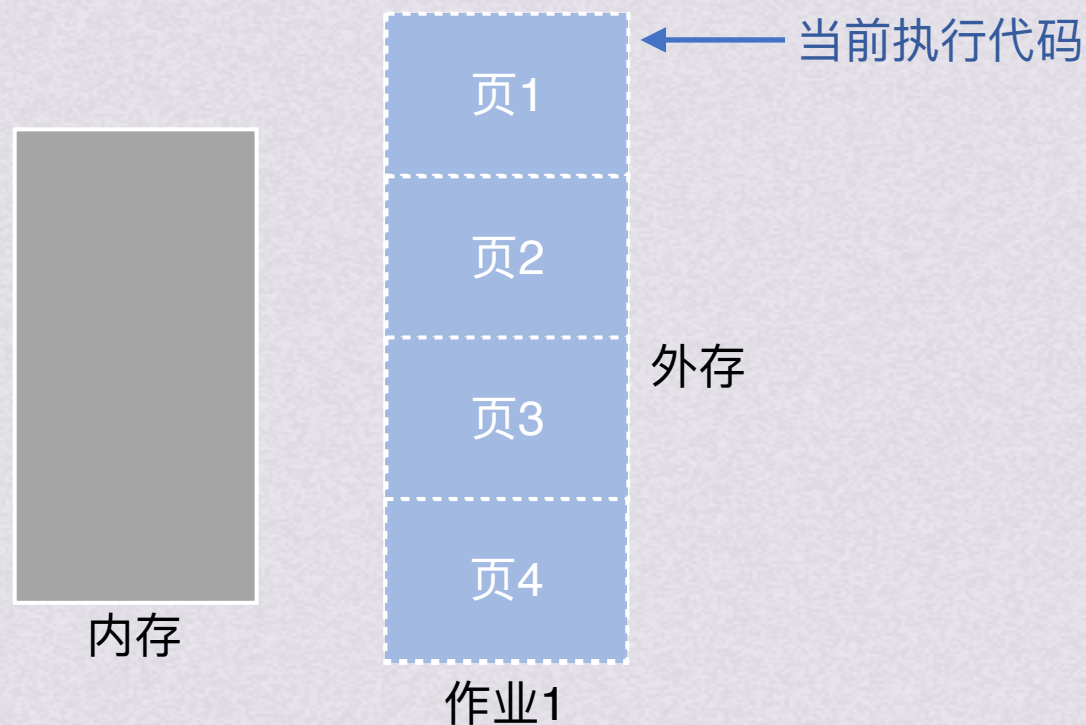
产生原因: 程序会顺序执行



概述

引入虚拟存储技术

所以根据局部性原理可知,程序运行之前没有必要全部装入内存,只需要把当前要运行的少数页面/段先装入内存便可运行,其余部分留在盘上。如果程序运行时发现需要访问的页(段)还没调入内存(称为缺页或者缺段),于是发出缺页(段)中断请求,OS再利用请求调页(段)功能将它们调入内存,方便进程继续执行下去





概述

引入虚拟存储技术

所以根据局部性原理可知,程序运行之前没有必要全部装入内存,只需要把当前要运行的少数页面/段先装入内存便可运行,其余部分留在盘上。如果程序运行时发现需要访问的页(段)还没调入内存(称为缺页或者缺段),于是发出缺页(段)中断请求,OS再利用请求调页(段)功能将它们调入内存,方便进程继续执行下去

(这三个功能均对用户透明)

之所以称为虚拟存储器,是因为这种这种存储器实际不存在,只是系统提供了部分装入、请求调入、置换功能后,给用户的感觉是好像存在一个比实际物理内存大得多的存储器。

虚拟存储器有三个主要特征

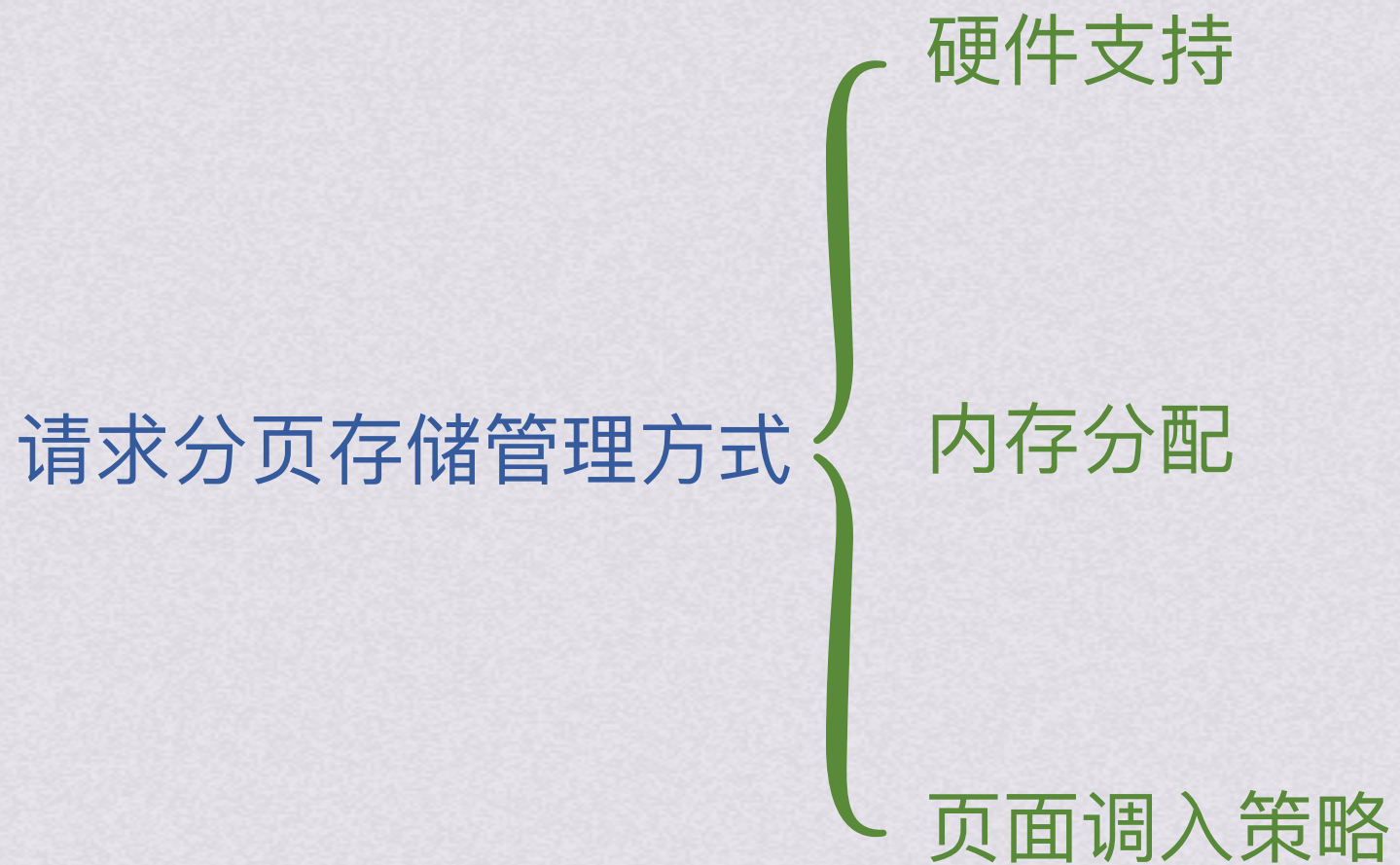
- 1.多次性: 不用在作业运行时一次全部装入内存,允许被分成多次调入内存,只需将当前要运行的那部分程序和数据装入内存就可以开始运行,每当运行到尚未调入的那部分程序/数据时,再将它们调入
- 2.对换性: 作业运行时无需一直常驻内存,允许在作业运行过程中将那些暂不使用的程序和数据从内存换出到外存,需要时再换回来
- 3.虚拟性: 逻辑上扩充了内存容量,用户看到的内存容量远大于实际容量



概述



请求分页存储管理方式





请求分页存储管理方式

硬件支持

基本分页存储管理方式
(之前学的)

+

请求调页功能
页面置换功能



请求分页存储管理方式

硬件支持

请求分页存储管理方式



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

因为请求分页是基于基本分页的,所以页号和物理块号是最基本的,和之前学的一样



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

请求分页的机制是只把程序的一部分调入内存,所以要增加一个比特记录一下他在内存还是在外存
状态位也叫存在位或者有效位



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

访问字段是用来记录本页在一段时间内被访问次数

用来给置换算法做参考

经常被访问的页就更应该被留在内存中

不被访问的页面就该调出外存



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

标识页面在调入内存后是否被修改过

因为所有页面在外存中也有一份,如果页面在内存中被修改过,就应该在恰当的时候把外存的那一份也改成被修改后的版本
如果没被修改,那换出的时候就可以不用写回外存



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

记录这一页在外存上的地址



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

记录这一页在内存上的地址

记录这一页在外存上的地址

别搞混了



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

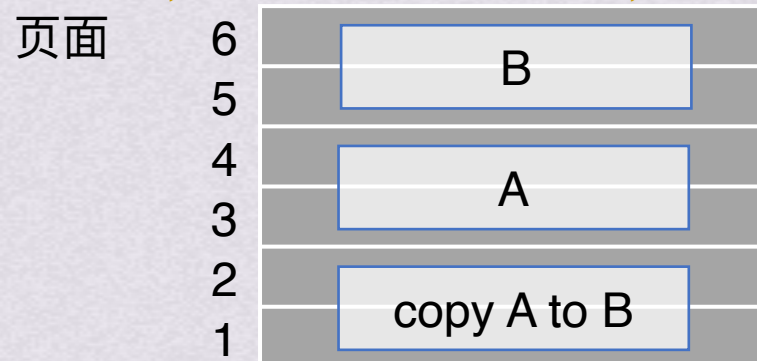
页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

2.缺页中断机构：要访问的页面不在内存的时候,要产生缺页中断,让OS把缺少的页面调入内存。缺页中断是一种中断机制,所以需要经历保护CPU环境、分析中断原因、转入缺页中断处理程序的流程,但是也有区别：

2-1：缺页中断在指令执行期间产生和处理中断信号,常规的中断要在指令执行完后才响应

2-2：一条指令在执行期间可能产生多次缺页中断,比如执行copy A to B

指令本身横跨两个页面,A和B再横跨两个页面,那可能产生6次缺页中断





请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

2.缺页中断机构：要访问的页面不在内存的时候,要产生缺页中断,让OS把缺少的页面调入内存。缺页中断是一种中断机制,所以需要经历保护CPU环境、分析中断原因、转入缺页中断处理程序的流程,但是也有区别：

2-1：缺页中断在指令执行期间产生和处理中断信号,常规的中断要在指令执行完后才响应

2-2：一条指令在执行期间可能产生多次缺页中断,比如执行copy A to B

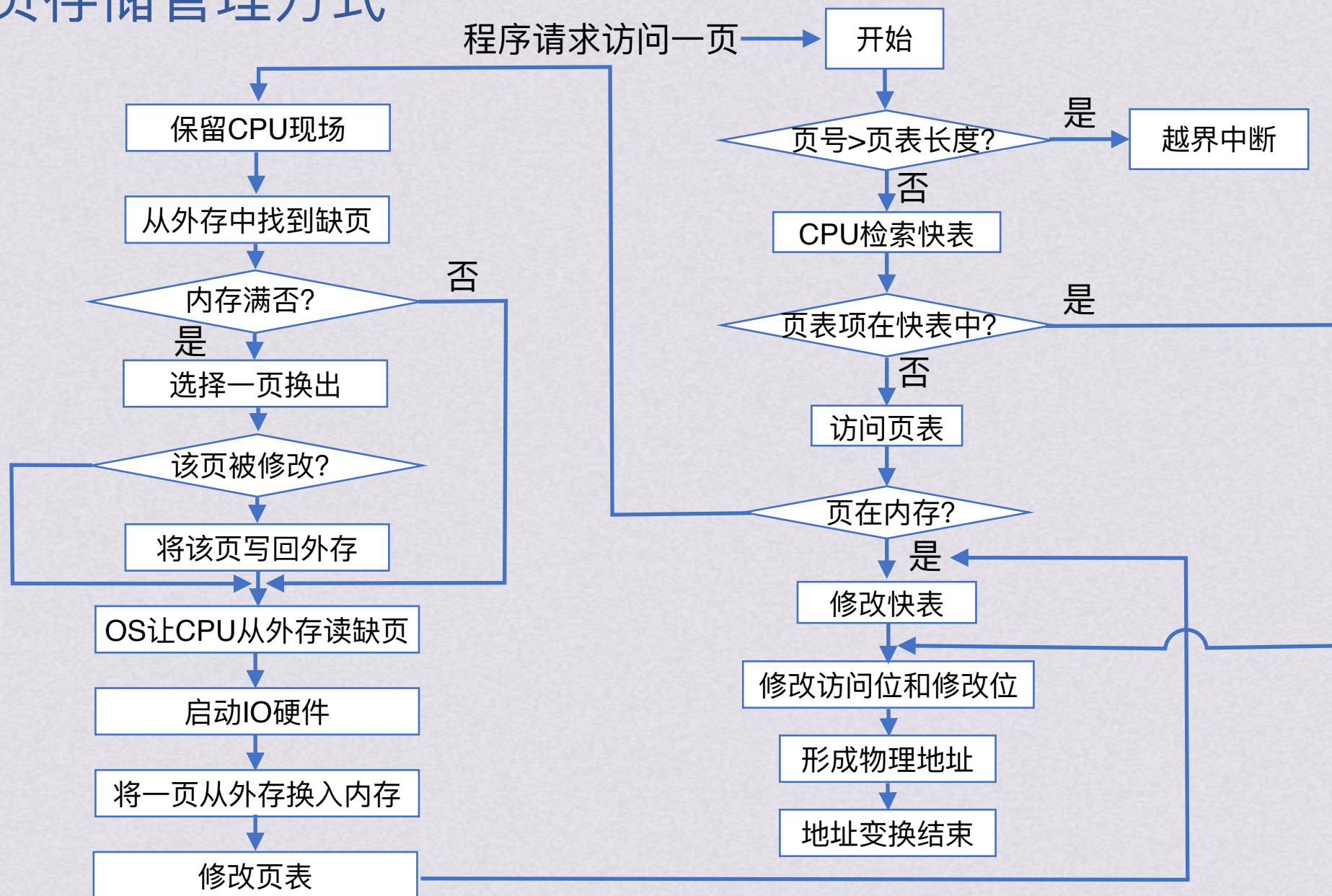
指令本身横跨两个页面,A和B再横跨两个页面,那可能产生6次缺页中断

3.地址变换机构：请求分页的地址变换机构是在分页系统地址变换机构的基础上实现虚拟存储器所需功能实现的



请求分页存储管理方式

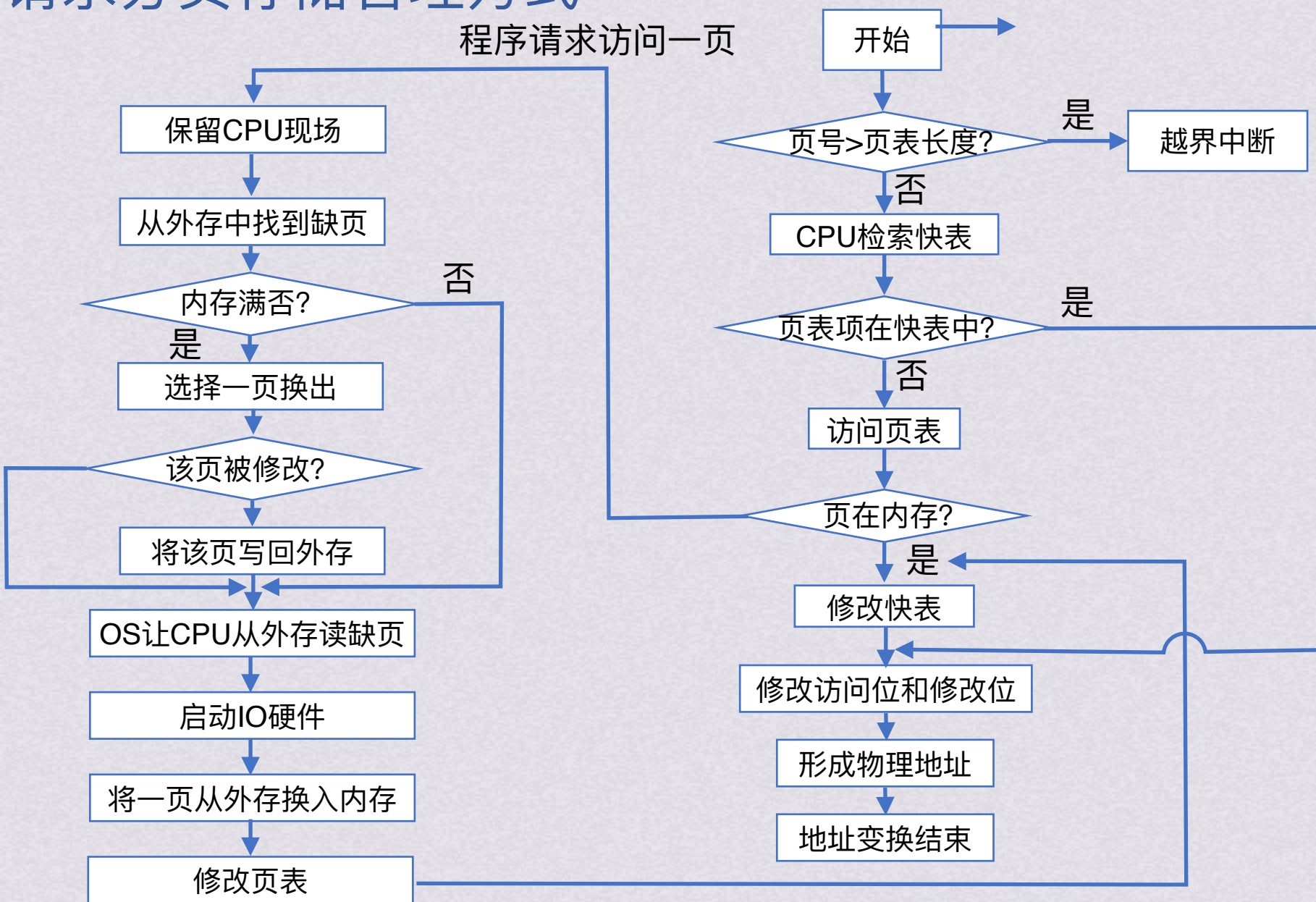
3.地址变换机构：请求分页的地址变换机构是在分页系统地址变换机构的基础上实现虚拟存储器所需功能实现的





请求分页存储管理方式

3.地址变换机构：请求分页的地址变换机构是在分页系统地址变换机构的基础上实现虚拟存储器所需功能实现的



地址变换时,首先检索快表,如果找到就修改访问位供置换算法参考。

对于写指令,还需要改修改位,表示该页调入内存后已被修改。然后利用页表项中给出的物理块号和页内地址形成物理地址

如果快表中没找到,就要去内存中查找页表,再从找到的页表项中的状态位判断是否已调入内存。

如果已经调入内存,则把该页页表项写入快表。如果快表满,则按照算法调出,写入该页的页表项;

如果没有调入内存,则产生缺页中断,请求OS从外存把该页调入内存



请求分页存储管理方式

硬件支持

为了实现 请求分页存储管理方式 系统必须提供一定的硬件支持,计算机系统除了要求内存外存,还需要请求页表机制、缺页中断机制和地址变换机构

1.请求页表机制：作用是把用户的逻辑地址映射到内存空间的物理地址,考虑到需要换进换出,请求分页系统中的每个页表要包含这些东西

页号	物理块号	状态位P	访问字段A	修改位M	外存地址
----	------	------	-------	------	------

2.缺页中断机构：要访问的页面不在内存的时候,要产生缺页中断,让OS把缺少的页面调入内存。缺页中断是一种中断机制,所以需要经历保护CPU环境、分析中断原因、转入缺页中断处理程序的流程,但是也有区别：

2-1：缺页中断在指令执行期间产生和处理中断信号,常规的中断要在指令执行完后才响应

2-2：一条指令在执行期间可能产生多次缺页中断,比如执行copy A to B

指令本身横跨两个页面,A和B再横跨两个页面,那可能产生6次缺页中断

3.地址变换机构：请求分页的地址变换机构是在分页系统地址变换机构的基础上实现虚拟存储器所需功能实现的



请求分页存储管理方式

硬件支持



请求分页存储管理方式

内存分配



请求分页存储管理方式

内存分配

为进程分配内存的时候需要确定三个问题：

- 1.进程能正常运行最少需要多少物理块？
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的？
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配？



请求分页存储管理方式

内存分配

- 2. 为每个进程分配物理块时, 分配的物理块是固定的还是可变的?
- 3. 为不同进程分配的物理块数, 是采取平均分配, 还是按进程的大小按比例分配?

为进程分配内存的时候需要确定三个问题:

1. 进程能正常运行最少需要多少物理块?

最小物理块是保证进程正常运行所需的最小物理块数量。当系统为进程分配的物理块数量少于此值时, 进程无法运行。这个数量和计算机的硬件结构有关, 取决于指令的格式、功能、寻址方式



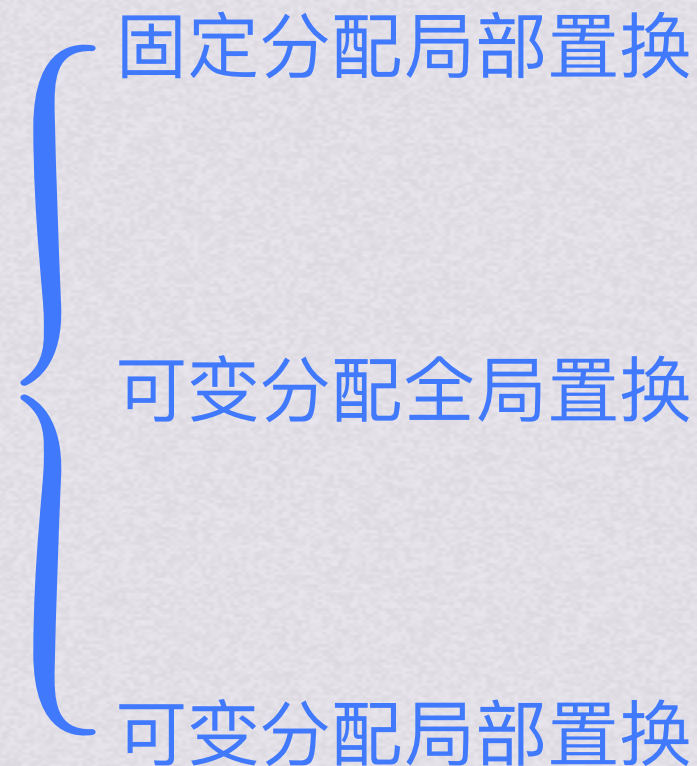
请求分页存储管理方式

内存分配

1. 进程能正常运行最少需要多少物理块?
3. 为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

为进程分配内存的时候需要确定三个问题:

2. 为每个进程分配物理块时,分配的物理块是固定的还是可变的?





请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

固定分配局部置换

固定分配指的是为每个进程分配固定数目的物理块,进程运行期间不再改变

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

请求分页存储管理方式

内存分配

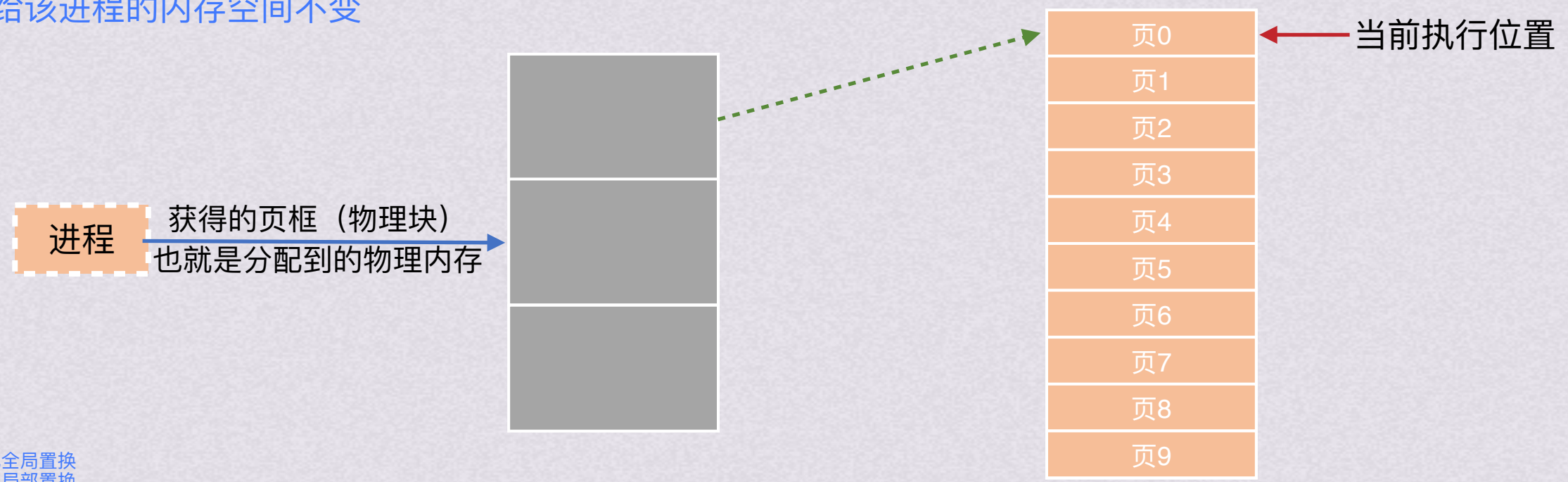
- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

固定分配局部置换

固定分配指的是为每个进程分配固定数目的物理块,进程运行期间不再改变

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变



程序空间的分页情况

- 可变分配全局置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

固定分配局部置换

- 可变分配全局置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
 - 页面置换算法
 - 抖动与工作集
 - 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



请求分页存储管理方式

内存分配

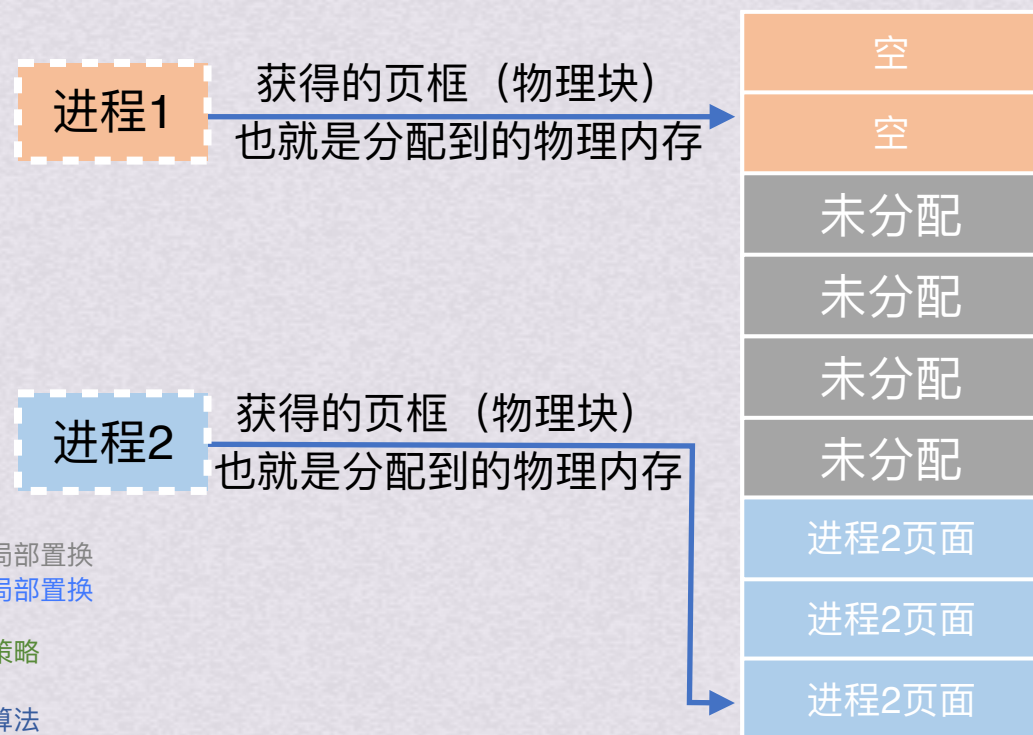
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件

请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件

请求分页存储管理方式

内存分配

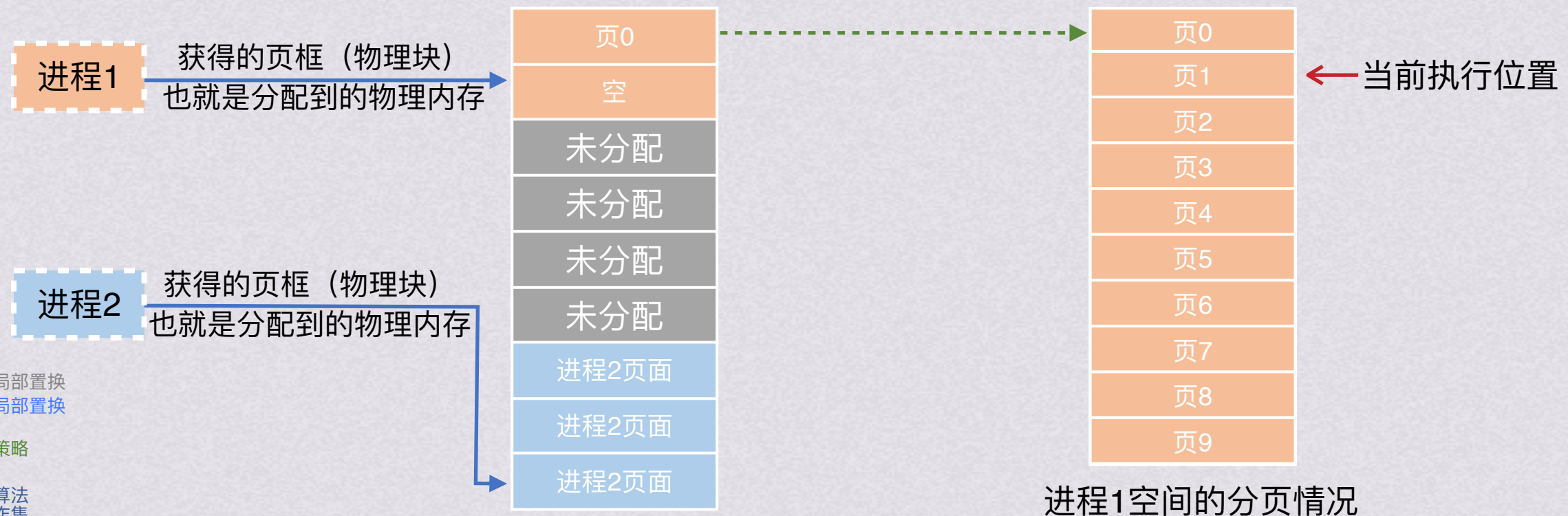
- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件

请求分页存储管理方式

内存分配

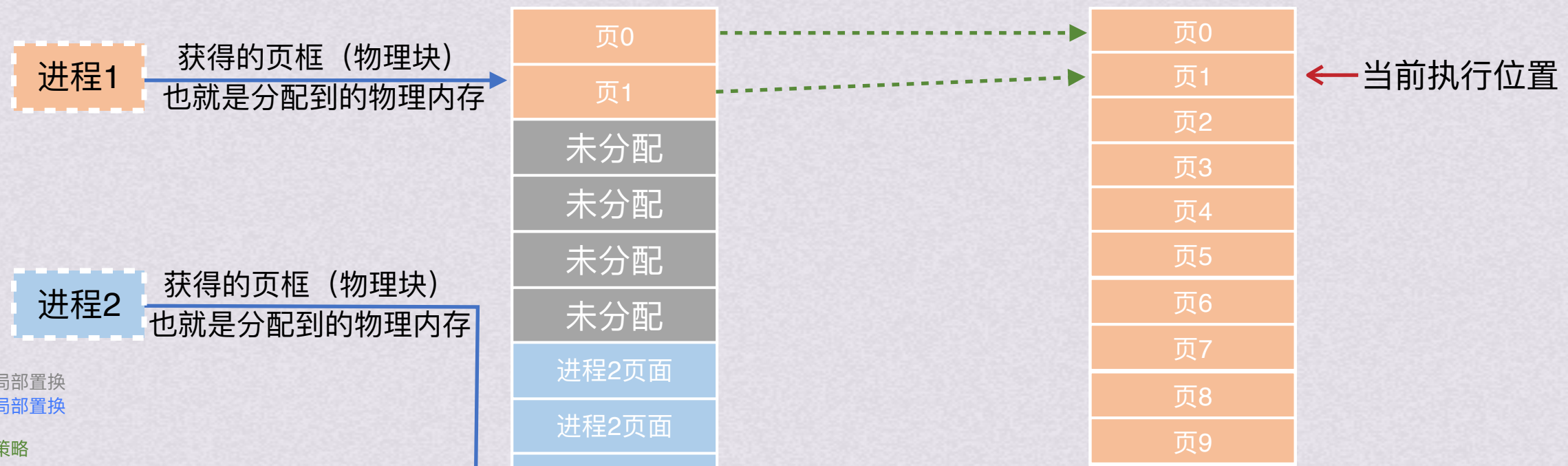
- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件

请求分页存储管理方式

内存分配

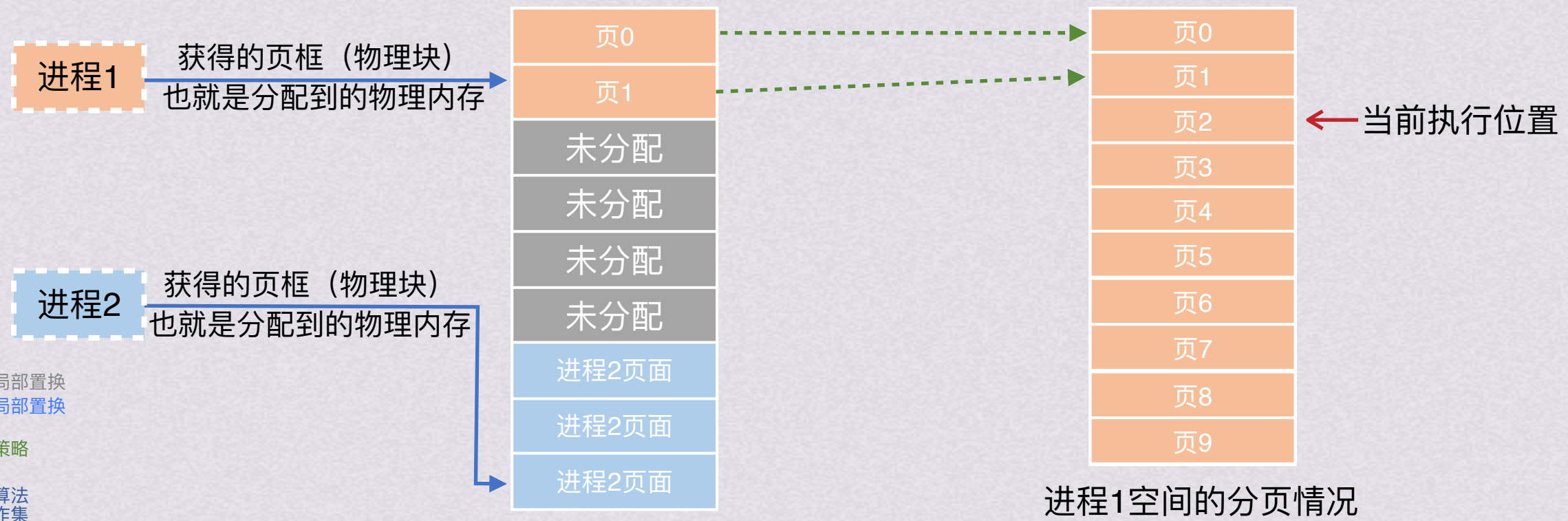
- 1. 进程能正常运行最少需要多少物理块?
- 3. 为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2. 为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

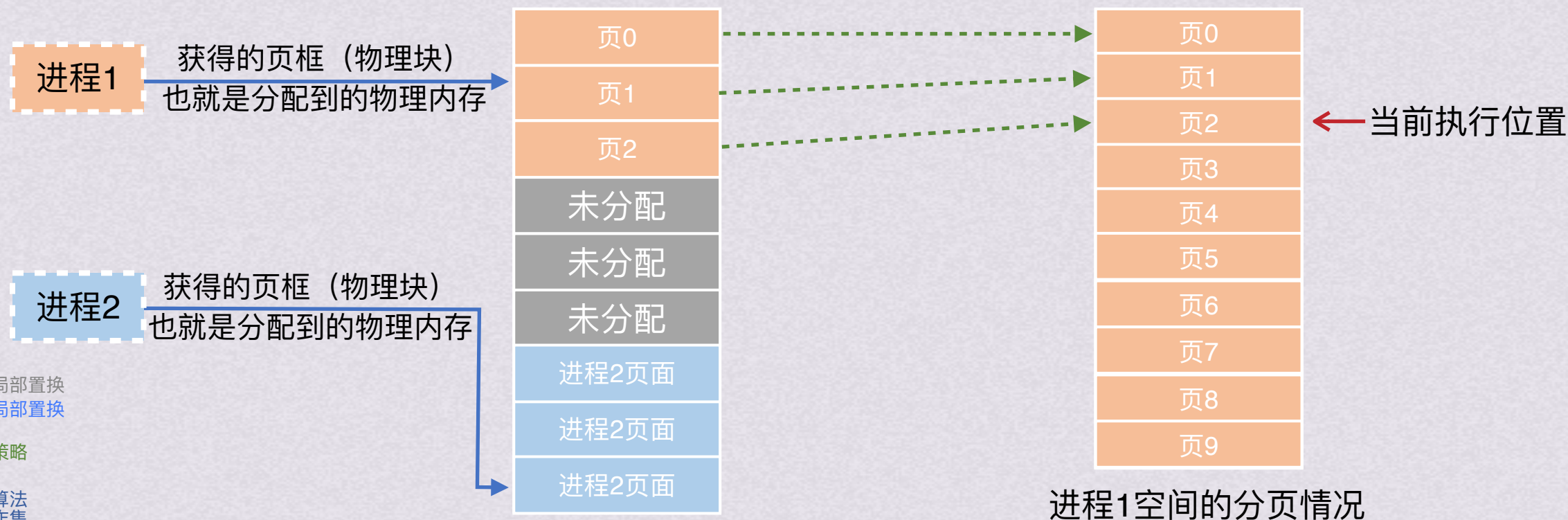
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

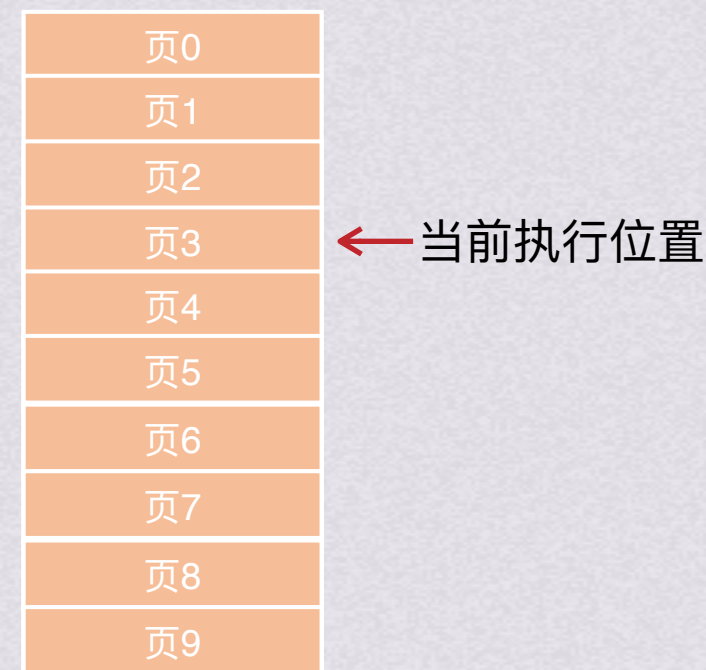
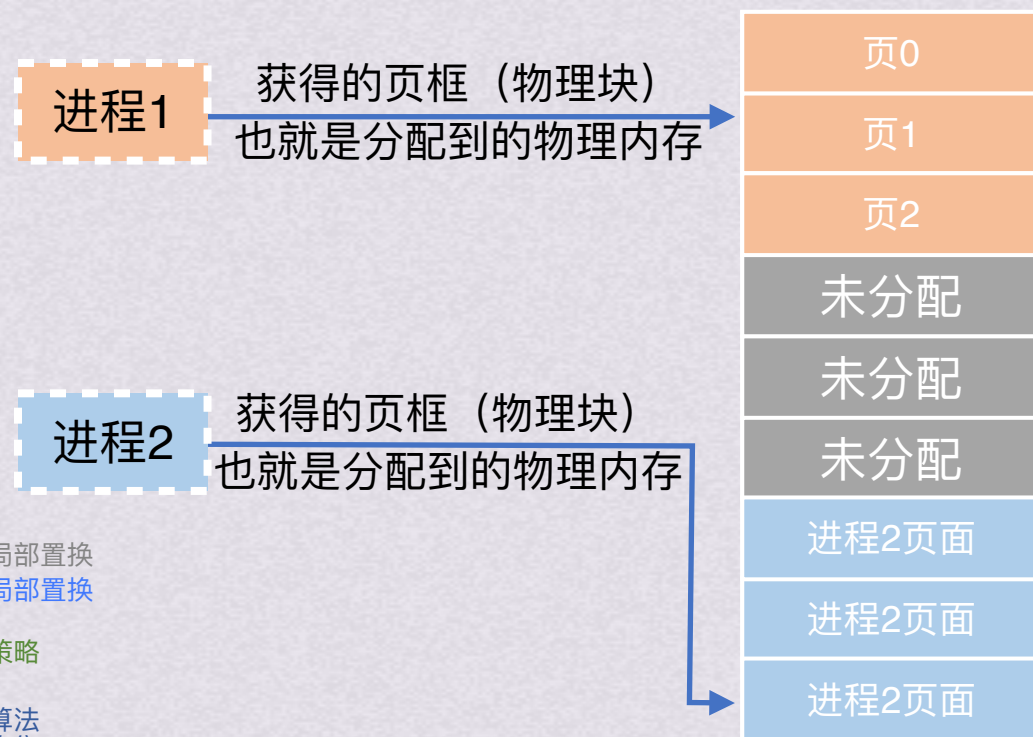
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况



请求分页存储管理方式

内存分配

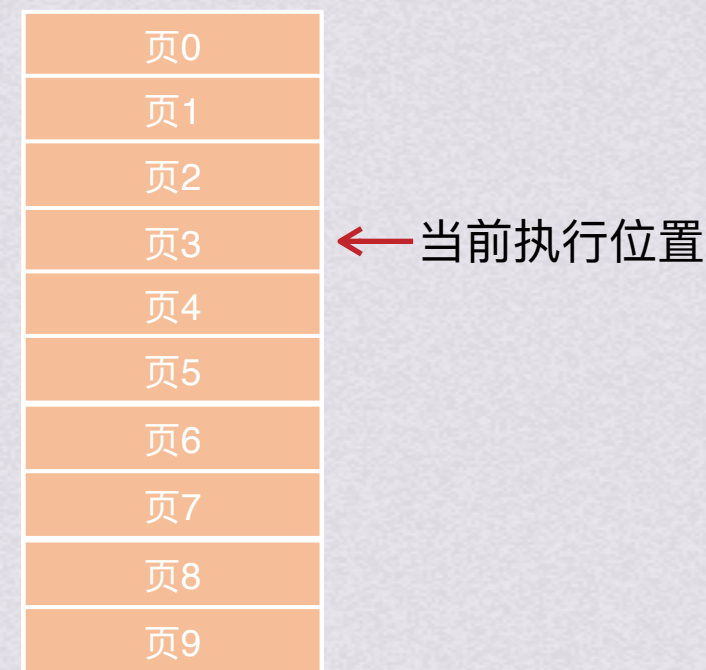
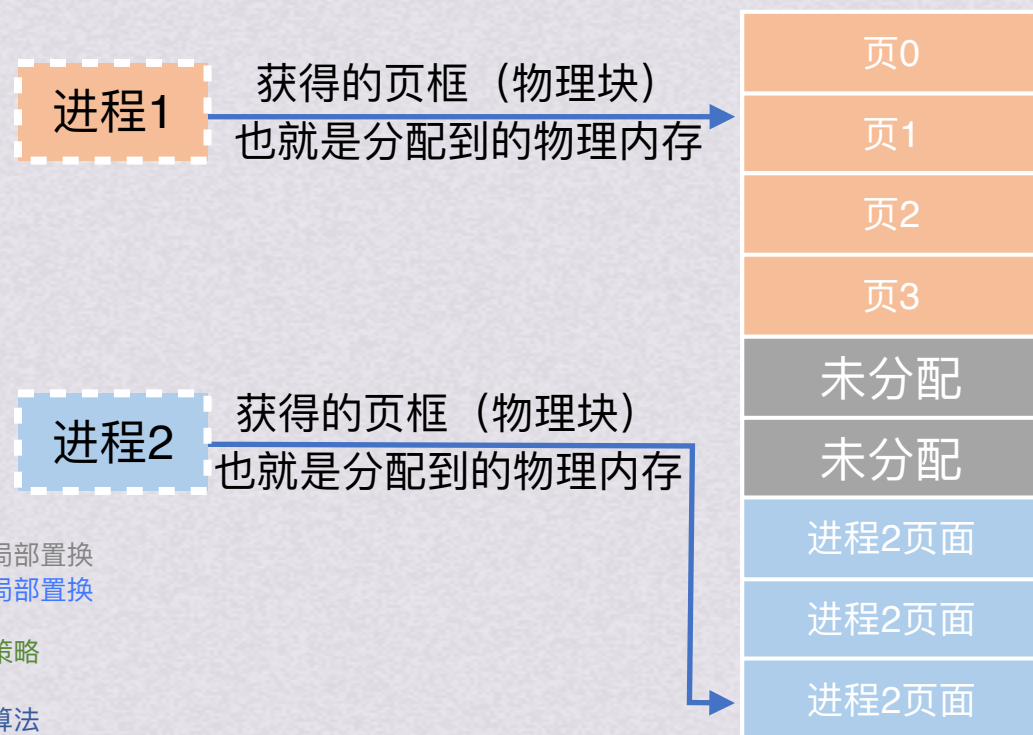
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况



请求分页存储管理方式

内存分配

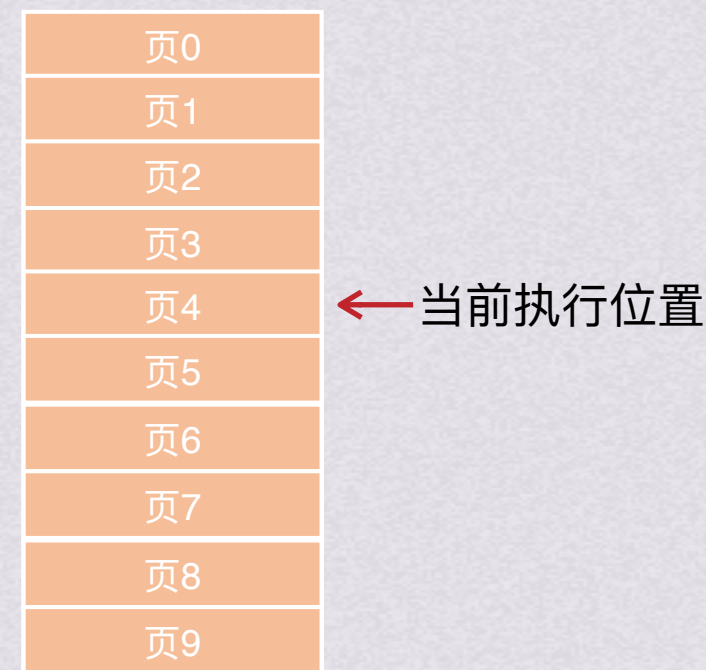
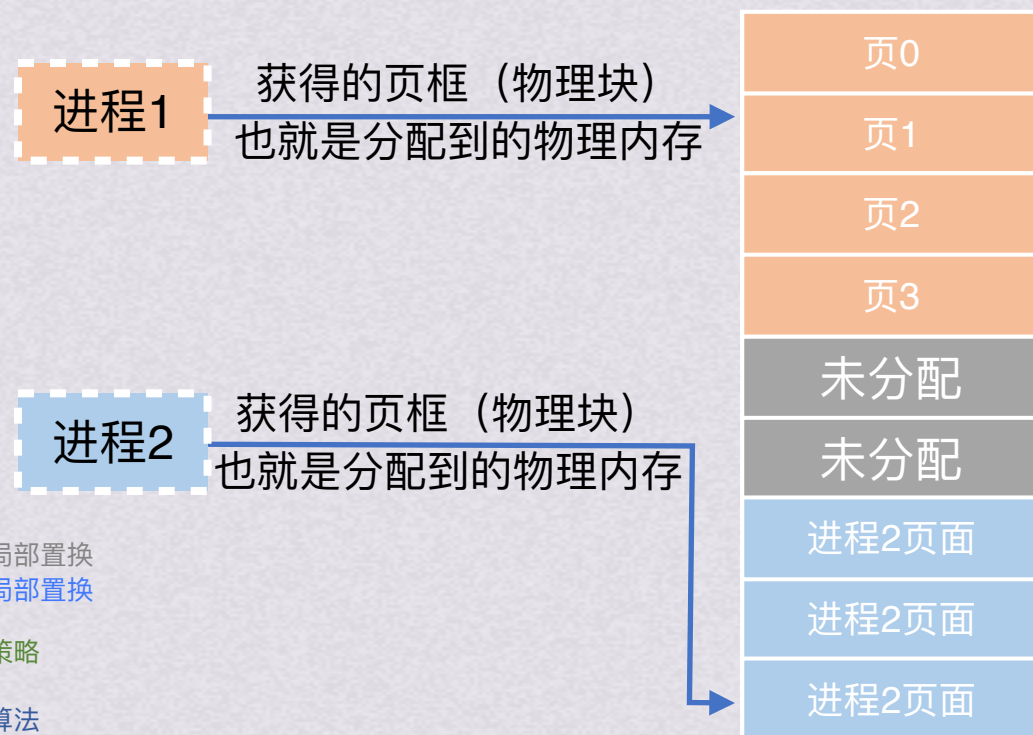
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

请求分页存储管理方式

内存分配

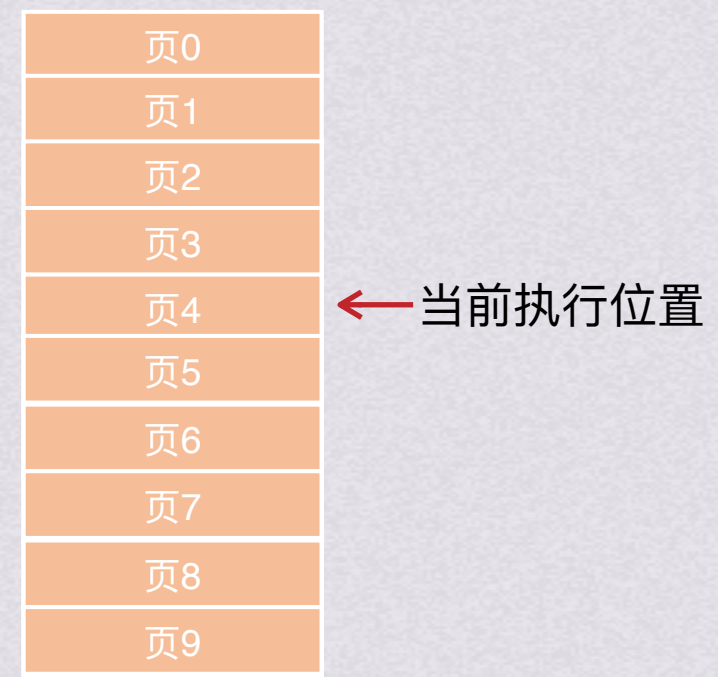
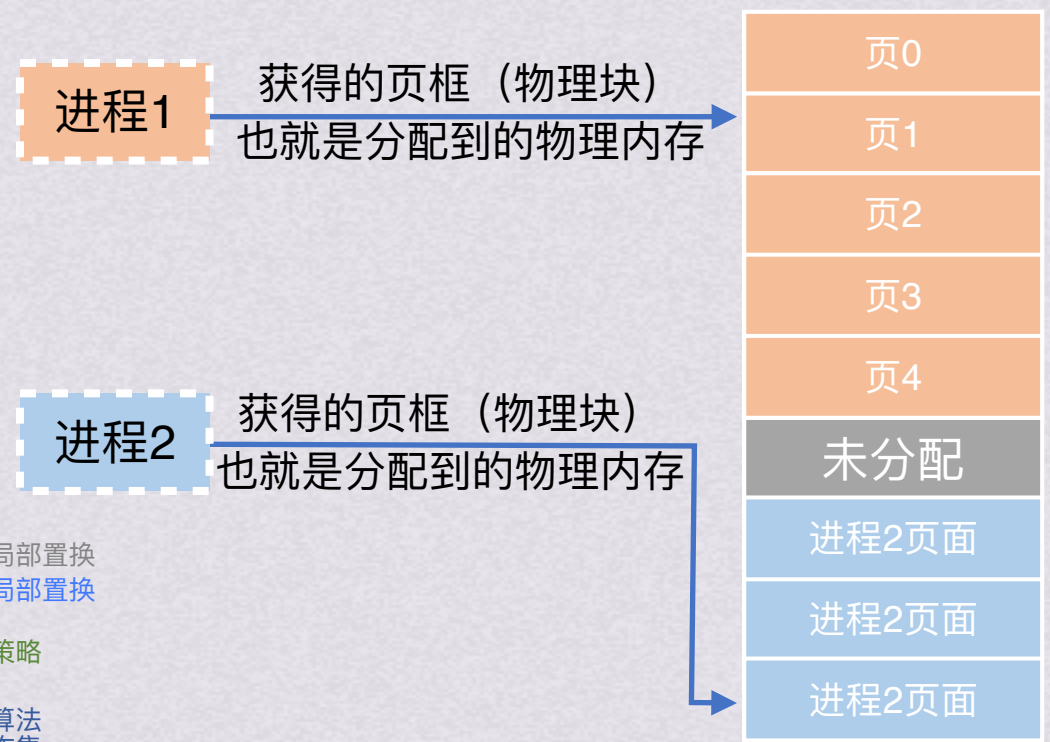
- 1. 进程能正常运行最少需要多少物理块?
- 3. 为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2. 为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

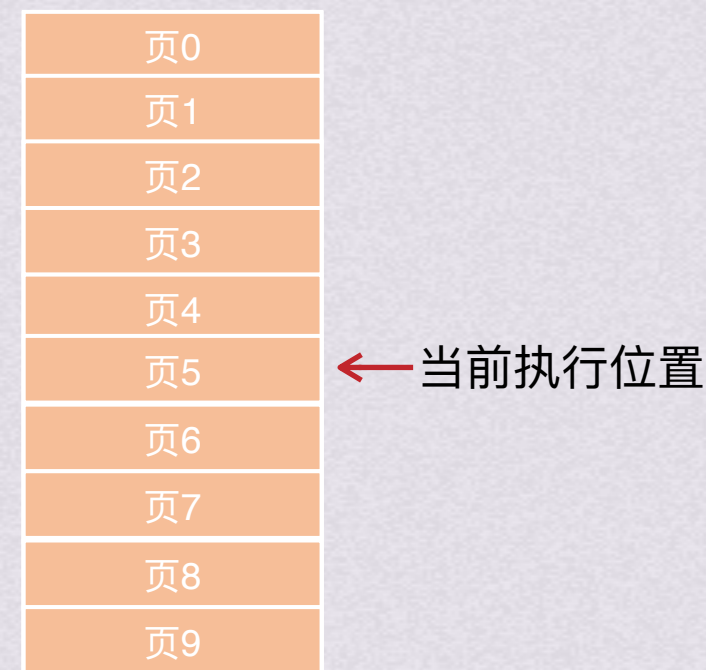
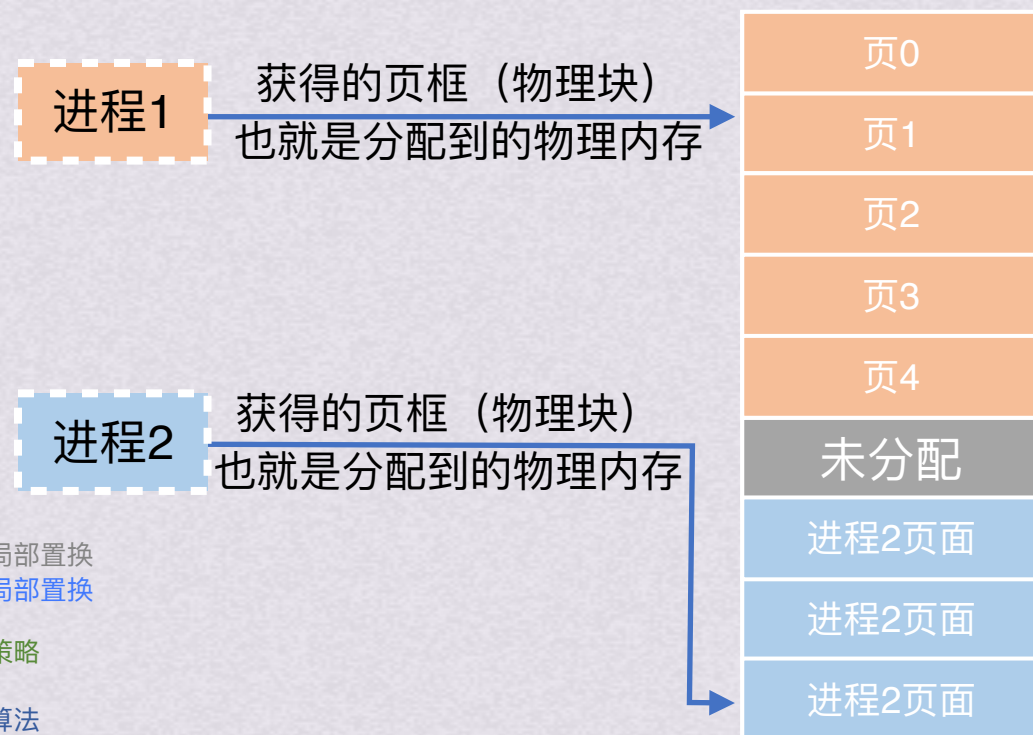
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

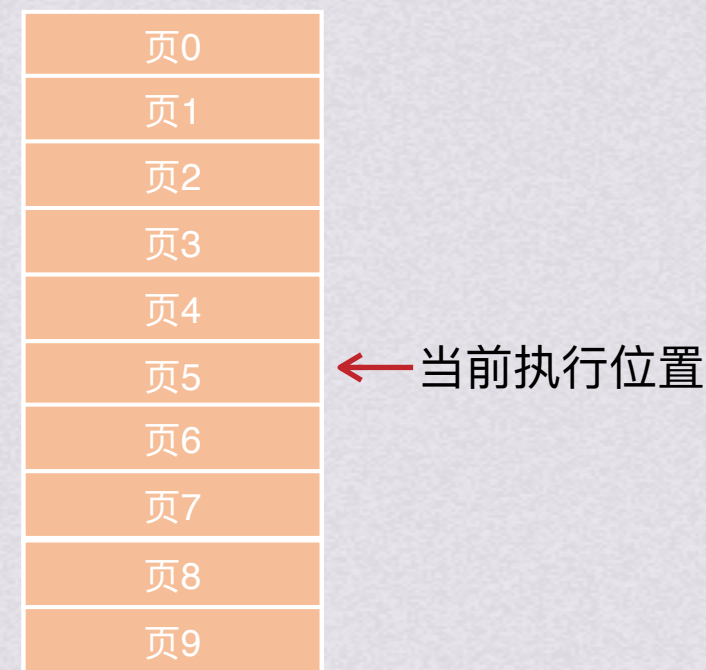
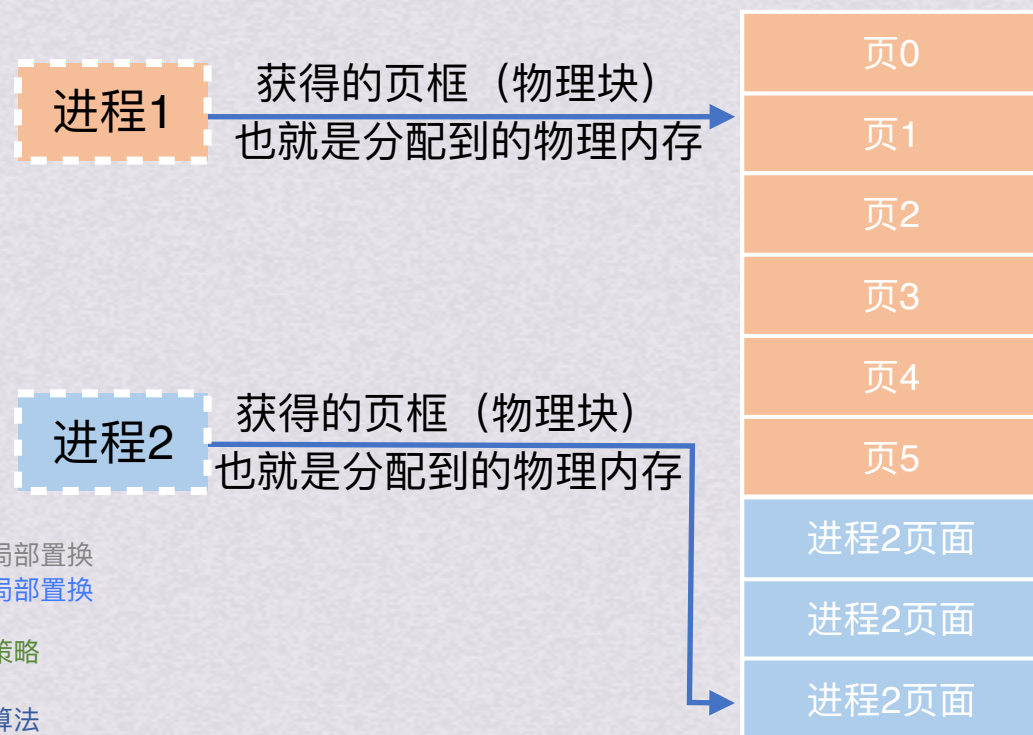
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

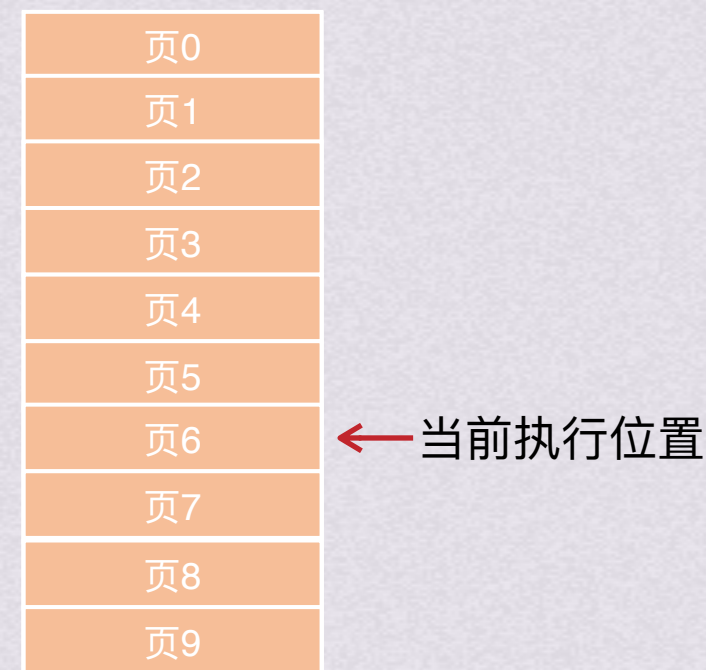
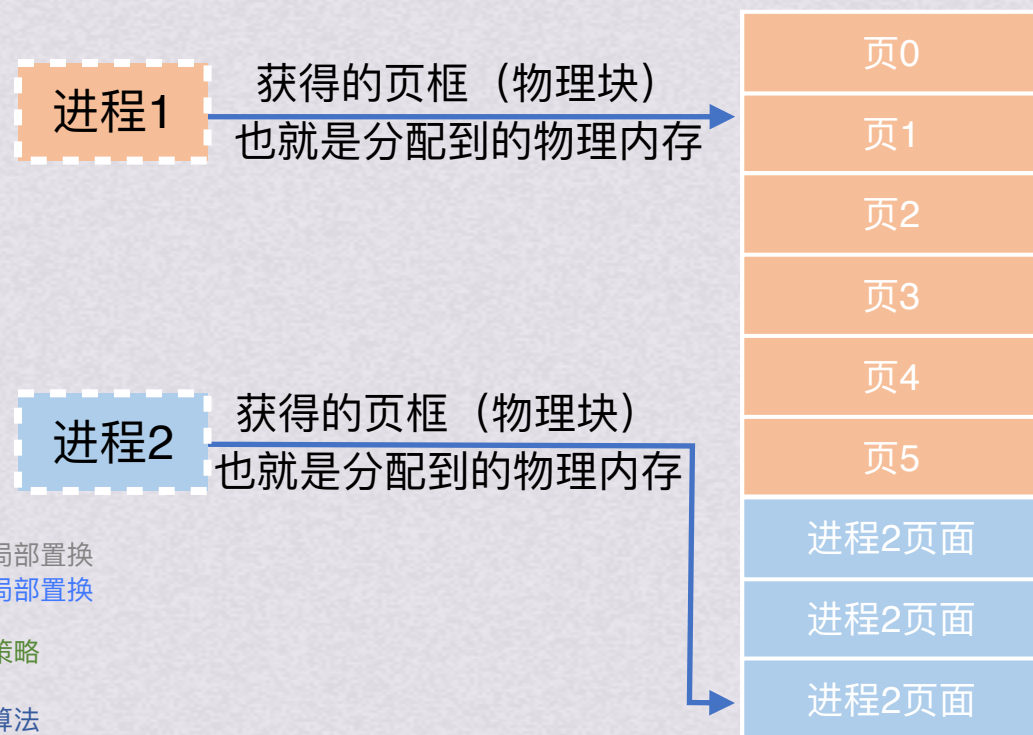
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

1. 进程能正常运行最少需要多少物理块?
2. 为每个进程分配物理块时, 分配的物理块是固定的还是可变的?
3. 为不同进程分配的物理块数, 是采取平均分配, 还是按进程的大小按比例分配?

2. 为每个进程分配物理块时, 分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块, 运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页, OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程, 都会获得新的物理块, 空闲物理块队列中的物理块用完时, OS才能从内存中选择一页调出, 被选择调出的页可能是任何一个进程的, 被选中的进程拥有的物理块会减少



- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

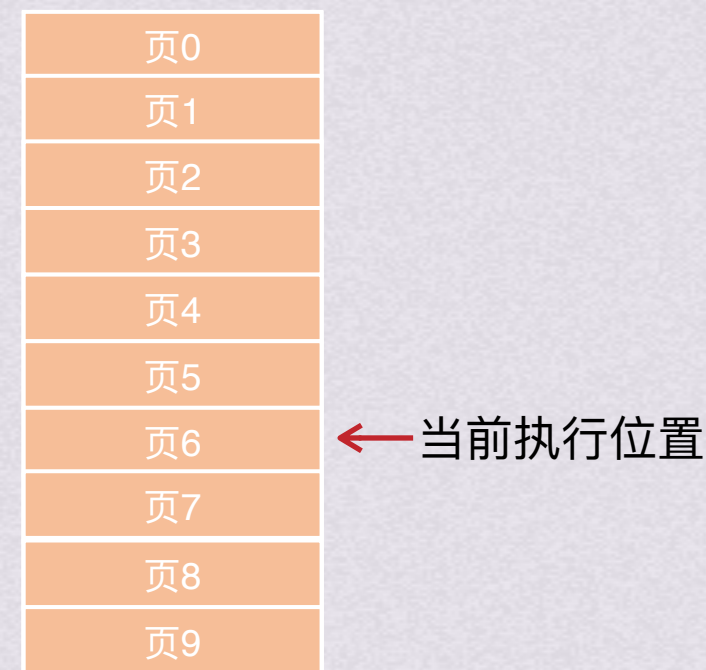
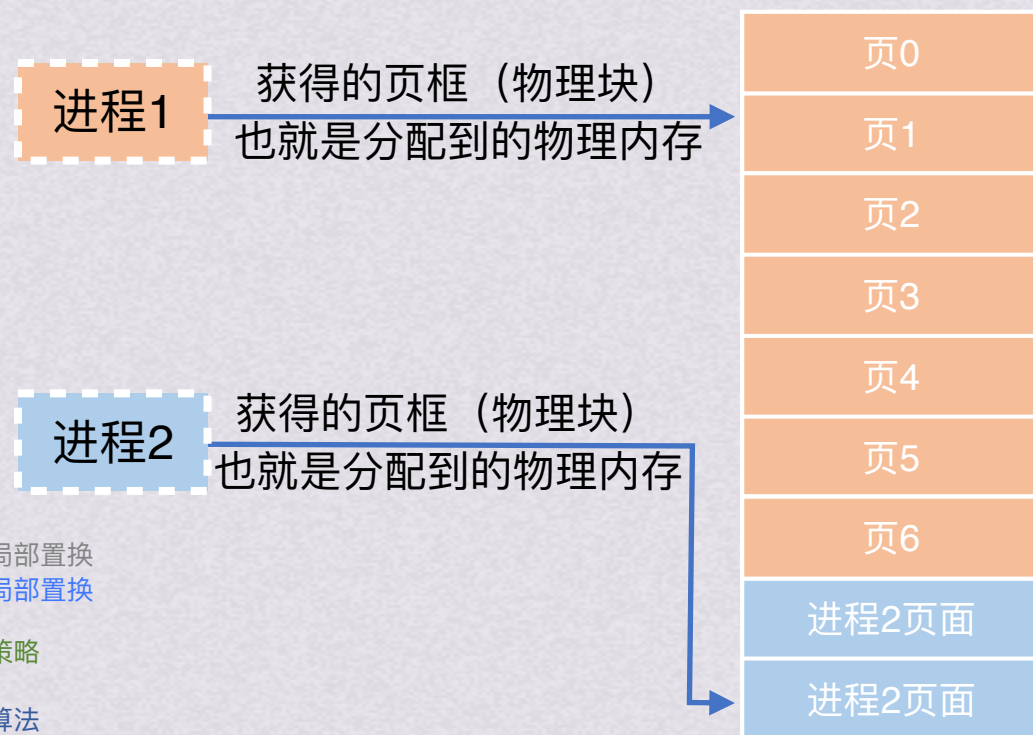
- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

全局置换指如果进程运行过程中发现缺页,OS会在空闲物理块中取出一块分配给该进程。这种策略下凡产生缺页的进程,都会获得新的物理块,空闲物理块队列中的物理块用完时,OS才能从内存中选择一页调出,被选择调出的页可能是任何一个进程的,被选中的进程拥有的物理块会减少



进程1空间的分页情况

- 固定分配局部置换
- 可变分配局部置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配全局置换



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

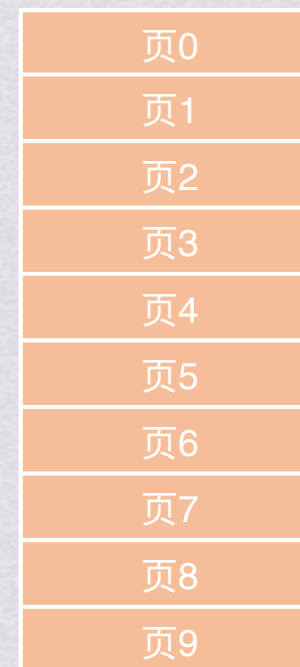
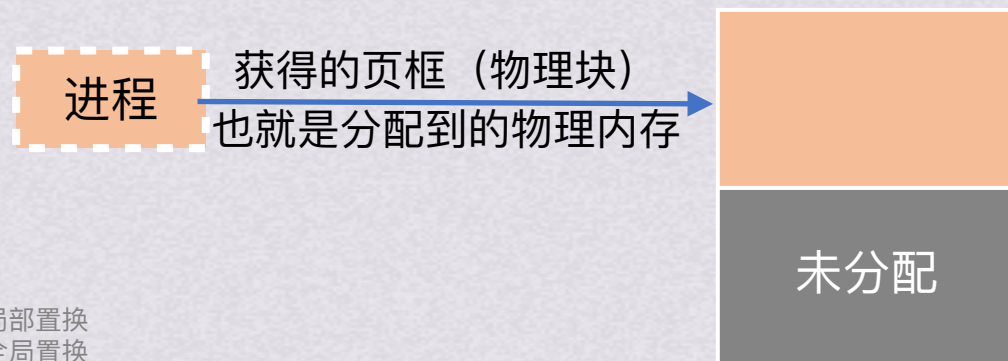
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

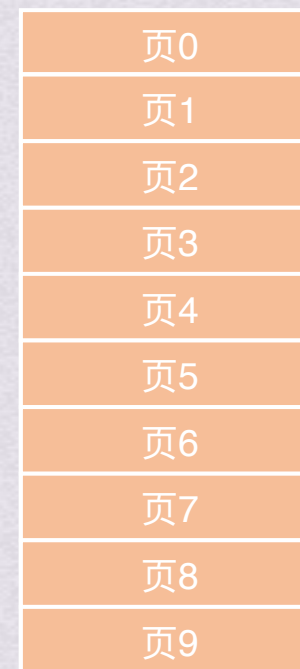
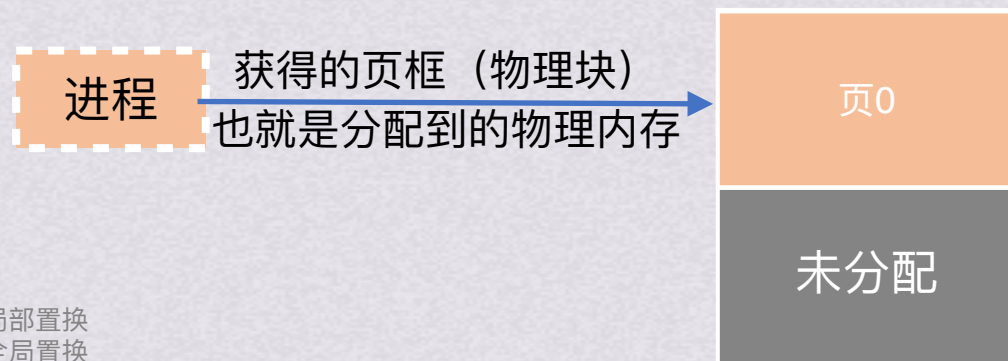
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

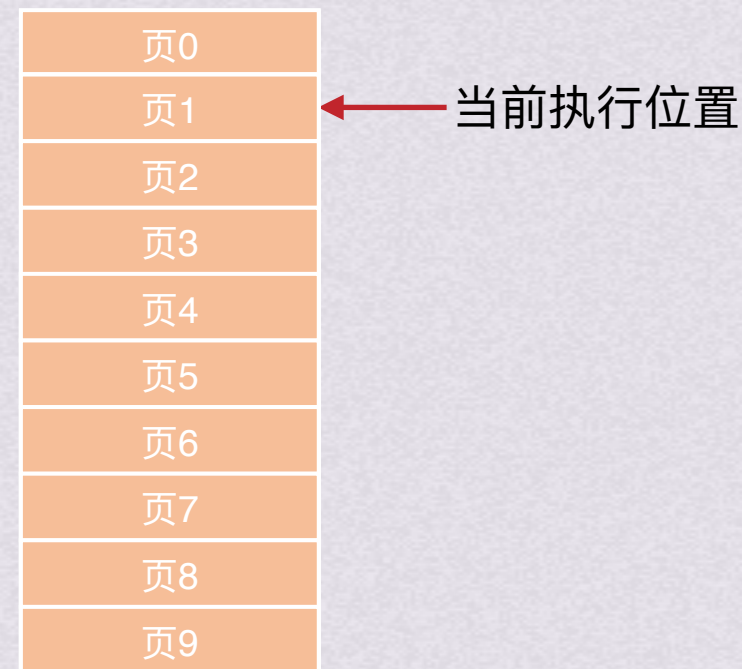
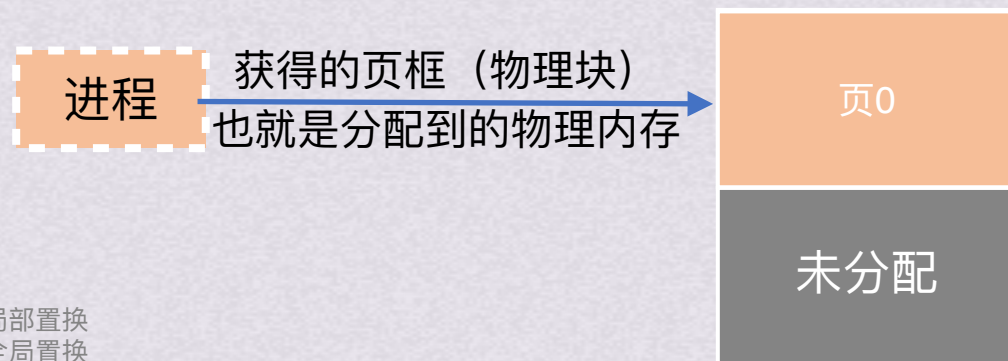
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

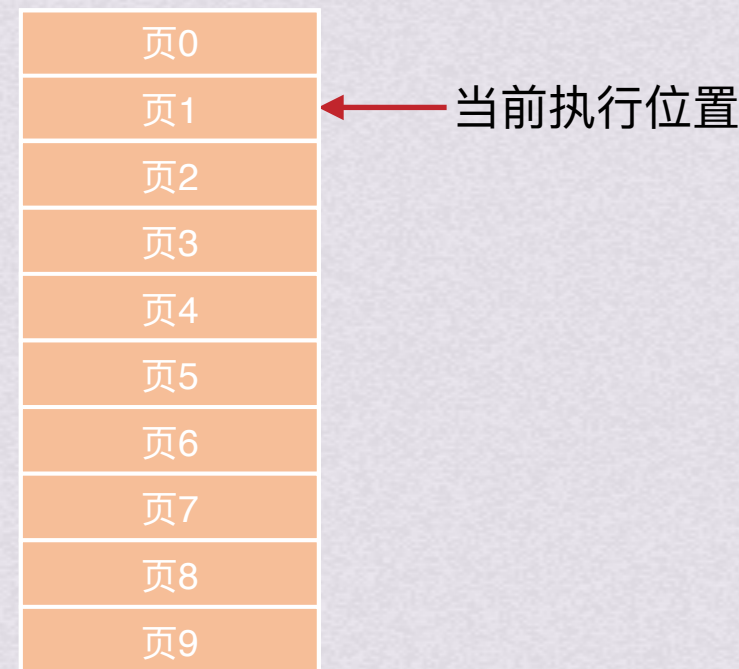
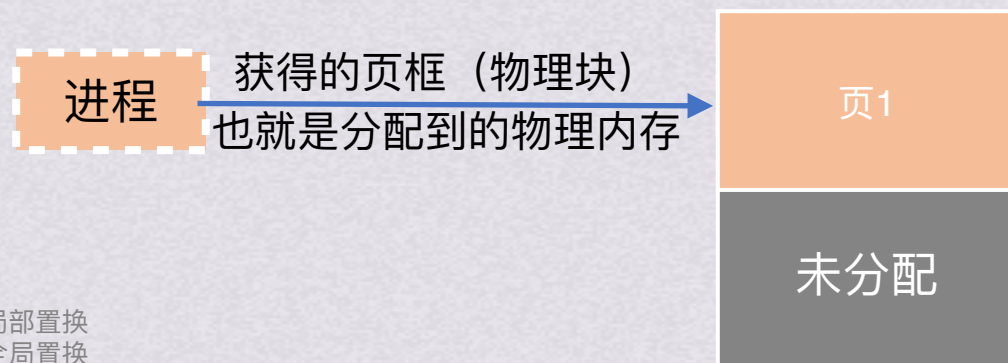
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

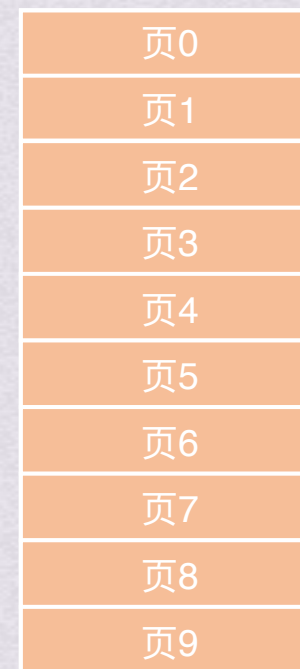
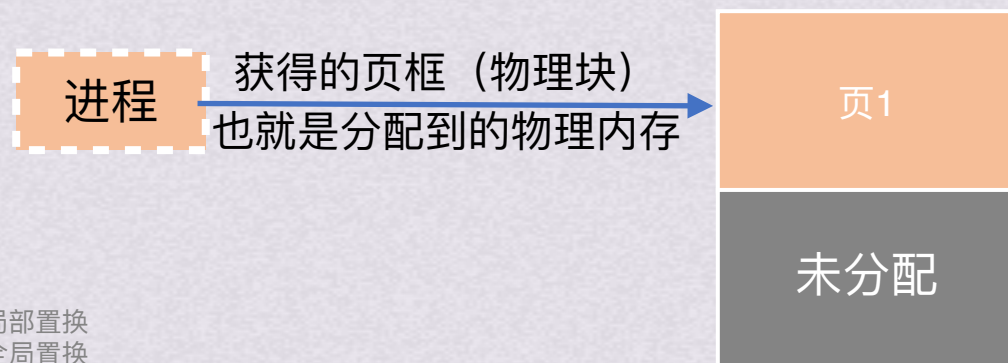
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

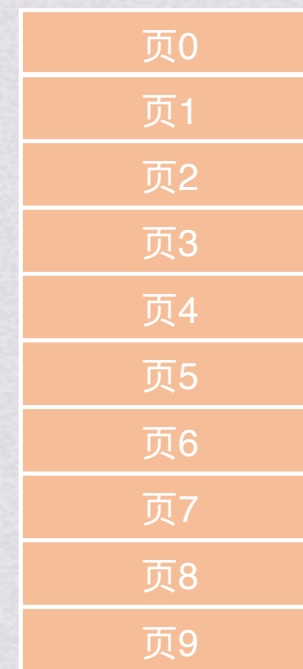
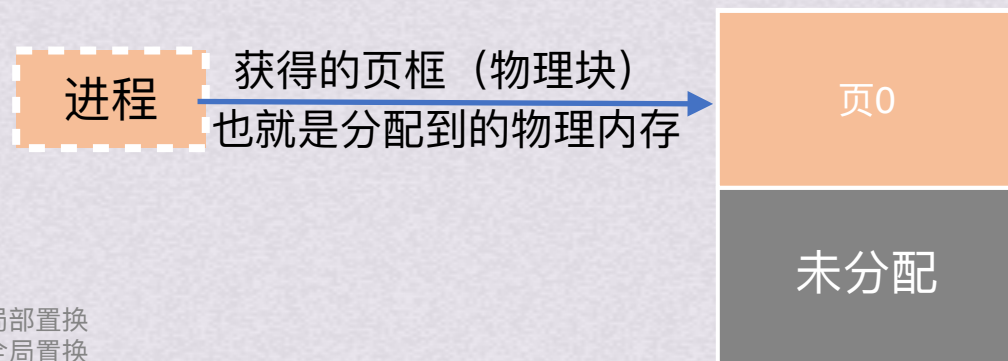
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

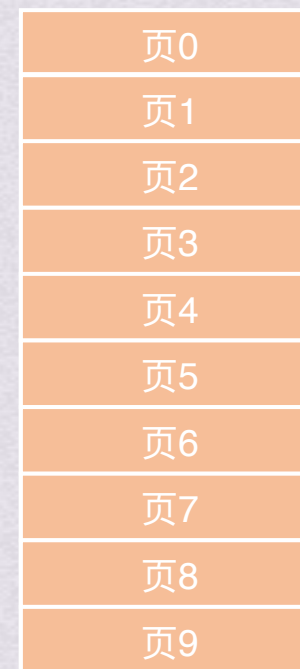
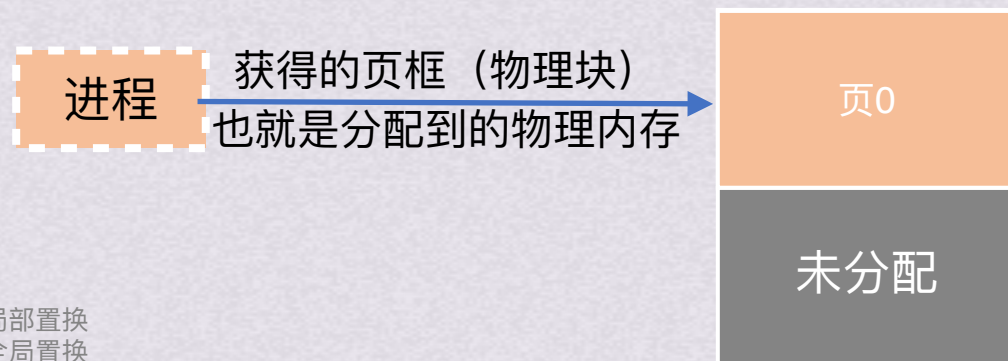
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

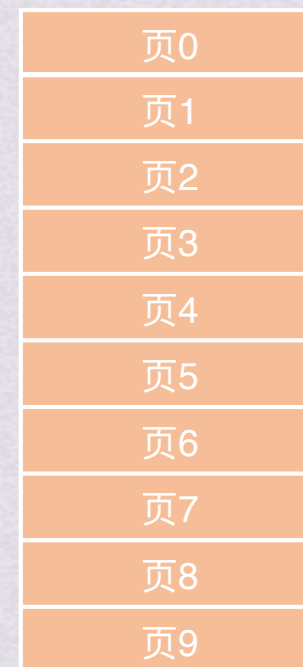
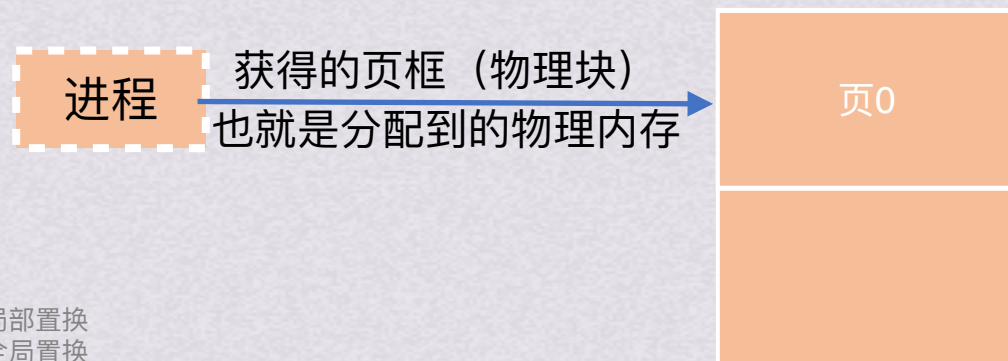
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

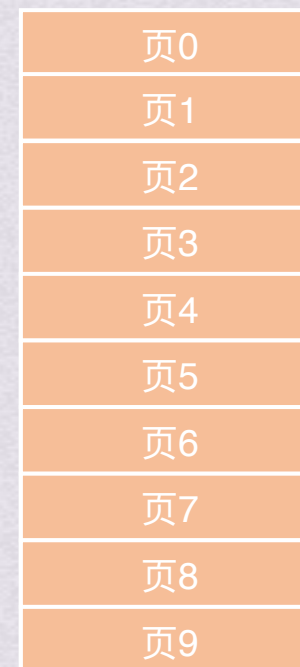
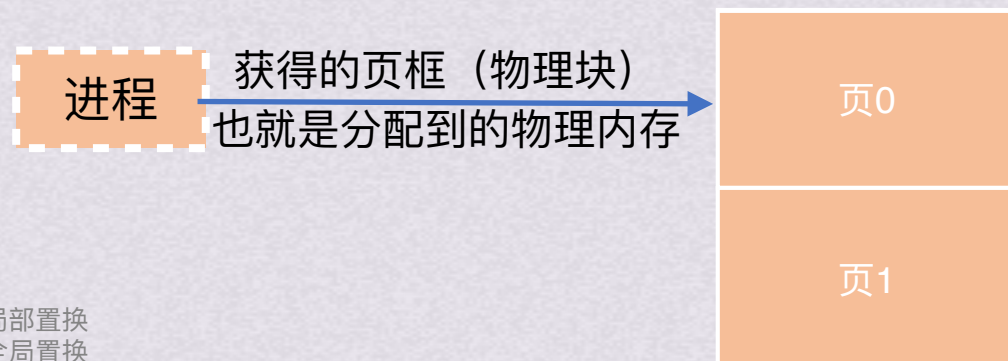
2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

可变分配指的是先为每个进程分配一些物理块,运行期间可以根据情况增加减少

局部置换指的是如果进程在运行中发现了缺页,只能从分配给该进程的n个页框中选择一页换出,然后调入一页,这样来保证分配给该进程的内存空间不变

如果进程在运行中频繁发生缺页中断,系统会给该进程增加物理块; 如果缺页率特别低,则会适当减少分配的物理块



← 当前执行位置

程序空间的分页情况

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



请求分页存储管理方式

内存分配

- 1.进程能正常运行最少需要多少物理块?
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

2.为每个进程分配物理块时,分配的物理块是固定的还是可变的?

可变分配局部置换

- 固定分配局部置换
- 可变分配全局置换
- 硬件支持
- 页面调入策略
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



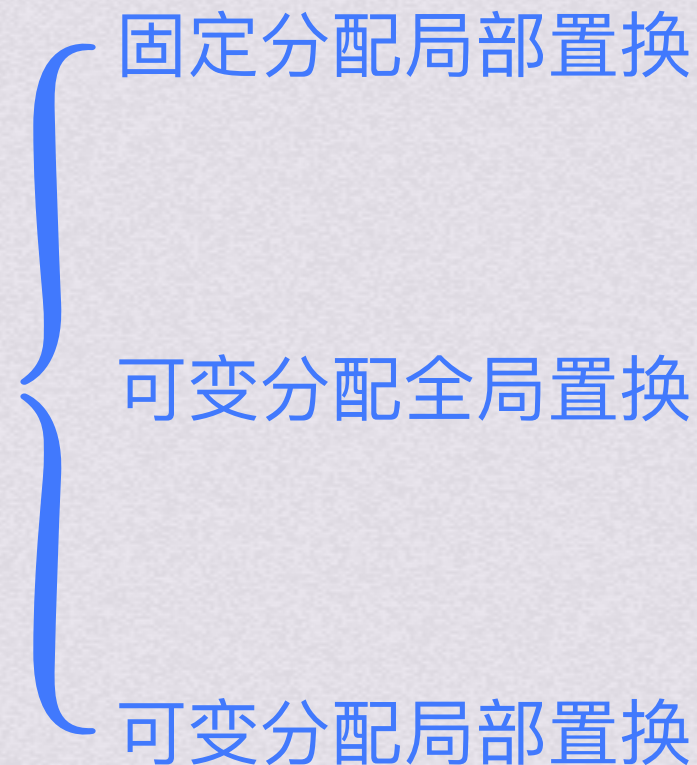
请求分页存储管理方式

内存分配

1. 进程能正常运行最少需要多少物理块?
3. 为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配?

为进程分配内存的时候需要确定三个问题:

2. 为每个进程分配物理块时,分配的物理块是固定的还是可变的?





请求分页存储管理方式

内存分配

1. 进程能正常运行最少需要多少物理块?
2. 为每个进程分配物理块时, 分配的物理块是固定的还是可变的?

为进程分配内存的时候需要确定三个问题:

3. 为不同进程分配的物理块数, 是采取平均分配, 还是按进程的大小按比例分配?

都行, 可以平均, 可以按比例, 也可以按照进程优先级, 根据需求来



请求分页存储管理方式

内存分配

为进程分配内存的时候需要确定三个问题：

- 1.进程能正常运行最少需要多少物理块？
- 2.为每个进程分配物理块时,分配的物理块是固定的还是可变的？
- 3.为不同进程分配的物理块数,是采取平均分配,还是按进程的大小按比例分配？



请求分页存储管理方式

内存分配

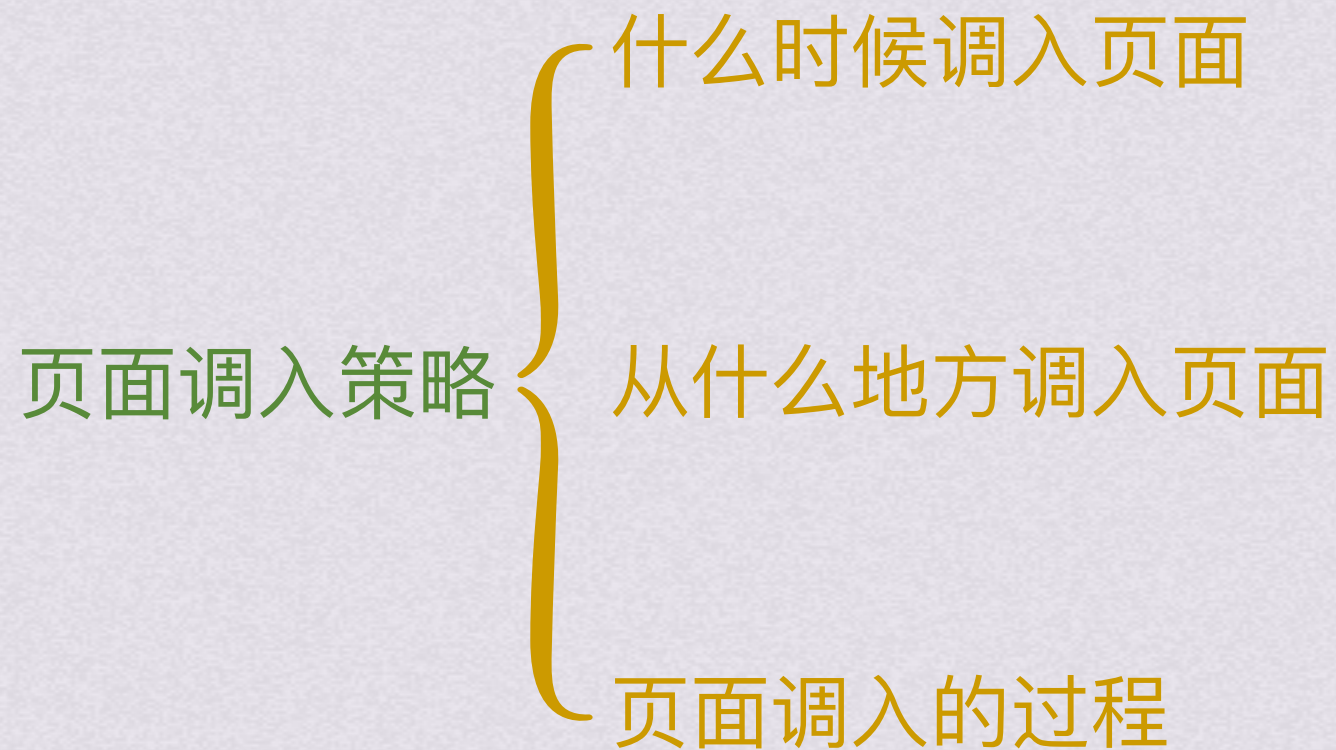


请求分页存储管理方式

页面调入策略



请求分页存储管理方式





页面调入策略

什么时候调入页面

系统在进程运行缺页的时候会把缺少的页面调入内存,有**预调页**和**请求调页**两种策略

预调页: 对页面的访问进行预测,把那些预计在之后不就不会被访问的页面预先调入内存。但目前预调页成功率仅约50%

请求调页: 进程在运行中需要访问某部分程序或数据时,若发现页面不在内存,就立即提出请求,让OS将其所需页面调入内存。这样的策略易于实现,所以虚拟存储器大多采用此策略。但这样每次仅调入一页,会花费较大的系统开销,增加了磁盘IO的启动频率



页面调入策略

从什么地方调入页面



页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区





页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区

用于存放对换页面的对换区通常用连续分配方式

用于存放文件的文件区通常用离散分配方式



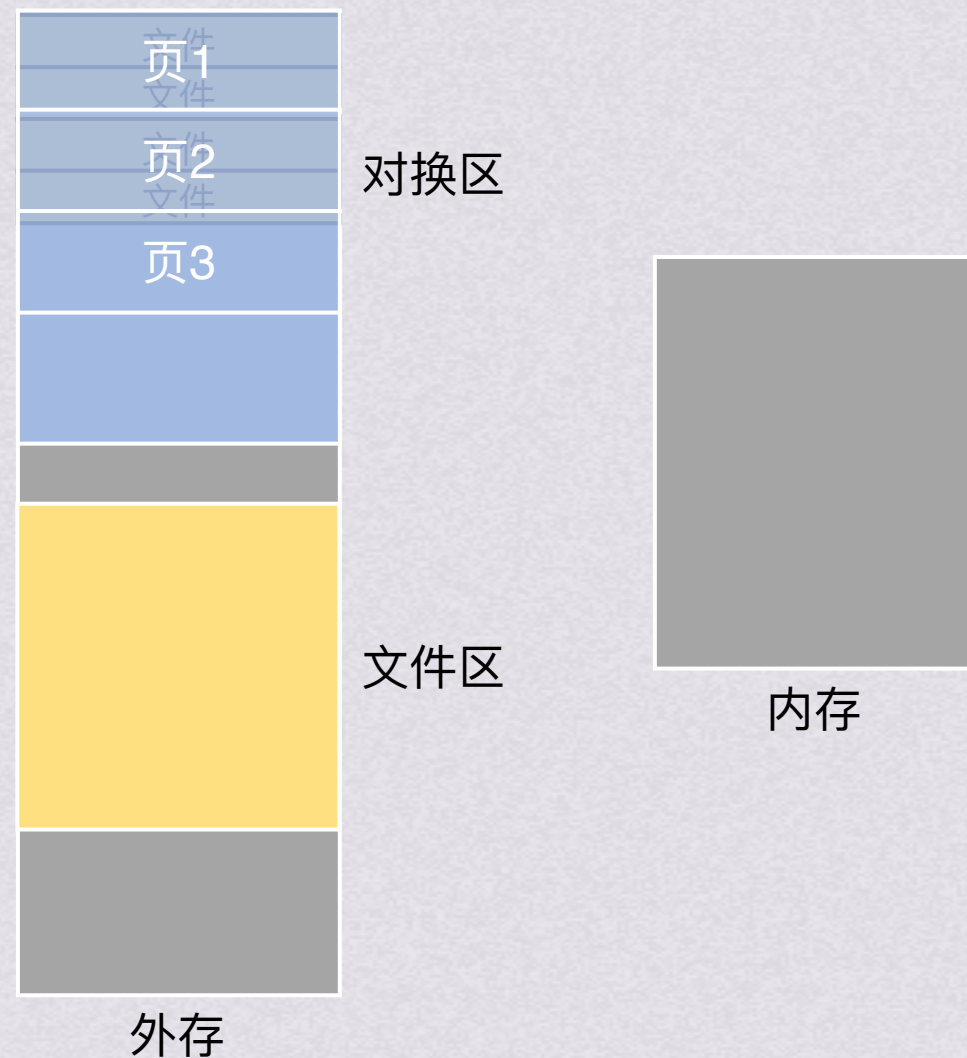


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
 在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区



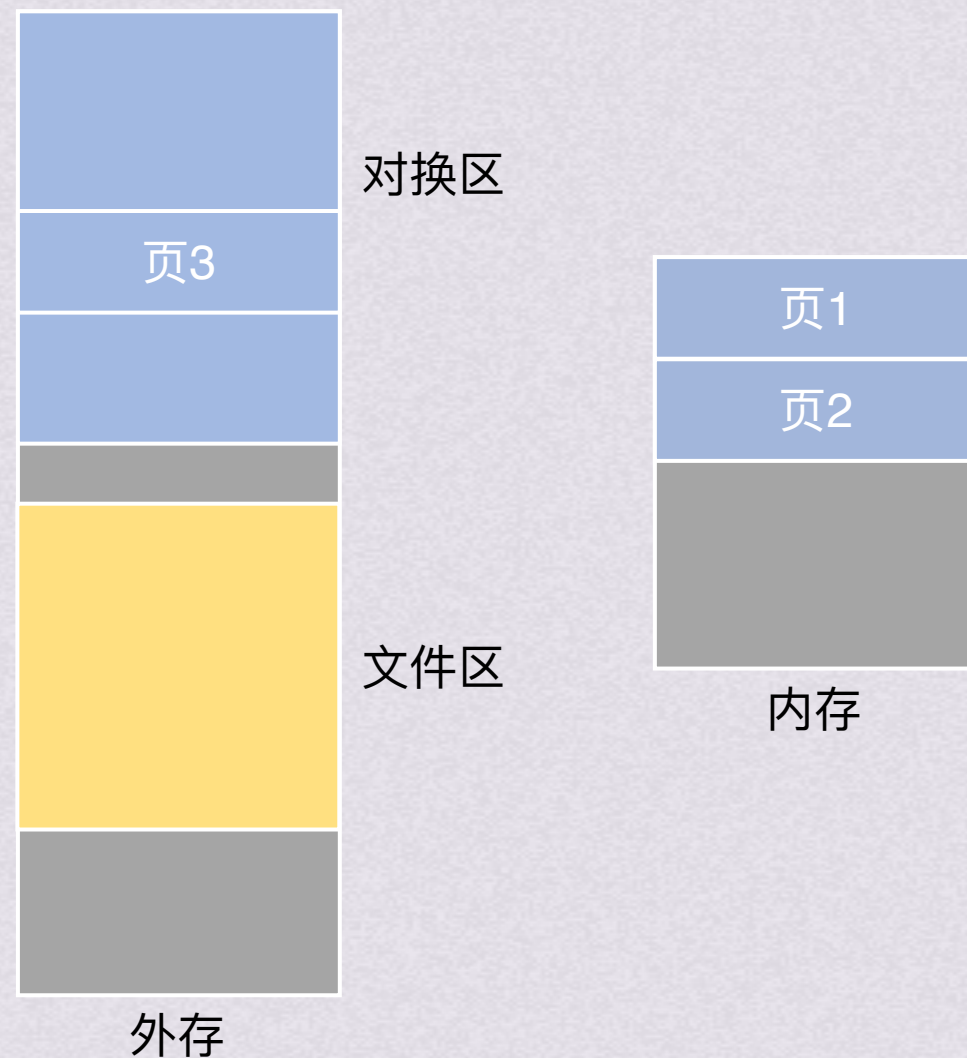


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
 在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区



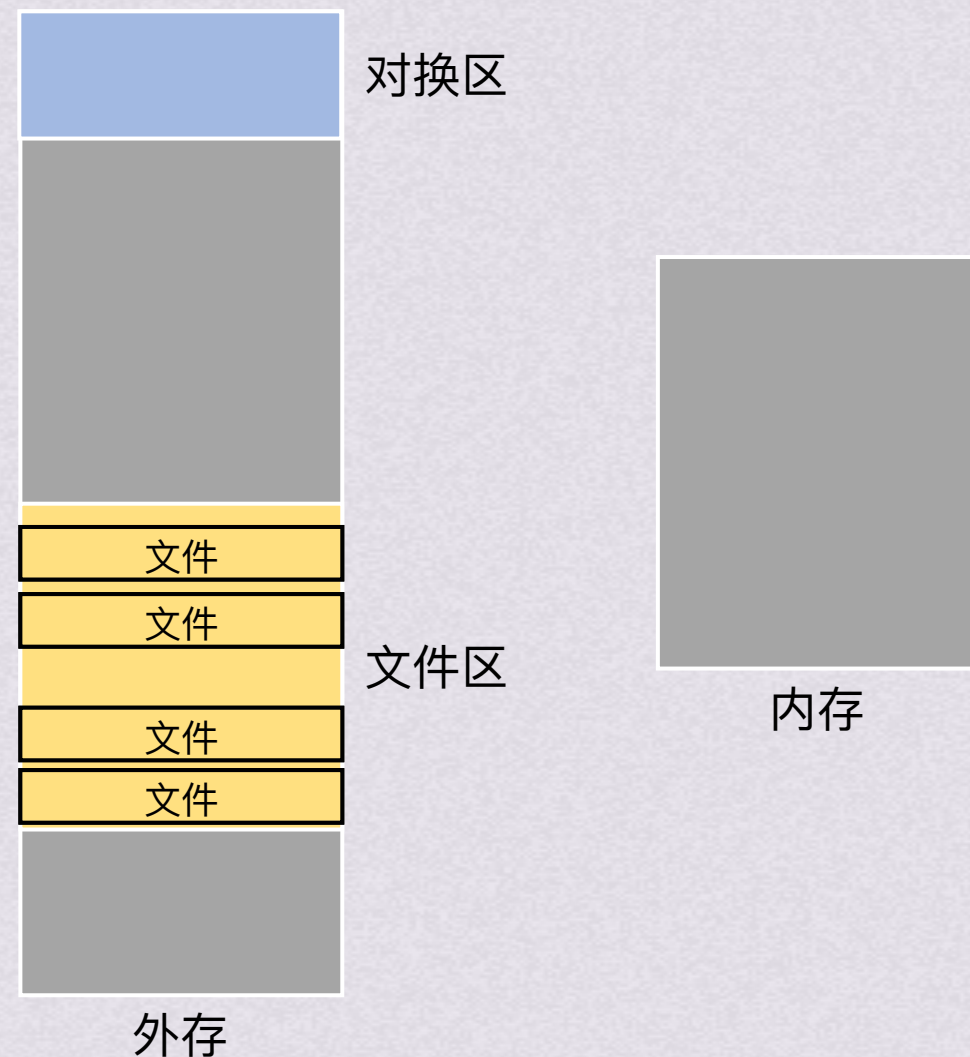


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入



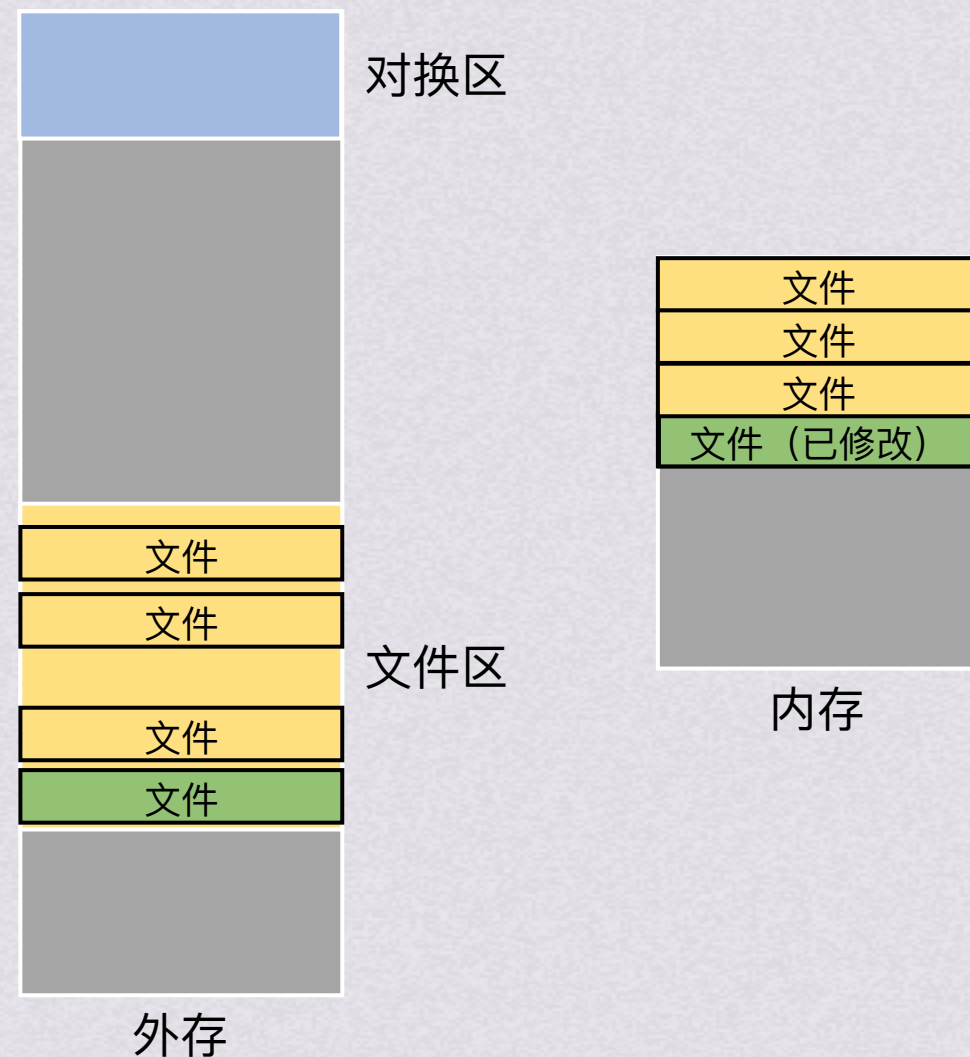


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
 在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入



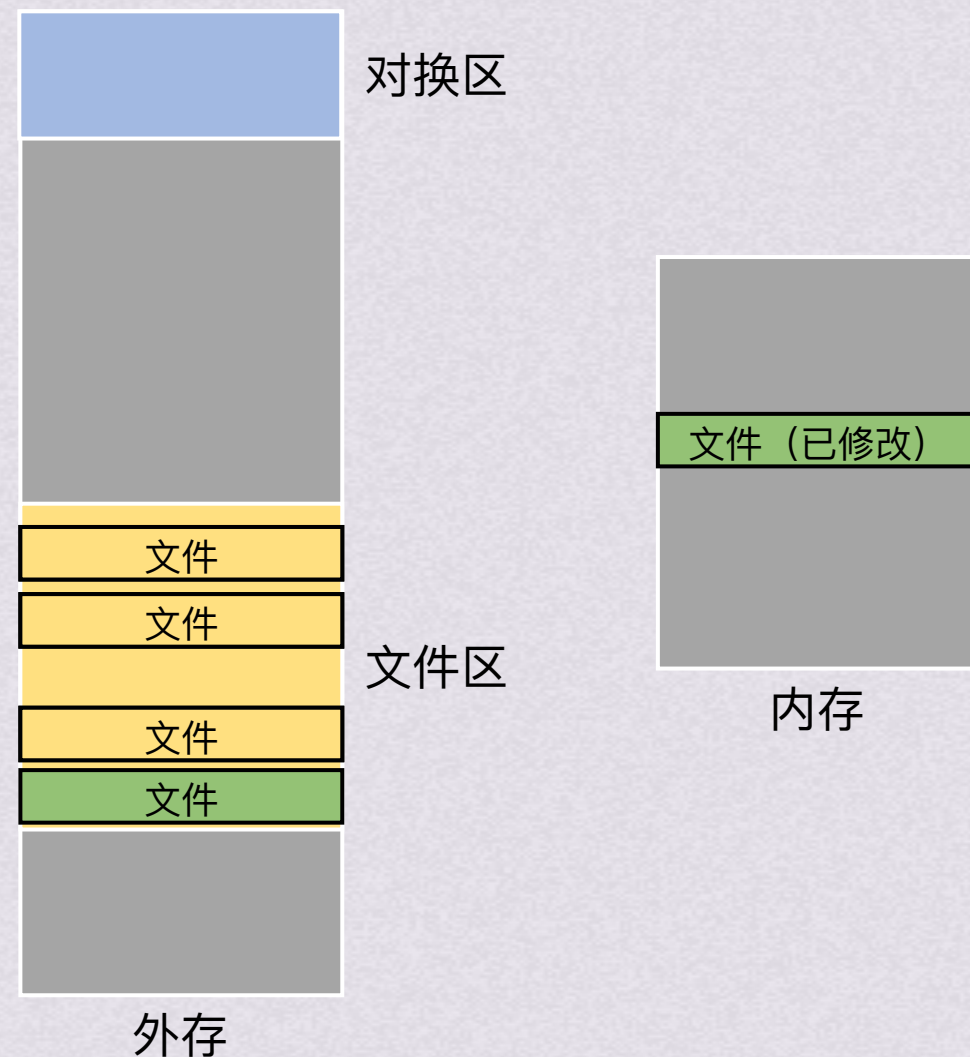


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入



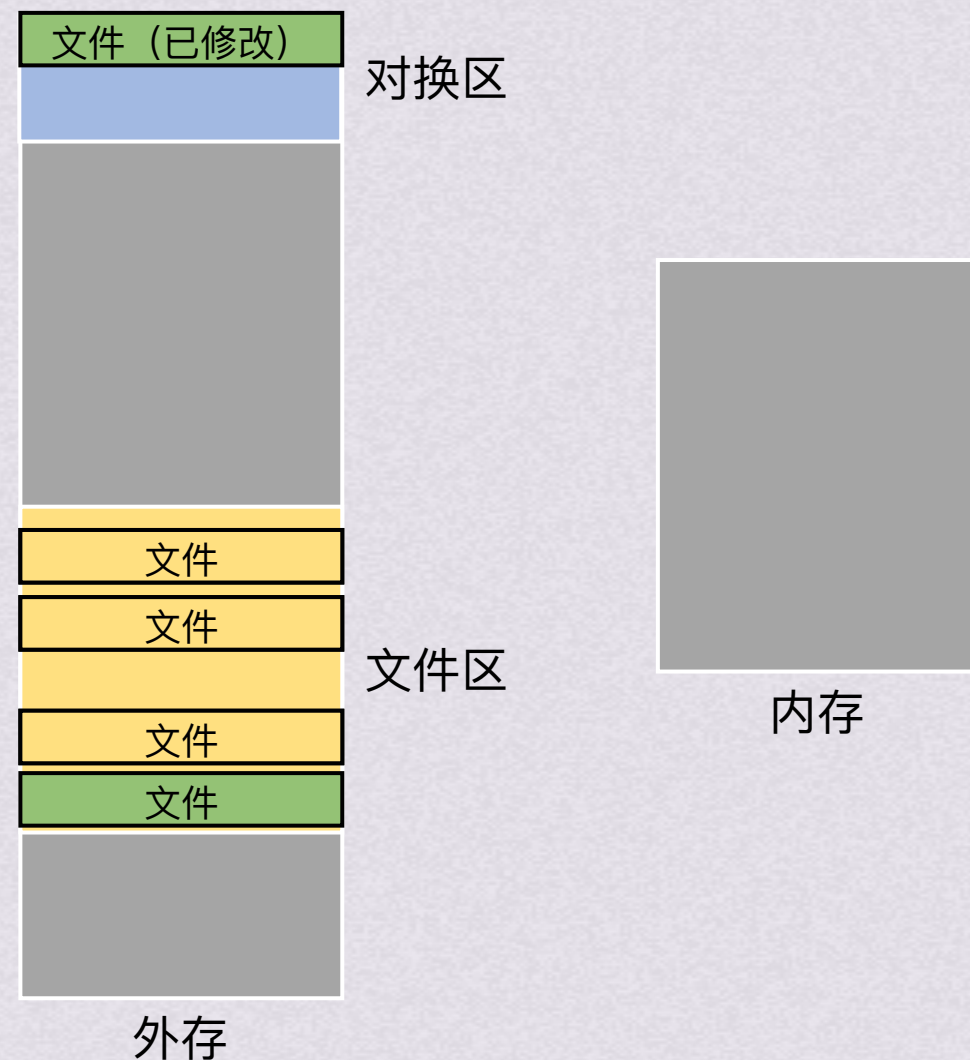


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
 在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入



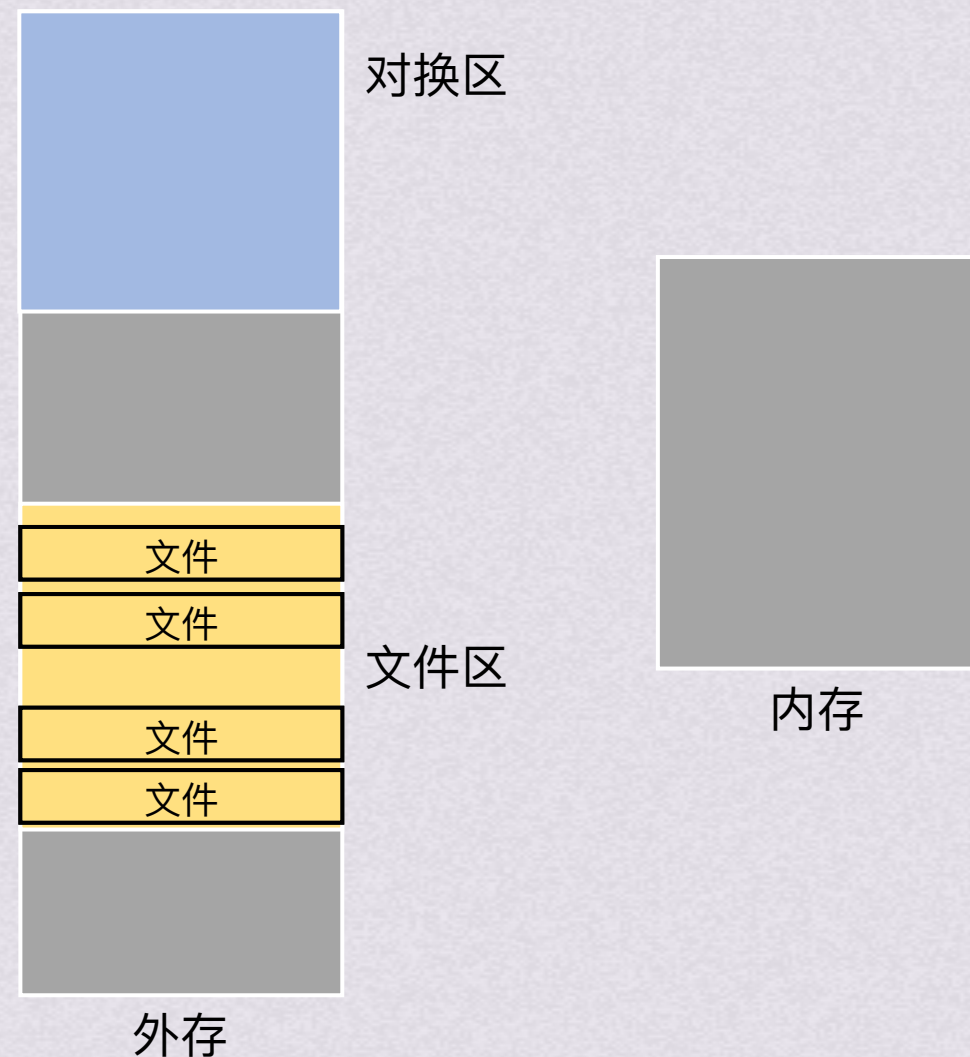


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
 用于存放对换页面的对换区通常用连续分配方式
 用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入
- 3.UNIX方式：凡是未运行过的页面,都从文件区调入。运行过被换出的页面放在对换区,下次从对换区调入。并且允许页面共享,进程请求的页面可能已被其他进程调入内存,此时就无需从对换区调入



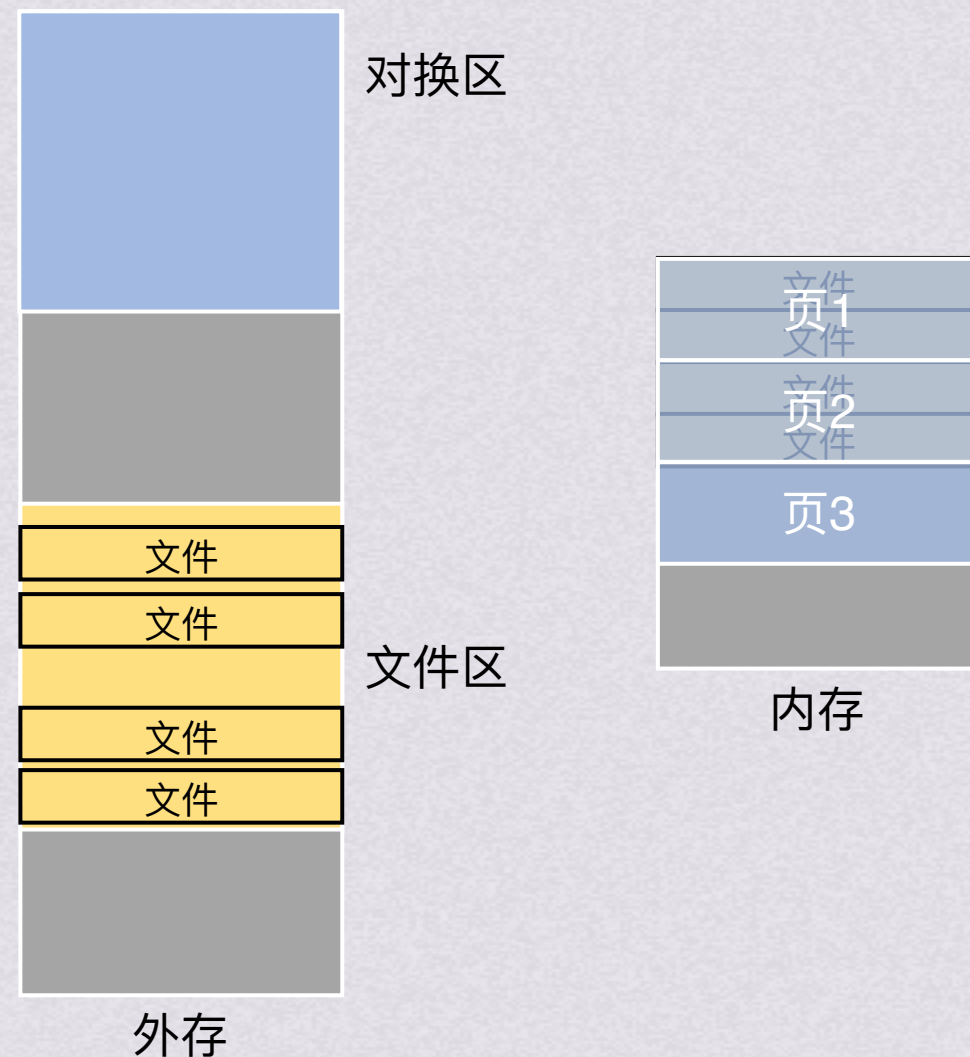


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
用于存放对换页面的对换区通常用连续分配方式
用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入
- 3.UNIX方式: 凡是未运行过的页面,都从文件区调入。运行过被换出的页面放在对换区,下次从对换区调入。并且允许页面共享,进程请求的页面可能已被其他进程调入内存,此时就无需从对换区调入



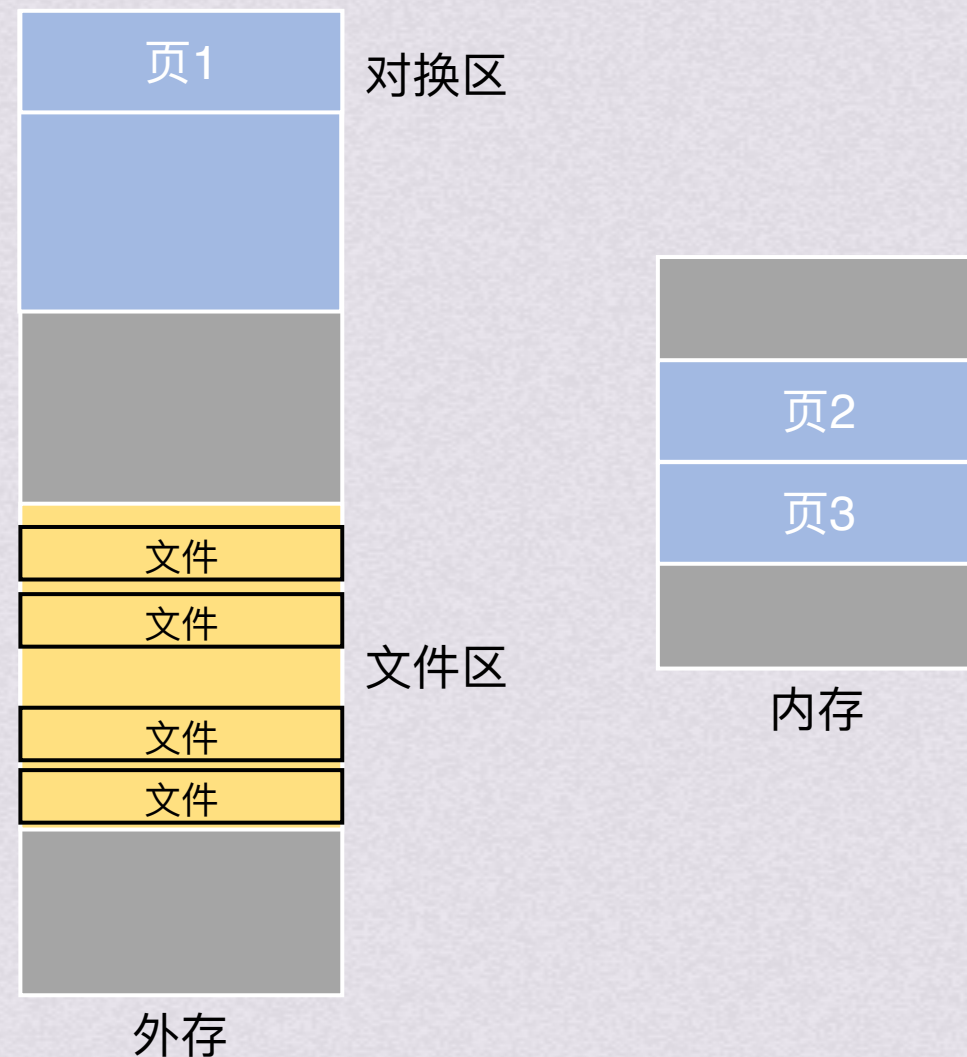


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
用于存放对换页面的对换区通常用连续分配方式
用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入
- 3.UNIX方式: 凡是未运行过的页面,都从文件区调入。运行过被换出的页面放在对换区,下次从对换区调入。并且允许页面共享,进程请求的页面可能已被其他进程调入内存,此时就无需从对换区调入



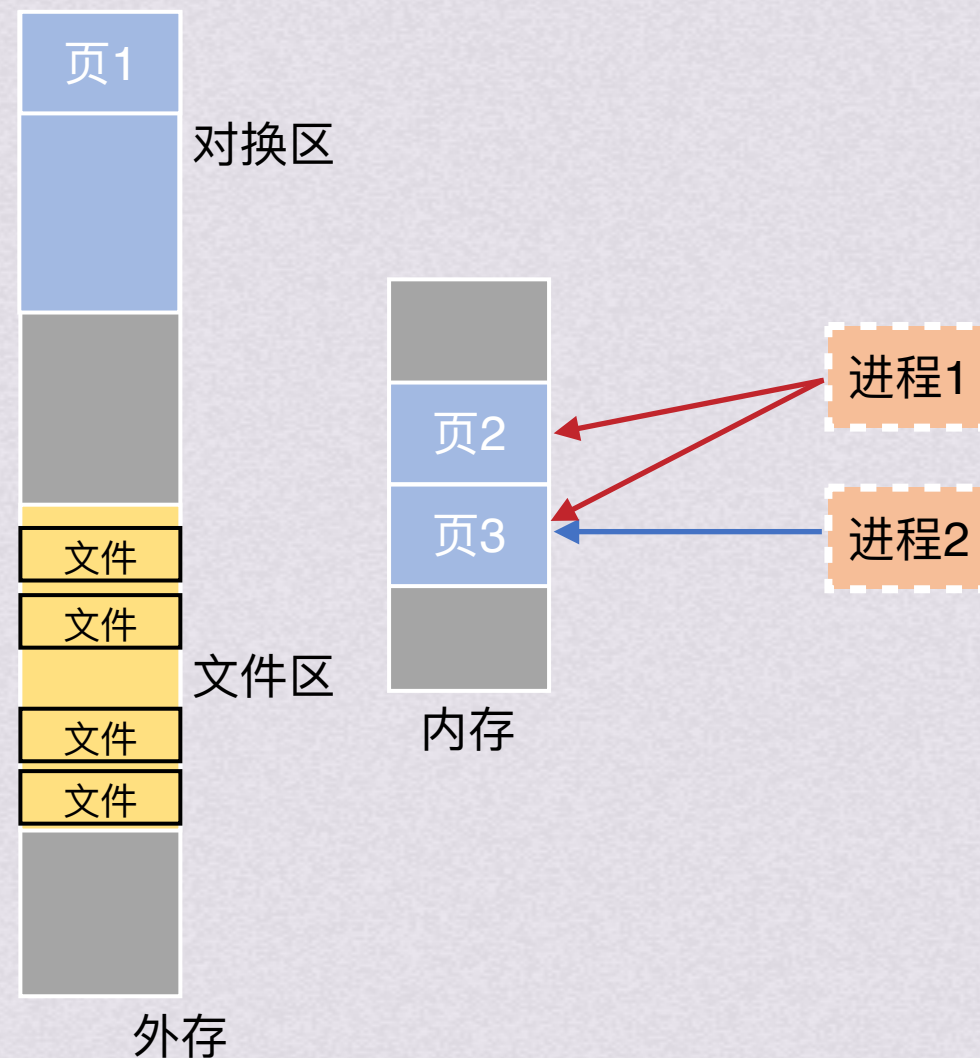


页面调入策略

从什么地方调入页面

请求分页系统中的外存分为两部分,对换区和文件区
用于存放对换页面的对换区通常用连续分配方式
用于存放文件的文件区通常用离散分配方式

- 1.若对换区足够有充足空间,此时可以全部从对换区调入所需页面。
在进程运行前,需将与该进程有关的文件从文件区拷贝到对换区
- 2.若系统缺少足够的对换区空间,这时凡是不会被修改的文件,都直接从文件区调入,这样换出的时候不用写回磁盘。对于可能被修改的部分,换出的时候调到对换区,以后需要再从对换区调入
- 3.UNIX方式：凡是未运行过的页面,都从文件区调入。运行过被换出的页面放在对换区,下次从对换区调入。并且允许页面共享,进程请求的页面可能已被其他进程调入内存,此时就无需从对换区调入





页面调入策略

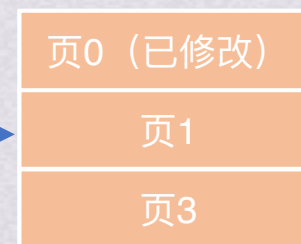
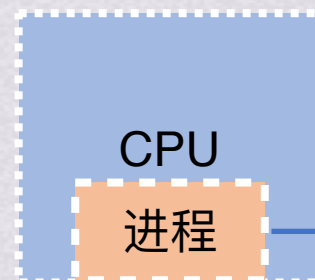
页面调入的过程

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件

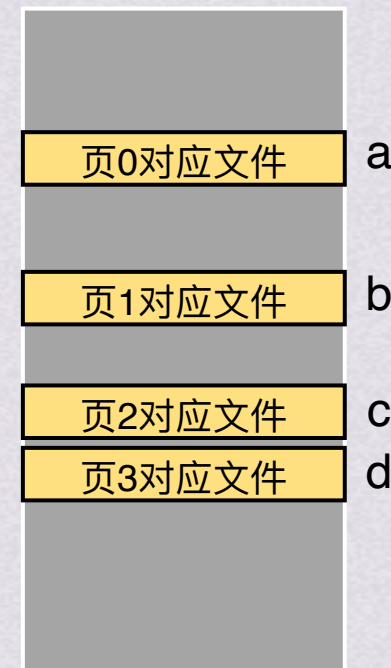
页面调入的过程

(内存中的)				
	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

程序要访问的页面没在内存的时候就向CPU发出一缺页中断

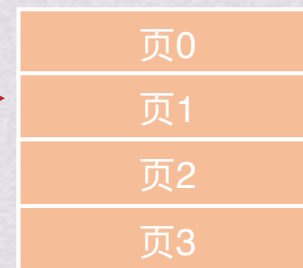


内存



外存

当前执行位置

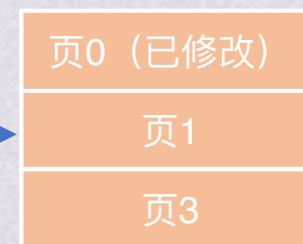
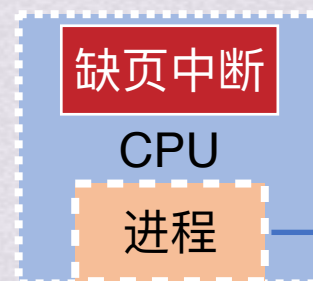


程序空间的分页情况

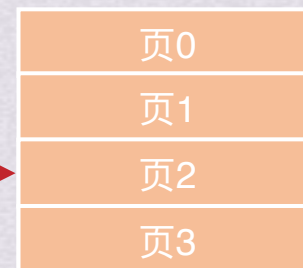
页面调入的过程

(内存中的)				
	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

程序要访问的页面没在内存的时候就向CPU发出一缺页中断

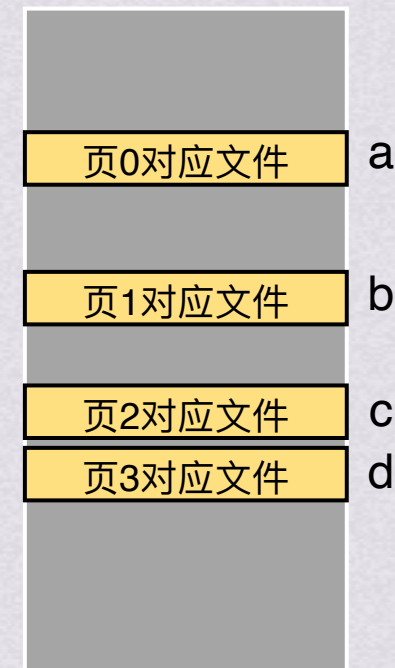


内存



当前执行位置

程序空间的分页情况



外存



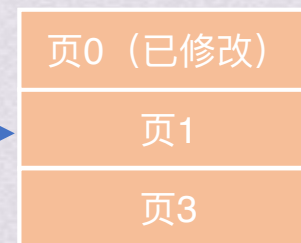
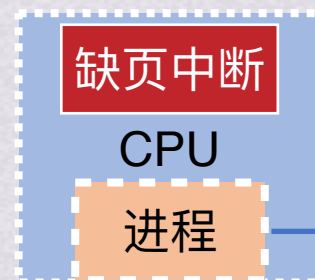
页面调入策略

页面调入的过程

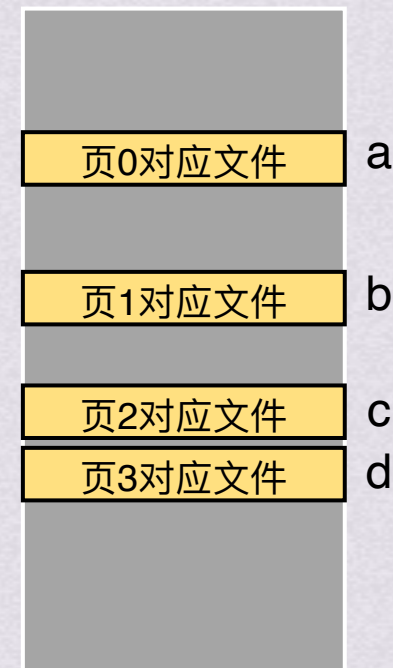
(内存中的)

	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

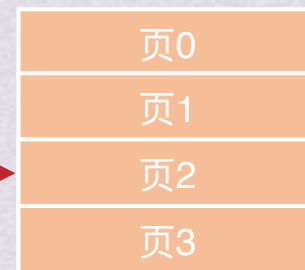
程序要访问的页面没在内存的时候就向CPU发出一缺页中断
 中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存



内存



外存



当前执行位置 →

程序空间的分页情况

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



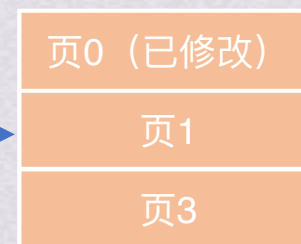
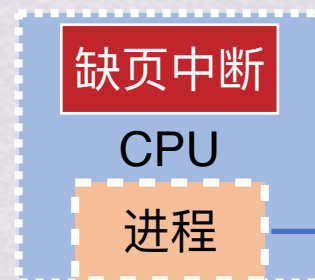
页面调入策略

页面调入的过程

(内存中的)

	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

程序要访问的页面没在内存的时候就向CPU发出一缺页中断
 中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存



内存



外存



当前执行位置

程序空间的分页情况

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



页面调入策略

页面调入的过程

(内存中的)

	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

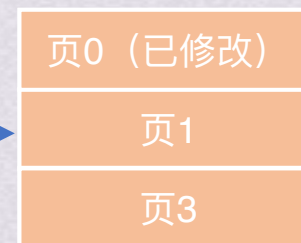
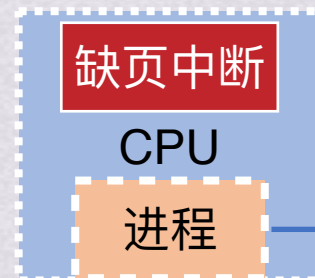
程序要访问的页面没在内存的时候就向CPU发出一缺页中断
 中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存

如果内存空,直接调入内存;

如果内存满,根据算法选择一页调出内存

调出内存时,如果页面被修改,要写回外存;

如果页面没被修改,可以直接覆盖



内存



当前执行位置

程序空间的分页情况



外存

{ 什么时候调入页面
 { 从什么地方调入页面
 { 硬件支持
 { 内存分配
 { 概述
 { 页面置换算法
 { 抖动与工作集
 { 内存映射文件



页面调入策略

页面调入的过程

(内存中的)				
	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

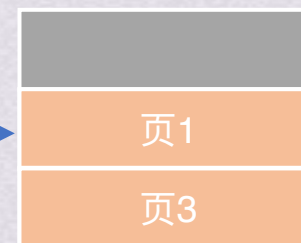
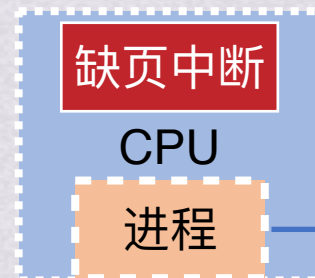
程序要访问的页面没在内存的时候就向CPU发出一缺页中断
 中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存

如果内存空,直接调入内存;

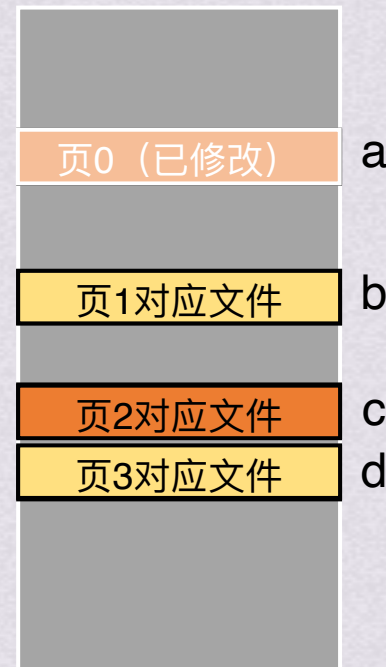
如果内存满,根据算法选择一页调出内存

调出内存时,如果页面被修改,要写回外存;

如果页面没被修改,可以直接覆盖



内存



外存



当前执行位置 →

程序空间的分页情况

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



页面调入策略

页面调入的过程

(内存中的)				
	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

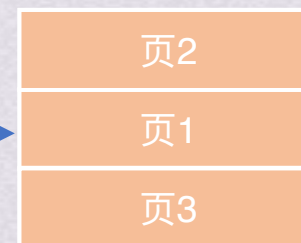
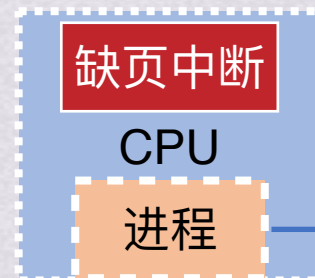
程序要访问的页面没在内存的时候就向CPU发出一缺页中断
 中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存

如果内存空,直接调入内存;

如果内存满,根据算法选择一页调出内存

调出内存时,如果页面被修改,要写回外存;

如果页面没被修改,可以直接覆盖

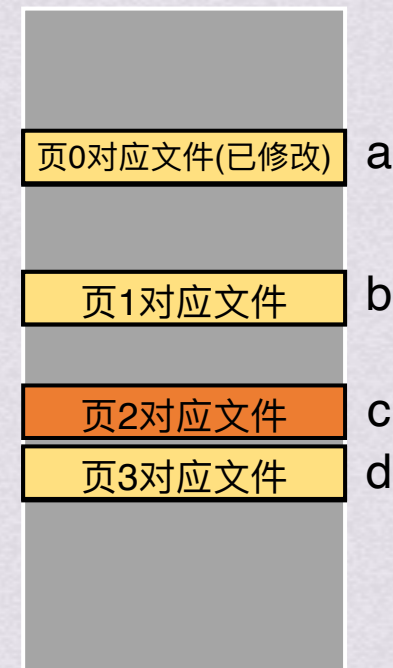


内存



当前执行位置

程序空间的分页情况



外存

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



页面调入策略

页面调入的过程

(内存中的)

	页表	有效位	外存地址	修改位
0	0	1	a	1
1	1	1	b	0
2	-	0	c	0
3	2	1	d	0

程序要访问的页面没在内存的时候就向CPU发出一缺页中断

中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存

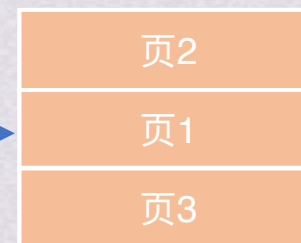
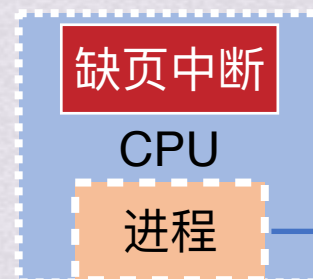
如果内存空,直接调入内存;

如果内存满,根据算法选择一页调出内存

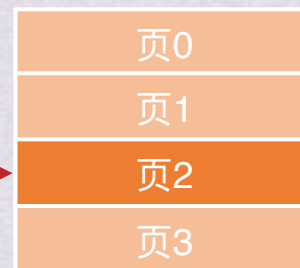
调出内存时,如果页面被修改,要写回外存;

如果页面没被修改,可以直接覆盖

修改页表中的相应表项,存在位改为1,并写入快表中(有的话)

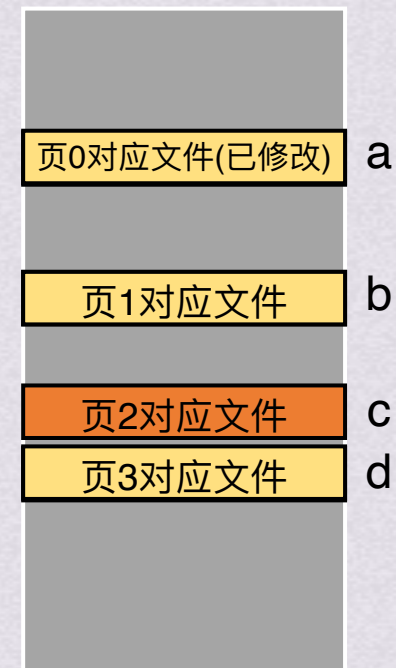


内存



当前执行位置

程序空间的分页情况



外存



页面调入策略

页面调入的过程

(内存中的)

	页表	有效位	外存地址	修改位
0	-	0	a	1
1	1	1	b	0
2	0	1	c	0
3	2	1	d	0

程序要访问的页面没在内存的时候就向CPU发出一缺页中断

中断处理程序首先保留CPU环境,分析中断原因后转入缺页中
断处理程序,查找页表找到外存物理块,准备调入内存

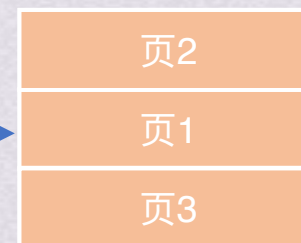
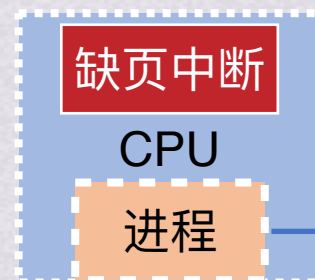
如果内存空,直接调入内存;

如果内存满,根据算法选择一页调出内存

调出内存时,如果页面被修改,要写回外存;

如果页面没被修改,可以直接覆盖

修改页表中的相应表项,存在位改为1,并写入快表中(有的话)

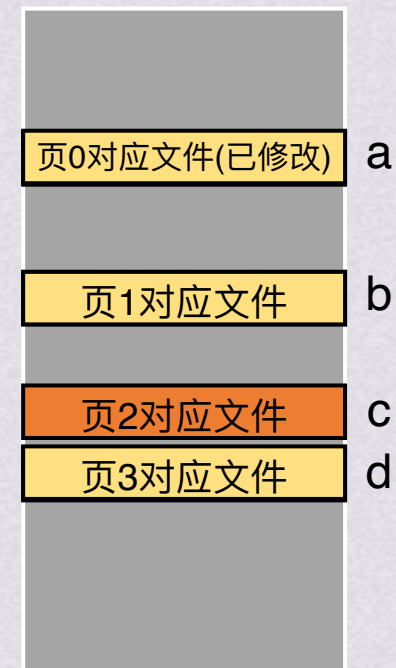


内存



当前执行位置

程序空间的分页情况



外存

什么时候调入页面
从什么地方调入页面
硬件支持
内存分配
概述
页面置换算法
抖动与工作集
内存映射文件



页面调入策略

页面调入的过程

(内存中的)

	页表	有效位	外存地址	修改位
0	-	0	a	1
1	1	1	b	0
2	0	1	c	0
3	2	1	d	0

程序要访问的页面没在内存的时候就向CPU发出一缺页中断

中断处理程序首先保留CPU环境,分析中断原因后转入缺页中断处理程序,查找页表找到外存物理块,准备调入内存

如果内存空,直接调入内存;

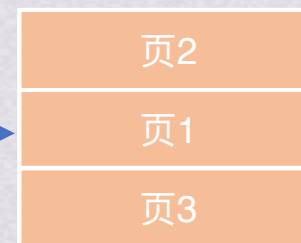
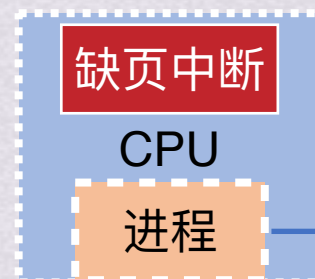
如果内存满,根据算法选择一页调出内存

调出内存时,如果页面被修改,要写回外存;

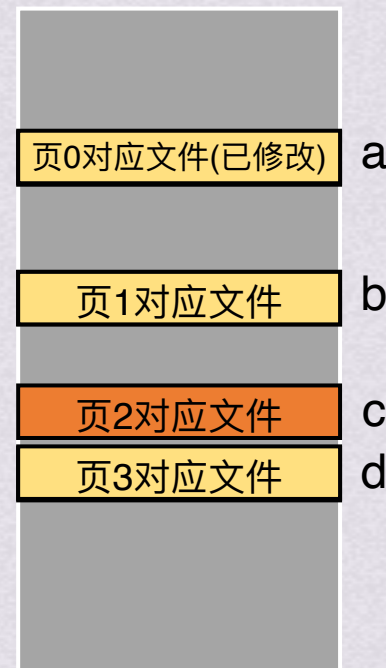
如果页面没被修改,可以直接覆盖

修改页表中的相应表项,存在位改为1,并写入快表中(有的话)

缺页调入内存后,利用修改后的页表形成所要访问数据的物理地址,再去访问内存数据。整个页面的调入过程对用户是透明的



内存



外存



当前执行位置

程序空间的分页情况

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



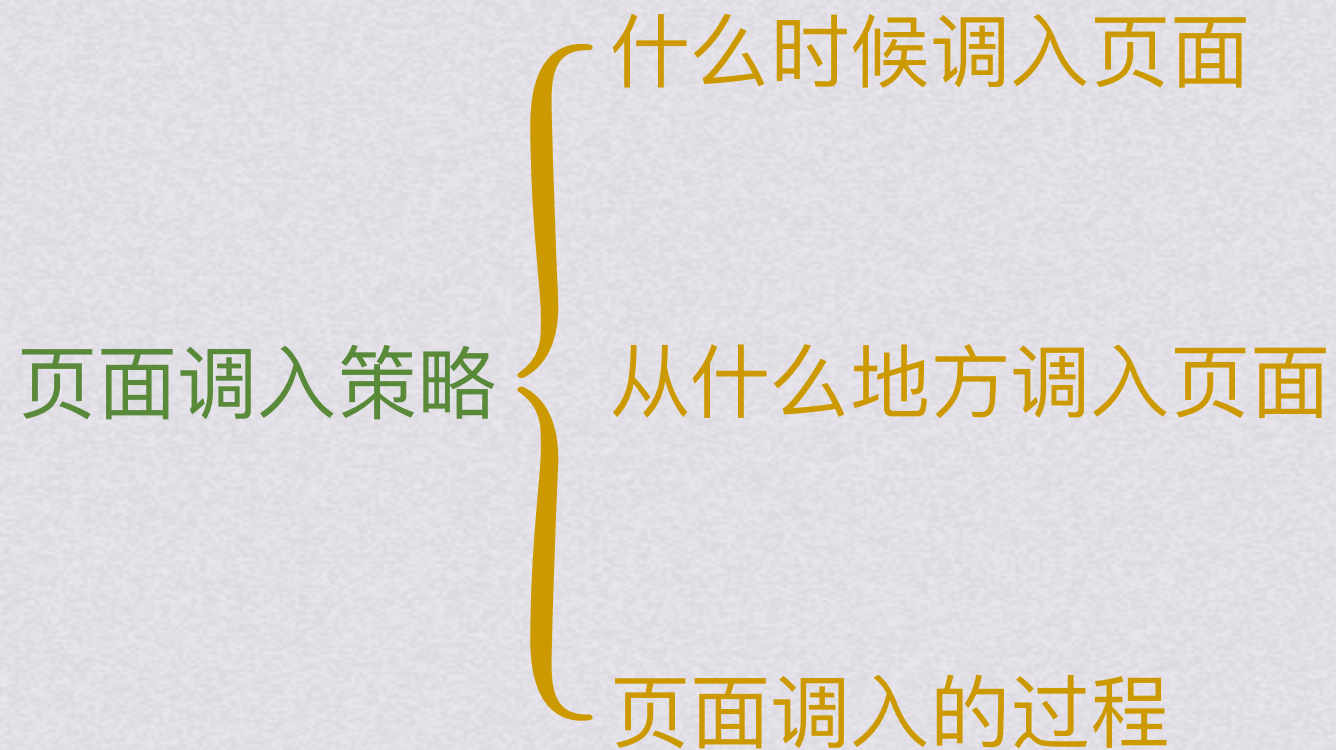
页面调入策略

页面调入的过程

- 什么时候调入页面
- 从什么地方调入页面
- 硬件支持
- 内存分配
- 概述
- 页面置换算法
- 抖动与工作集
- 内存映射文件



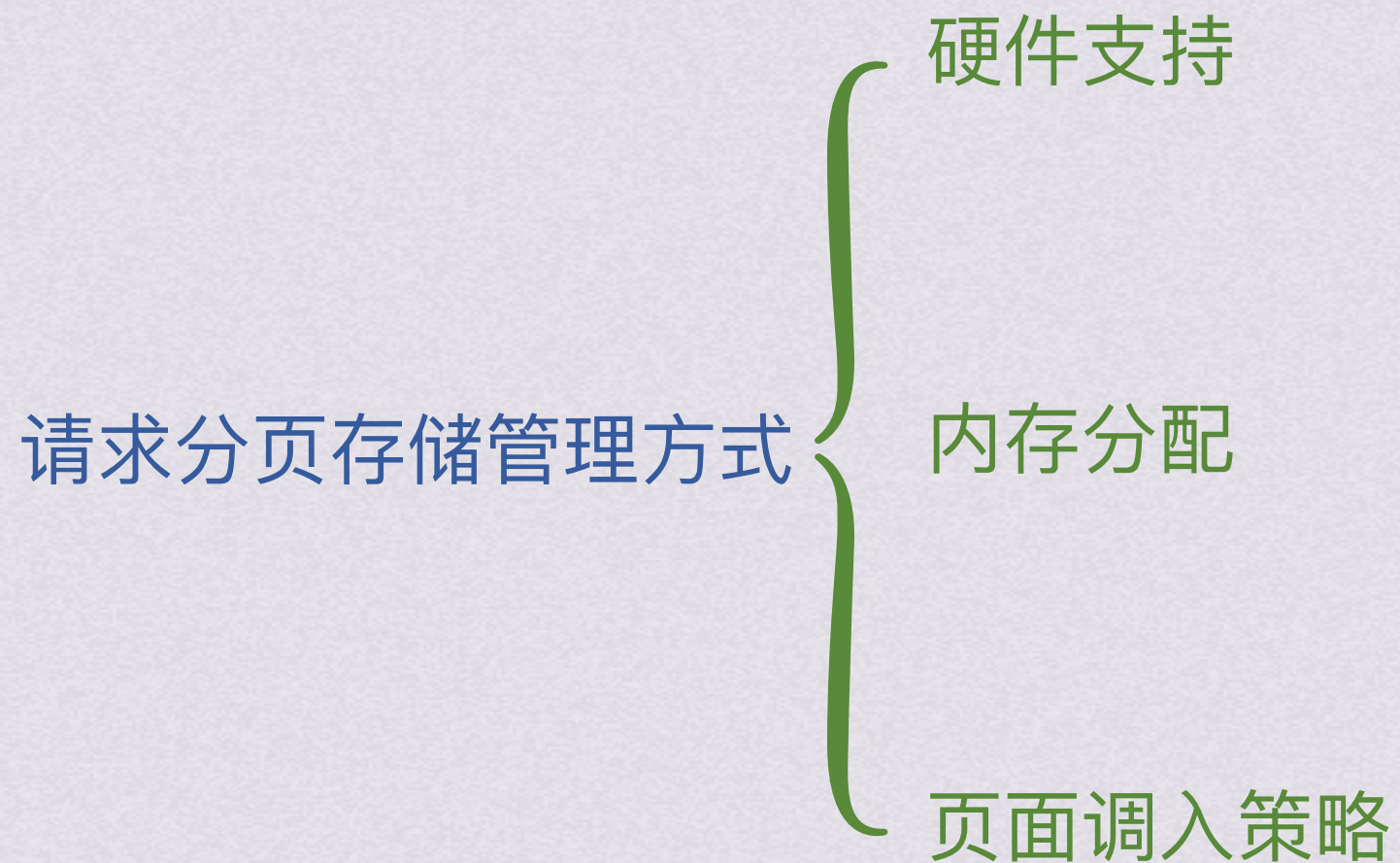
请求分页存储管理方式





请求分页存储管理方式

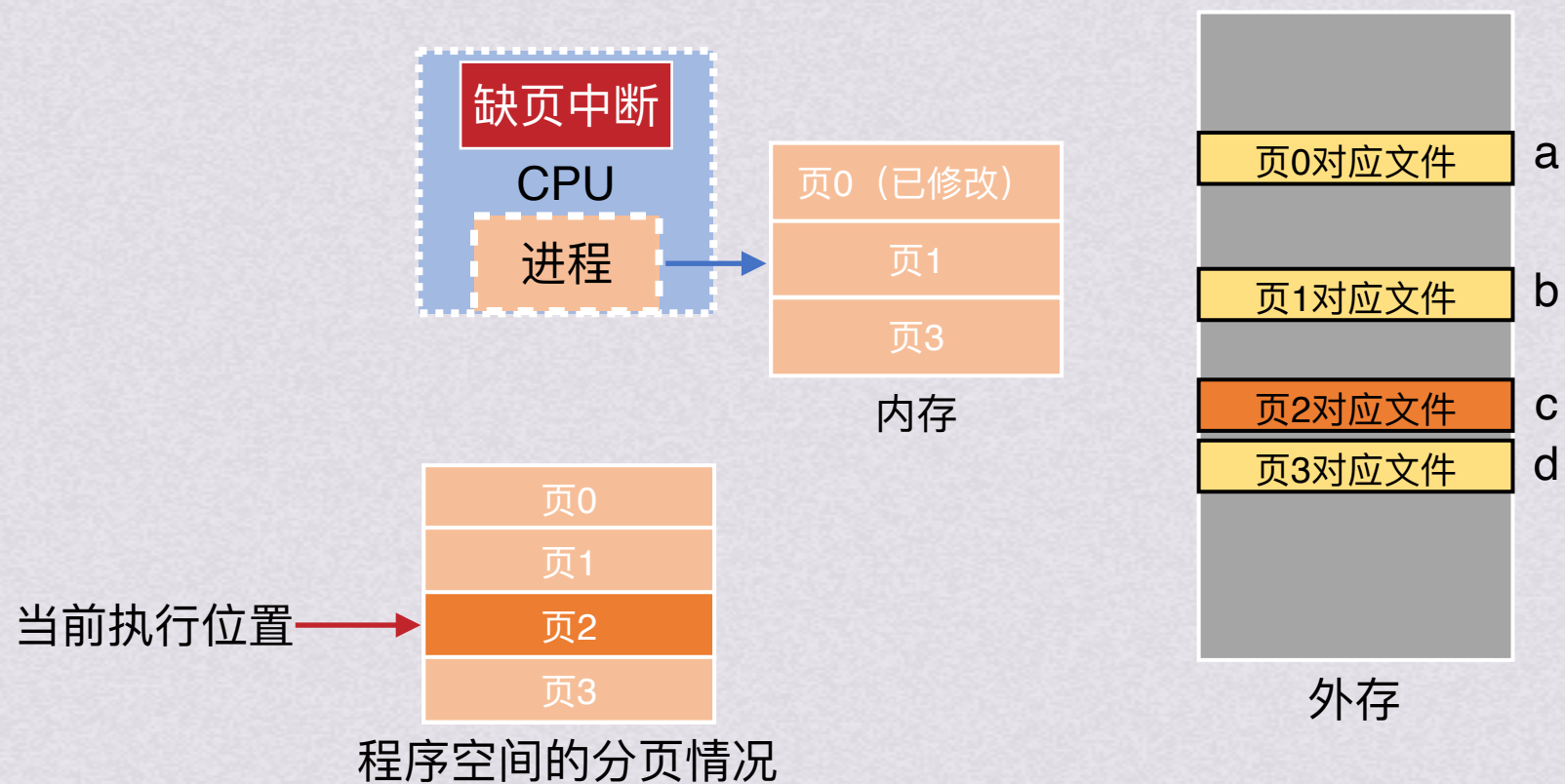
页面调入策略





请求分页存储管理方式

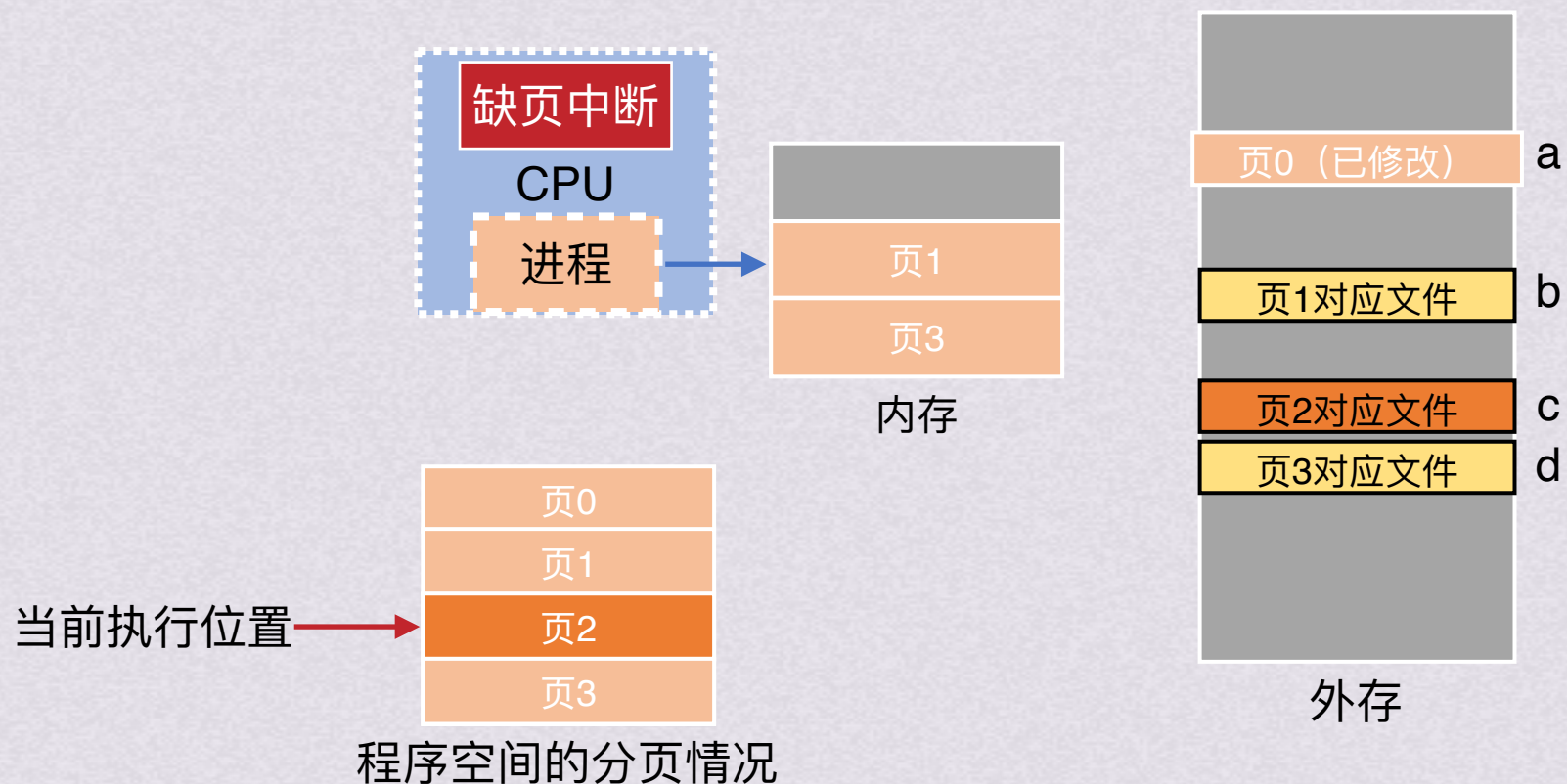
页面置换算法





页面置换算法

刚才我们手动选择了页0调出写回外存,那么到底是什么算法来决定谁被换出呢
就是由页面置换算法来决定的





页面置换算法

进程运行过程中,若其要访问的页面不在内存,需要把它们调入内存。但若内存已满,系统必须根据页面置换算法来决定将那个页面换出

最佳OPT置换算法

先进先出FIFO置换算法

最近最久未使用LRU置换算法

时钟CLOCK置换算法

改进型CLOCK置换算法



页面置换算法

最佳OPT置换算法

- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

最佳OPT置换算法

活在理论里的算法,每次选择未来最长时间不会被访问的页面进行淘汰,因为无法预知未来,所以暂时无法实现

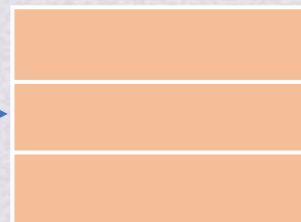
假设某进程有3物理块,并且访问页面号的顺序如下

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

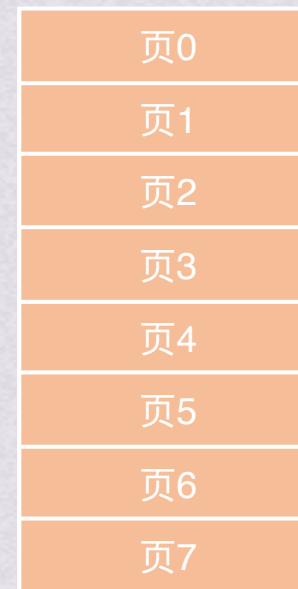
当前访问页面

进程

获得的页框 (物理块)
也就是分配到的物理内存



内存



外存

- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

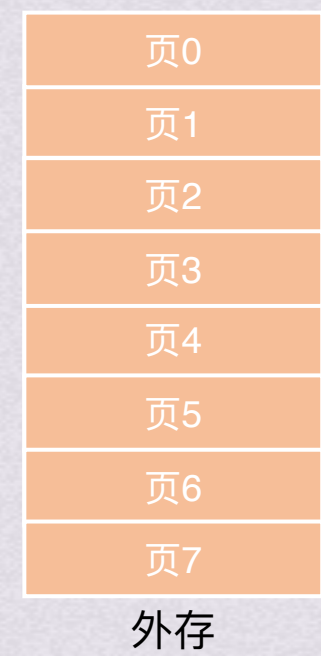
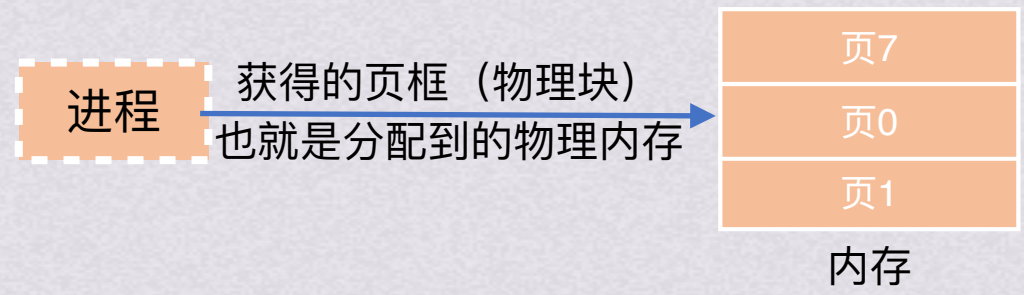
最佳OPT置换算法

活在理论里的算法,每次选择未来最长时间不会被访问的页面进行淘汰,因为无法预知未来,所以暂时无法实现

假设某进程有3物理块,并且访问页面号的顺序如下

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

一共缺页9次
发生页面置换6次



- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

最佳OPT置换算法

- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



先进先出FIFO置换算法



页面置换算法

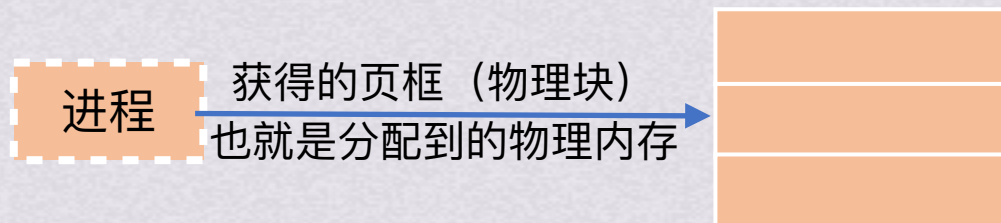
先进先出FIFO置换算法

每次淘汰在内存里驻留最久的页面，实现简单，把调入内存的页面按先后顺序链接成队列，设置一指针指向最老的页面即可

假设某进程有3物理块,并且访问页面号的顺序如下

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

当前访问页面



页0
页1
页2
页3
页4
页5
页6
页7

外存



页面置换算法

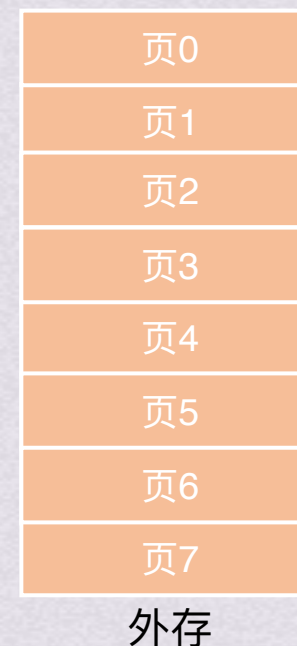
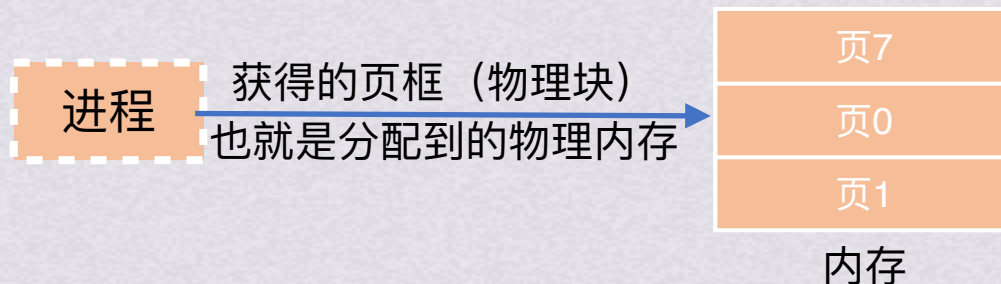
先进先出FIFO置换算法

每次淘汰在内存里驻留最久的页面，实现简单，把调入内存的页面按先后顺序链接成队列，设置一指针指向最老的页面即可

假设某进程有3物理块,并且访问页面号的顺序如下

7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1

一共缺页15次
发生页面置换12次



页面置换算法

先进先出FIFO置换算法

每次淘汰在内存里驻留最久的页面，实现简单，把调入内存的页面按先后顺序链接成队列，设置一指针指向最老的页面即可

FIFO算法可能发生Belady异常

Belady异常：为进程分配的物理块增多，缺页次数不减反增的异常现象

访问页面	3	2	1	0	3	2	4	3	2	1	0	4
物理块1	3	3	3	0	0	0	4			4	4	
物理块2		2	2	2	3	3	3			1	1	
物理块3			1	1	1	2	2			2	0	
缺页否	✓	✓	✓	✓	✓	✓	✓			✓	✓	
物理块1	3	3	3	3			4	4	4	4	0	0
物理块2		2	2	2			2	3	3	3	3	4
物理块3			1	1			1	1	2	2	2	2
物理块4				0			0	0	0	1	1	1
缺页否	✓	✓	✓	✓			✓	✓	✓	✓	✓	✓

- 最佳OPT置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

先进先出FIFO置换算法

- 最佳OPT置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

最近最久未使用LRU置换算法

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

最近最久未使用LRU置换算法

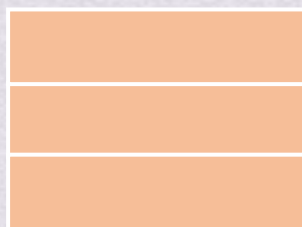
因为无法预测未来页面的使用情况，所以只能用过去的情况作未来情况的近似，LRU算法就是每次选择最近最久未被使用的页面淘汰

假设某进程有3物理块,并且访问页面号的顺序如下

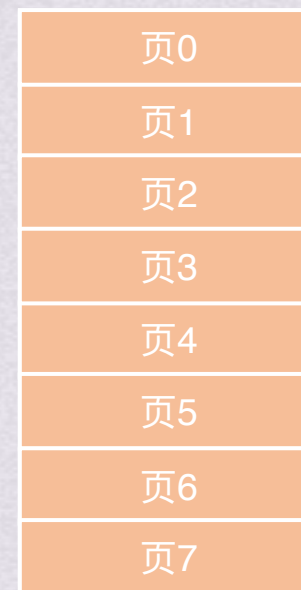
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1



当前访问页面



内存



外存

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

最近最久未使用LRU置换算法

因为无法预测未来页面的使用情况，所以只能用过去的情况作未来情况的近似，LRU算法就是每次选择最近最久未被使用的页面淘汰

假设某进程有3物理块,并且访问页面号的顺序如下

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

↑
当前访问页面

这时候内存已满，需要换出页面
选择2,0,1中，最久没被使用过的
此处是1

页2
页0
页3

内存

页0
页1
页2
页3
页4
页5
页6
页7

外存

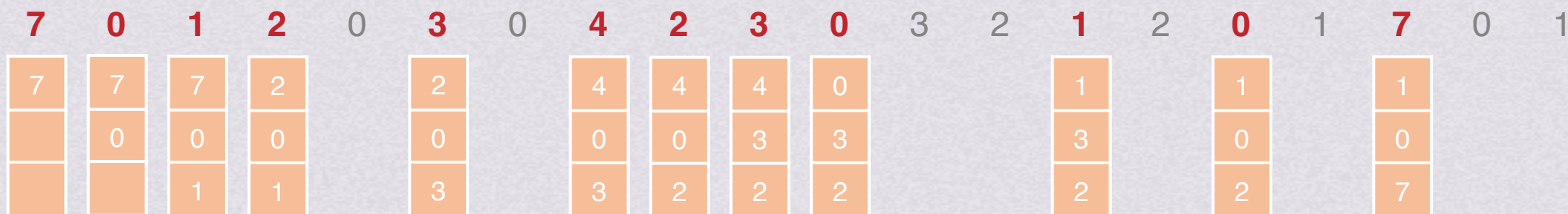


页面置换算法

最近最久未使用LRU置换算法

因为无法预测未来页面的使用情况，所以只能用过去的情况作未来情况的近似，LRU算法就是每次选择最近最久未被使用的页面淘汰

假设某进程有3物理块,并且访问页面号的顺序如下



最终整体置换情况如上

缺页12次

置换9次

LRU置换算法是一种比较好的算法，但是需要为每个页面设置一个访问字段，记录页面自上次被访问以来经历的时间并且还需要寄存器和栈的硬件支持

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 时钟CLOCK置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

最近最久未使用LRU置换算法



页面置换算法

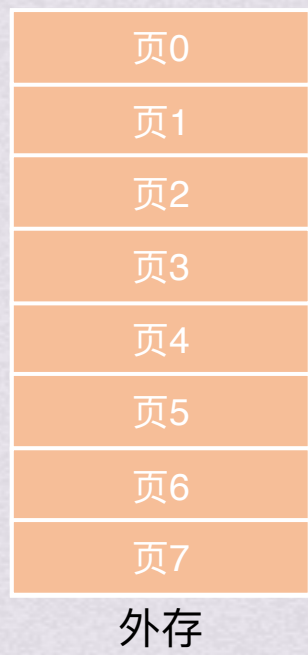
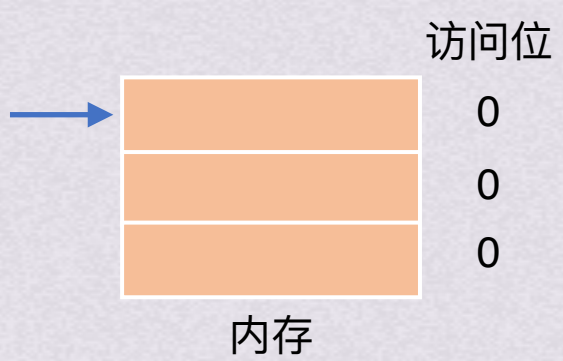
时钟CLOCK置换算法

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件

页面置换算法

时钟CLOCK置换算法

为每个页面设置一个访问位，再把内存中的所有页面通过链接指针链接成一个循环队列。访问某页时更改访问位为1，在淘汰页面时，检查页的访问位，如果是0就换出；如果是1就置0，去检查下一个页面。



- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

时钟CLOCK置换算法

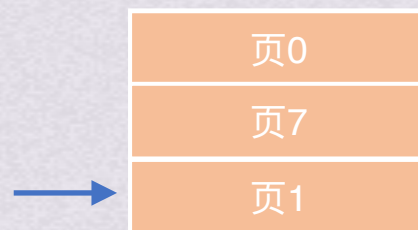
为每个页面设置一个访问位，再把内存中的所有页面通过链接指针链接成一个循环队列。访问某页时更改访问位为1，在淘汰页面时，检查页的访问位，如果是0就换出；如果是1就置0，去检查下一个页面。

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

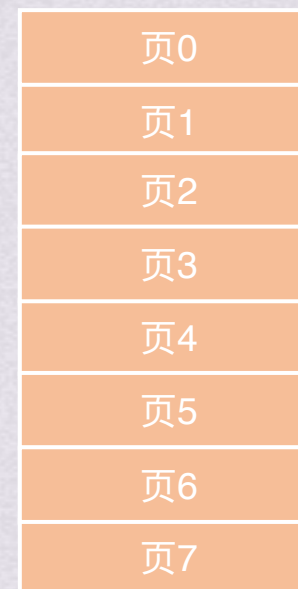
当前访问页面

某一页被替换时，指针会指向被替换页面的下一页

访问位



内存



外存

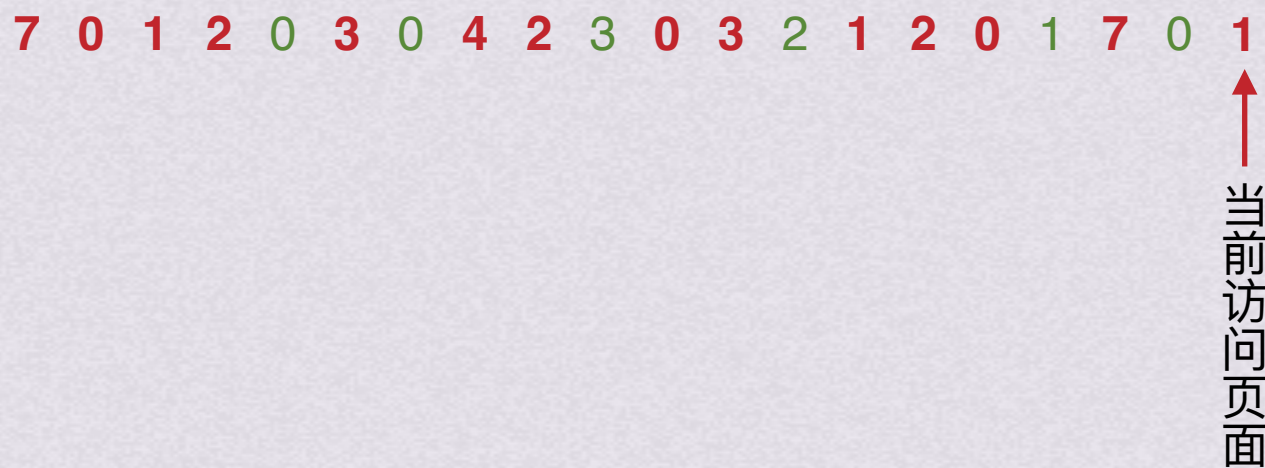
- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



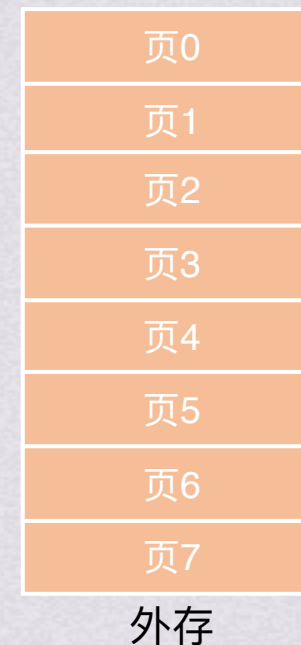
页面置换算法

时钟CLOCK置换算法

为每个页面设置一个访问位，再把内存中的所有页面通过链接指针链接成一个循环队列。访问某页时更改访问位为1，在淘汰页面时，检查页的访问位，如果是0就换出；如果是1就置0，去检查下一个页面。



某一页被替换时，指针会指向被替换页面的下一页



- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

时钟CLOCK置换算法

为每个页面设置一个访问位，再把内存中的所有页面通过链接指针链接成一个循环队列。访问某页时更改访问位为1，在淘汰页面时，检查页的访问位，如果是0就换出；如果是1就置0，去检查下一个页面。

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

缺页14次
换页11次



页面置换算法

时钟CLOCK置换算法

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 改进型CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

改进型CLOCK置换算法

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



页面置换算法

改进型CLOCK置换算法

在CLOCK算法的基础上，新增一个置换代价——修改位，替换的时候就有四类页面了

1类 访问位 $A=0$ ，修改位 $M=0$ ：最近未被访问，未被修改，是最佳淘汰页

2类 访问位 $A=0$ ，修改位 $M=1$ ：最近未被访问，已被修改，是次佳淘汰页

3类 访问位 $A=1$ ，修改位 $M=0$ ：最近已被访问，未被修改，可能再被访问

4类 访问位 $A=1$ ，修改位 $M=1$ ：最近已被访问，已被修改，可能再被访问

算法的具体执行过程如下：

1.从指针当前位置开始，扫描循环队列，寻找 $A=0, M=0$ 的一类页面，把遇到的第一个1类页面作为选中淘汰页。第一次扫描期间不改变访问位 A

2.若1失败，进行第二轮扫描，寻找 $A=0, M=1$ 的二类页面，遇到的第一个2类页面作为淘汰页。第二轮扫描期间，所有扫描过的页面访问位置0

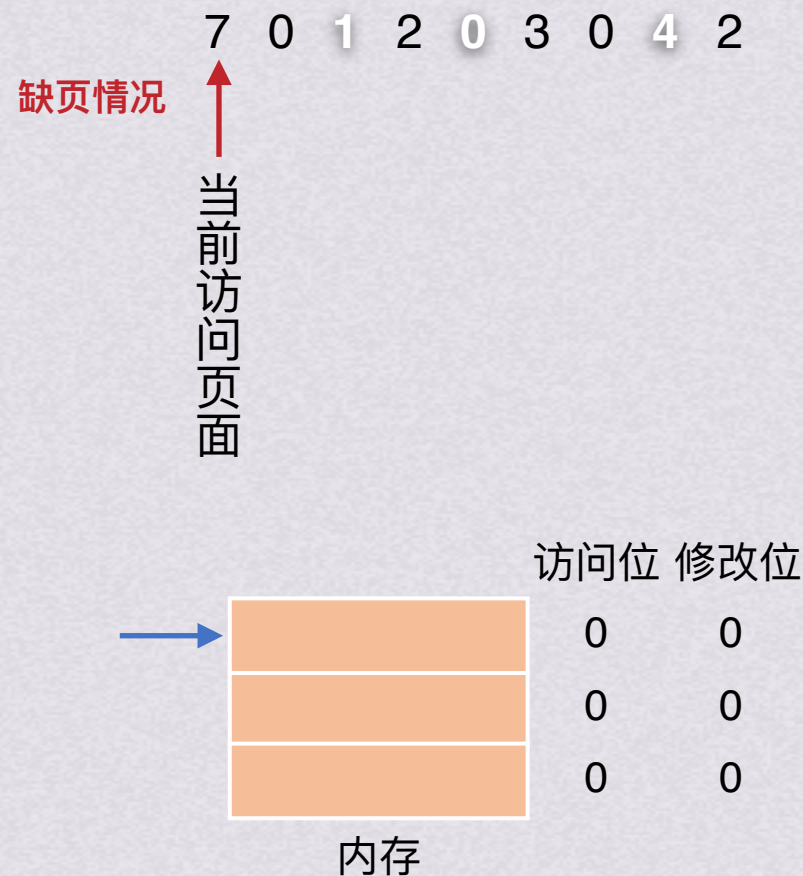
3.若2也失败，重复1，若有需要再重复2，直到找到被淘汰的页



页面置换算法

改进型CLOCK置换算法

在CLOCK算法的基础上，新增一个置换代价——修改位，替换的时候就有四类页面了
假设进程先后访问这些页面，并对其中的1,0,4页面在访问时进行了修改



页0
页1
页2
页3
页4
页5
页6
页7

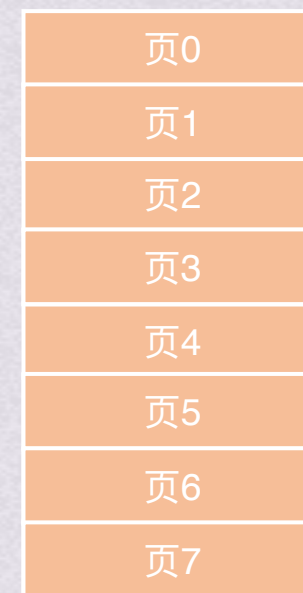
外存



页面置换算法

改进型CLOCK置换算法

在CLOCK算法的基础上，新增一个置换代价——修改位，替换的时候就有四类页面了
假设进程先后访问这些页面，并对其中的1,0,4页面在访问时进行了修改



外存

是否有A=0,M=0页面?



页面置换算法

改进型CLOCK置换算法

在CLOCK算法的基础上，新增一个置换代价——修改位，替换的时候就有四类页面了

假设进程先后访问这些页面，并对其中的1,0,4页面在访问时进行了修改

	7	0	1	2	0	3	0	4	2
缺页情况	✓	✓	✓	✓		✓		✓	✓

改进型CLOCK算法优于简单CLOCK算法的地方在于可以减少磁盘IO操作系数
但是找一个可置换的页可能需要扫描几轮，所以算法本身开销也有所增加



页面置换算法

改进型CLOCK置换算法

- 最佳OPT置换算法
- 先进先出FIFO置换算法
- 最近最久未使用LRU置换算法
- 时钟CLOCK置换算法
- 概述
- 请求分页存储管理方式
- 抖动与工作集
- 内存映射文件



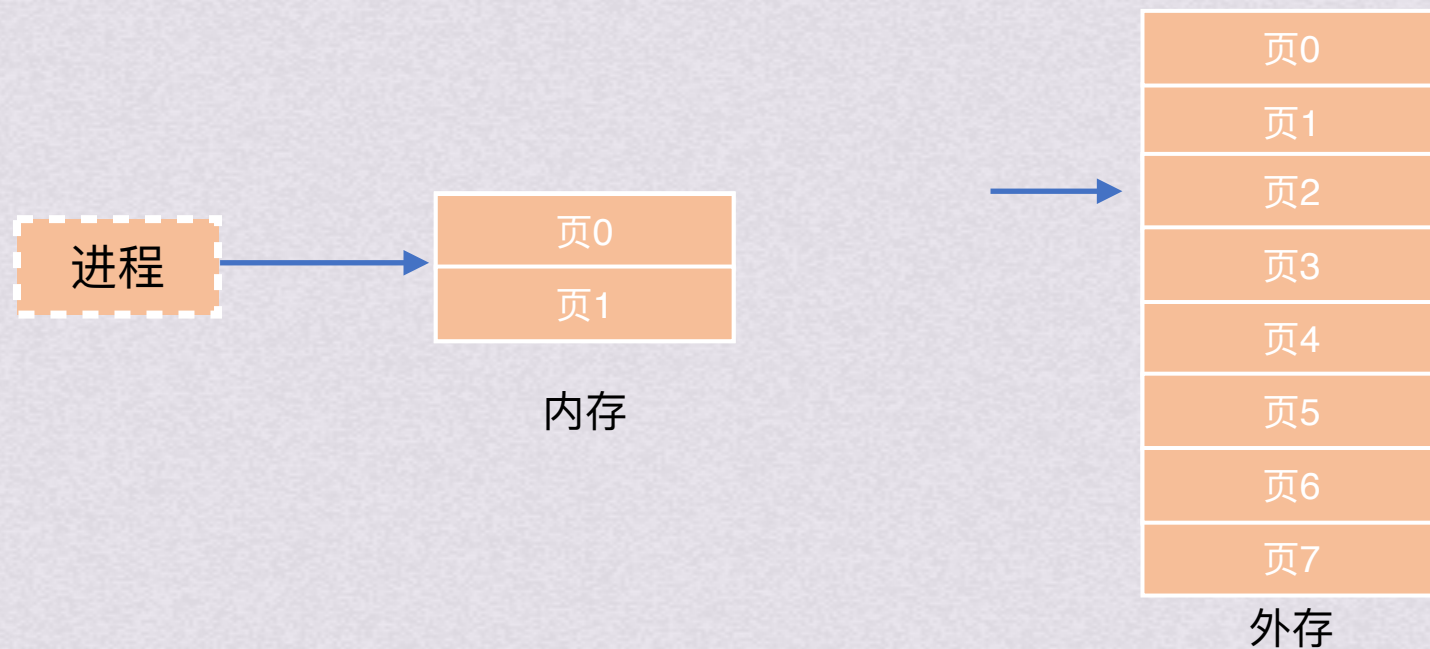
页面置换算法

抖动与工作集



抖动与工作集

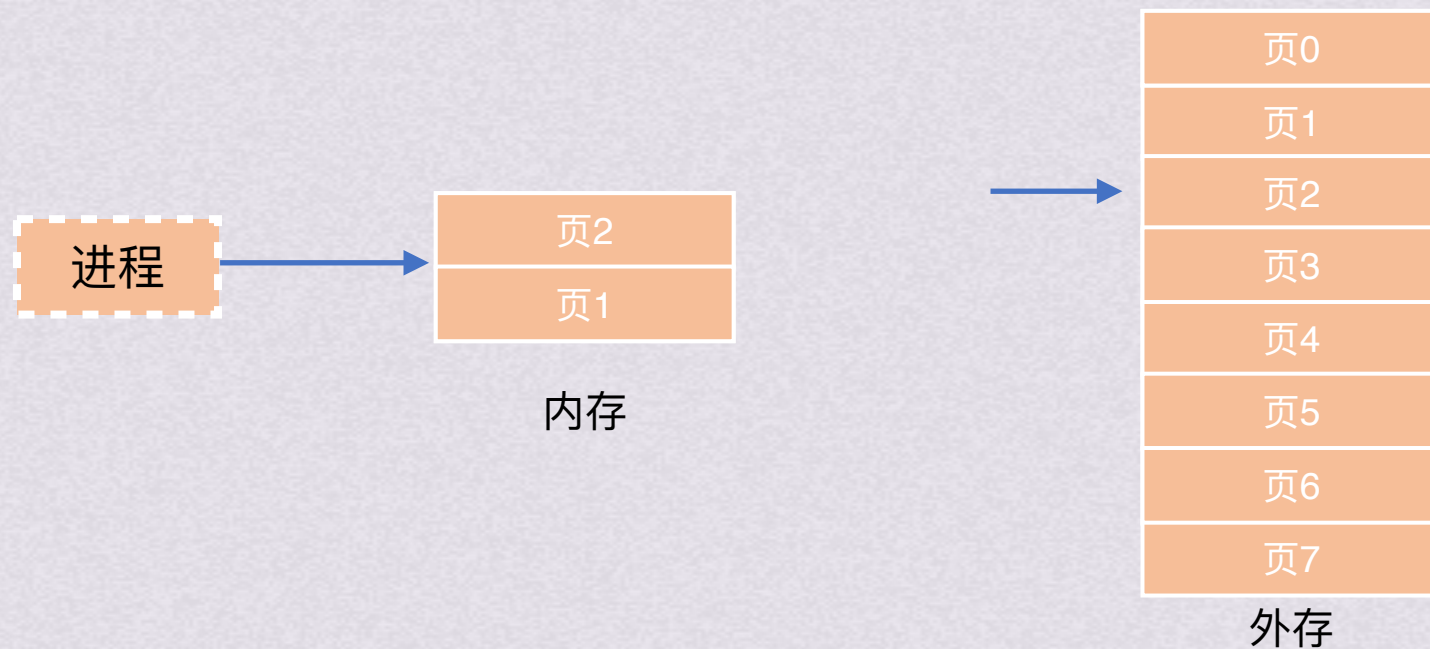
抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

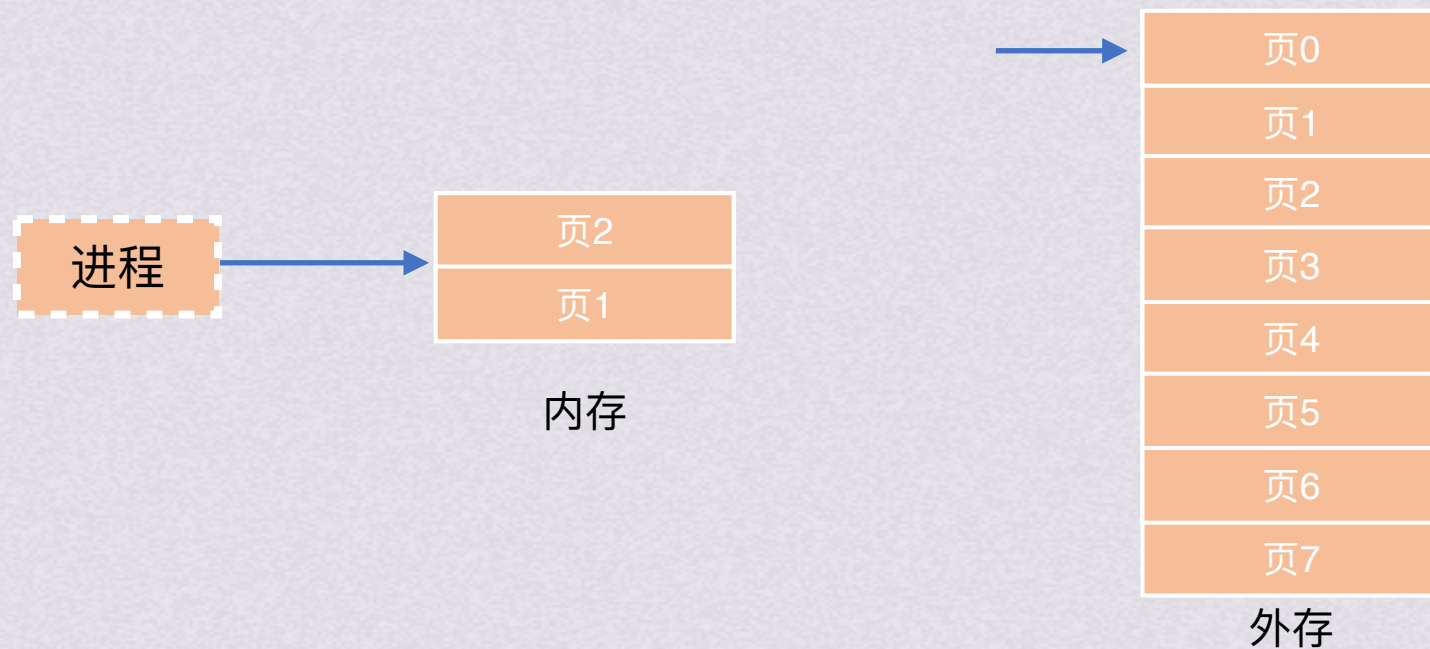
抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

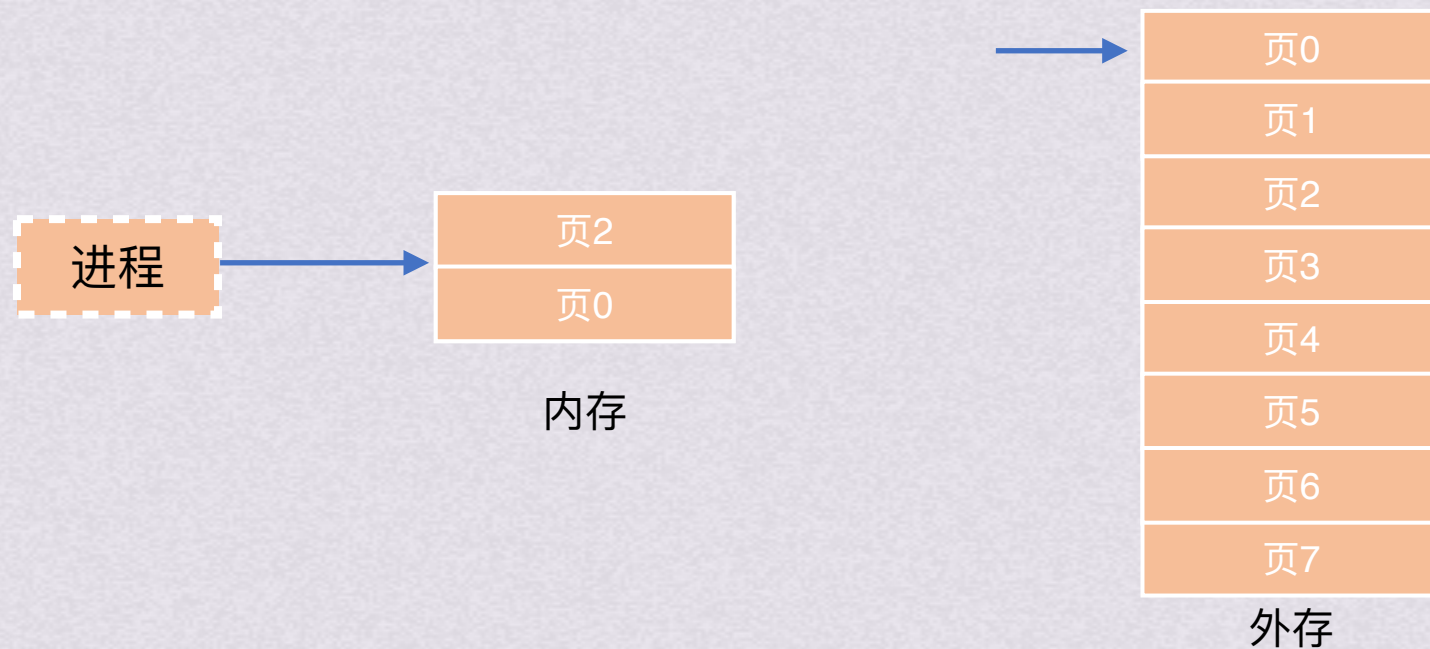
抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

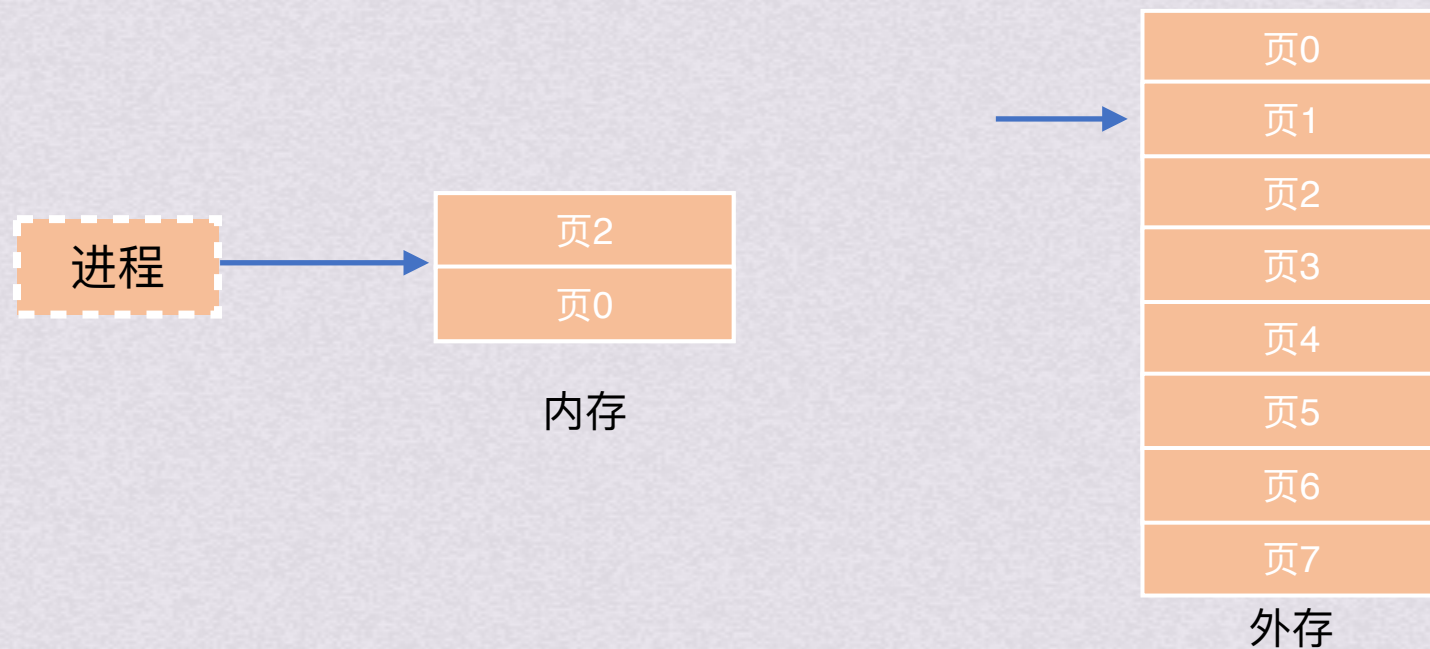
抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

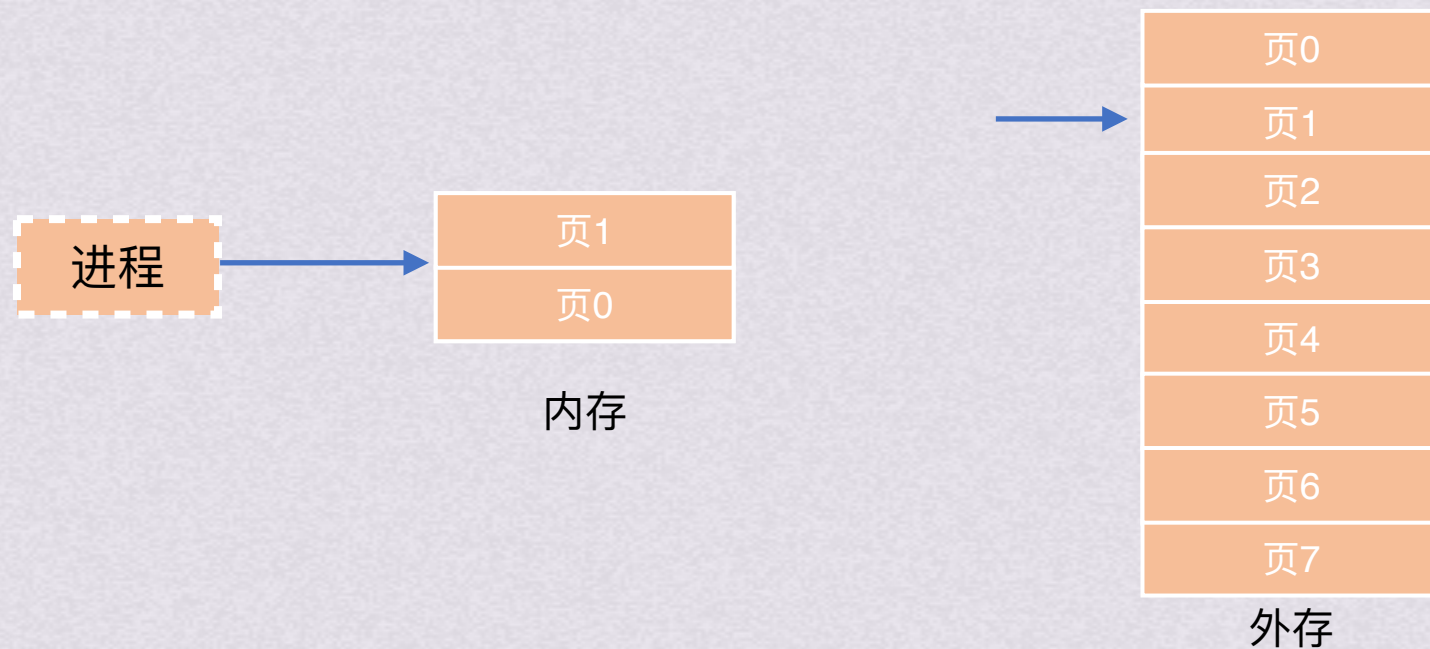
抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

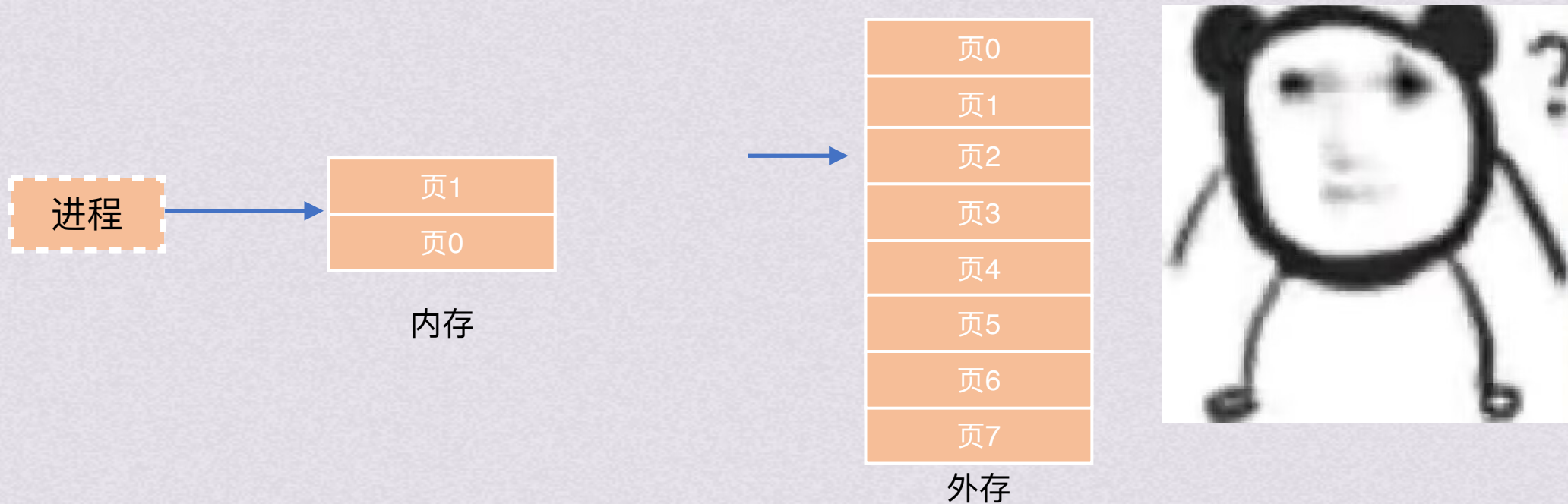
抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动





抖动与工作集

抖动：页面置换过程中，刚刚换出的页面马上又要换入内存，刚刚换入的页面马上又要换出内存，这种频繁的页面调度称抖动

抖动的根本原因是分配给每个进程的物理块太少，已经不能满足进程正常运行的基本要求，每个进程运行时频繁的页面调度导致CPU利用率急剧下降

因为抖动的发生与分配物理块的多少(即驻留集)有关，所以又提出了工作集的概念

工作集：某段时间间隔内，进程要访问的页面集合

某进程对页面访问次序如下：4,5,8,3,4,7,8,4,8,7,7,5,4,7,4,8

t1 t2

假设工作集窗口尺寸为3，t1时刻工作集为{4,3,8}，t2时刻工作集为{7,4,5}

假设工作集窗口尺寸为4，t1时刻工作集为{4,3,8,5}，t2时刻工作集为{7,4,5}

应用会把工作集窗口设置的很大，局部性好的程序工作集会比工作集窗口小很多

驻留集的大小不能小于工作集

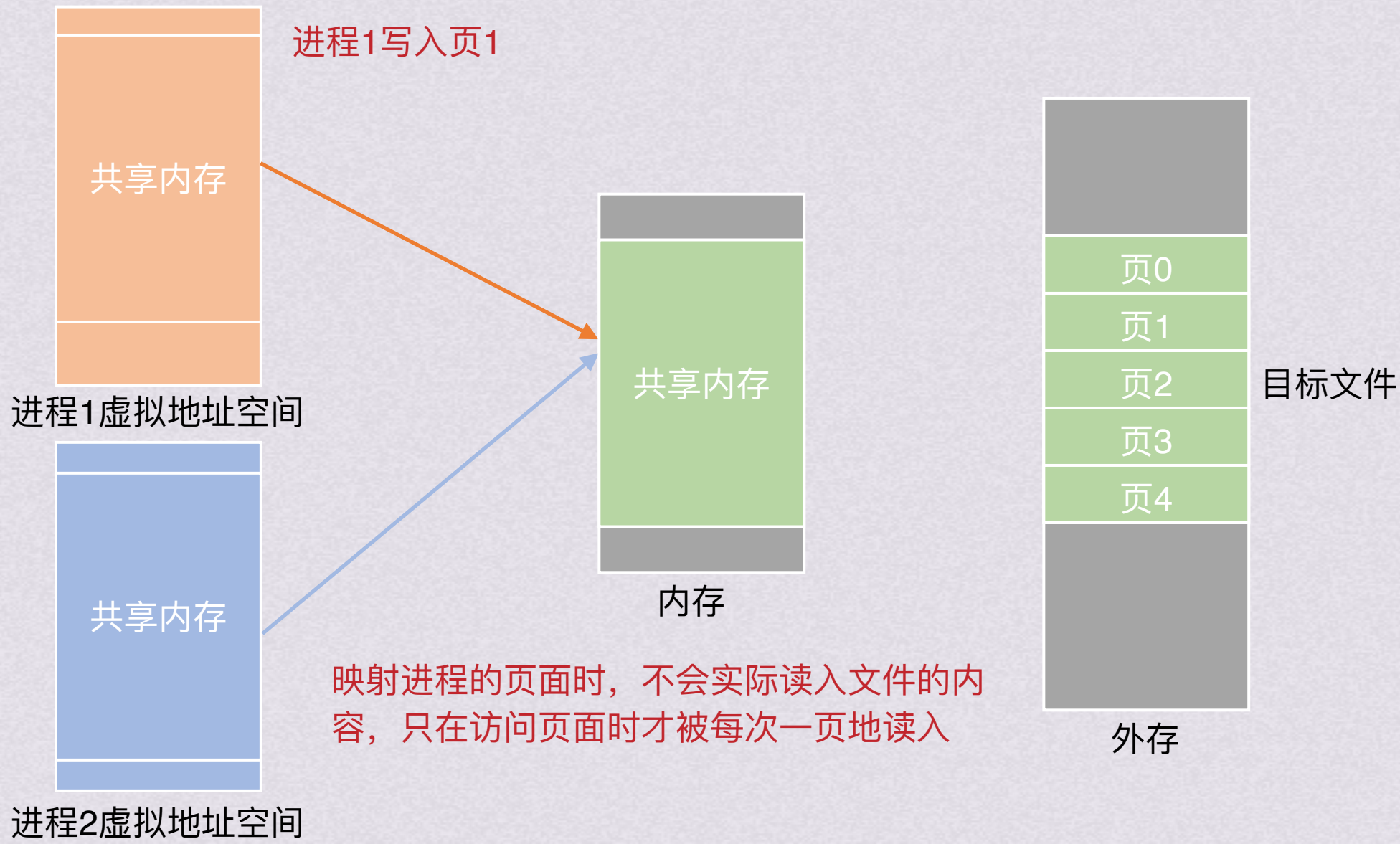
抖动与工作集

内存映射文件



内存映射文件

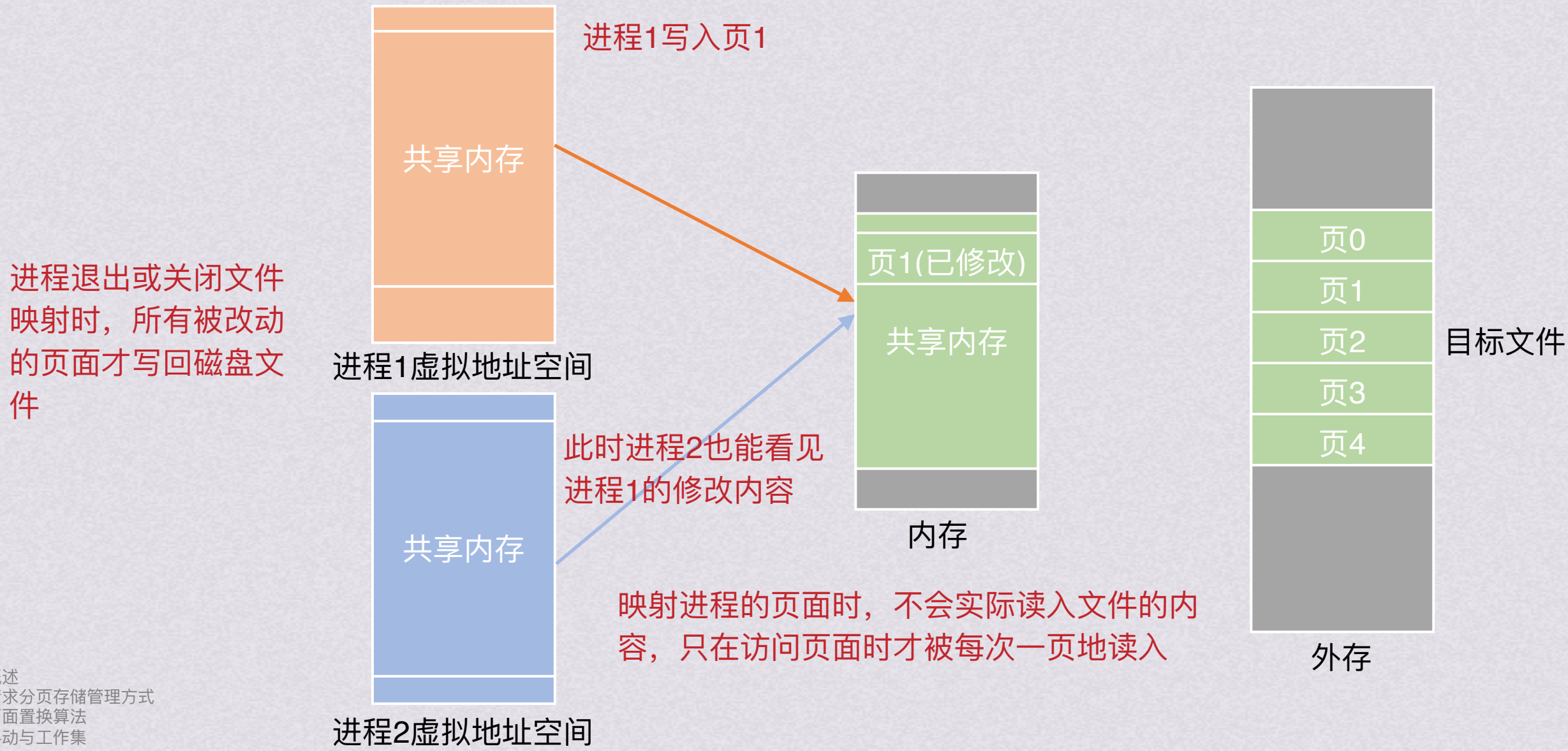
内存映射文件是操作系统向应用程序提供的一个系统调用，在磁盘文件和进程的虚拟地址空间之间建立映射关系





内存映射文件

内存映射文件是操作系统想应用程序提供的一个系统调用，在磁盘文件和进程的虚拟地址空间之间建立映射关系





内存映射文件

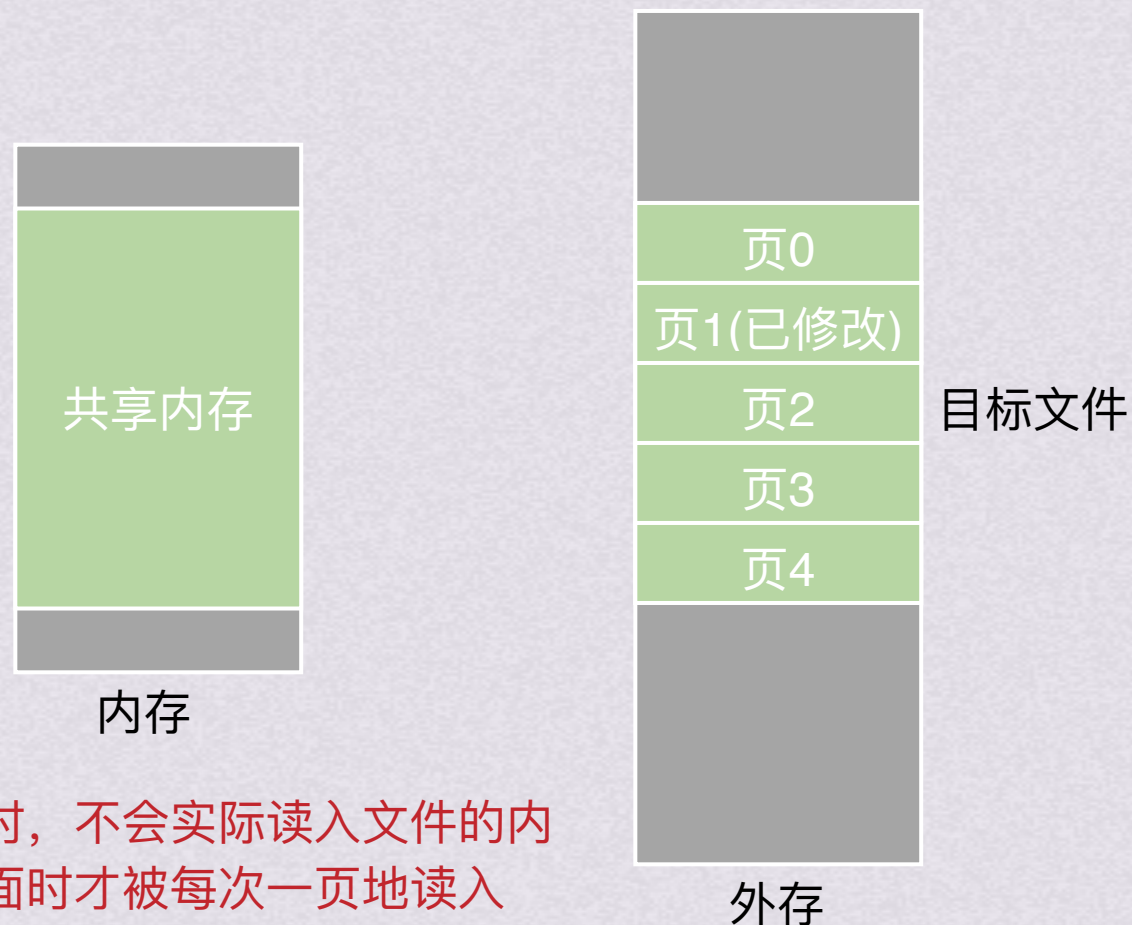
内存映射文件是操作系统想应用程序提供的一个系统调用，在磁盘文件和进程的虚拟地址空间之间建立映射关系

进程1写入页1

进程退出或关闭文件映射时，所有被改动的页面才写回磁盘文件

此时进程2也能看见进程1的修改内容

映射进程的页面时，不会实际读入文件的内容，只在访问页面时才被每次一页地读入





内存映射文件

内存映射文件是操作系统想应用程序提供的一个系统调用，在磁盘文件和进程的虚拟地址空间之间建立映射关系

所以内存映射文件是一种在内存中修改磁盘文件的方式

内存映射文件能让程序员的编程更简单，已建立映射的文件，只需按访问内存的方式进行读写

并且更方便让多个进程共享一个磁盘文件